MLFP mlfp01 — Textbook Chapter

- Module 1 — Data Pipelines and Visualisation Mastery with Python
  - Learning Outcomes
  - Prerequisites
  - How to Read This Chapter
- Lesson 1.1: Your First Data Exploration
  - Why This Matters
  - Core Concepts
    - FOUNDATIONS: What is a program?
    - FOUNDATIONS: Variables
    - FOUNDATIONS: The four types you will actually use in this lesson
    - FOUNDATIONS: Arithmetic
    - FOUNDATIONS: f-strings — the way you will build every output line
    - FOUNDATIONS: What is a DataFrame?
  - Mathematical Foundations
    - FOUNDATIONS: The three measures of central tendency
    - THEORY: Why the mean minimises squared error
    - FOUNDATIONS: Variance and standard deviation
    - ADVANCED: Robust alternatives to the mean and variance
  - The Kailash Engine: DataExplorer (first look)
  - Worked Example: Singapore Monthly Weather
    - Step 0: Set up the file
    - Step 1: Load the data
    - Step 2: Inspect the full shape
    - Step 3: Summary statistics via describe
    - Step 4: Find the hottest, coldest, and wettest months
    - Step 5: A formatted summary report
  - Try It Yourself
  - Cross-References
  - Reflection
    - Drill answers
- Lesson 1.2: Filtering and Transforming Data
  - Why This Matters
  - Core Concepts
    - FOUNDATIONS: Boolean logic
    - FOUNDATIONS: What `pl.col` actually is
    - FOUNDATIONS: `.filter` — keep rows where a Boolean is True
    - FOUNDATIONS: `.select` — pick columns
    - FOUNDATIONS: `.rename` — change column names
    - FOUNDATIONS: `.with_columns` — create new columns
    - FOUNDATIONS: `pl.when` — conditional column assignment
    - FOUNDATIONS: `.sort` — order rows
    - FOUNDATIONS: Method chaining
  - The Kailash Context
  - Worked Example: HDB Resale Flats
    - Step 1: Load the data
    - Step 2: Basic filters
    - Step 3: Combined filters
    - Step 4: Select and rename
    - Step 5: Derive new columns
    - Step 6: Conditional price tier
    - Step 7: Chain everything together
  - Try It Yourself
  - Cross-References
  - Reflection
    - Drill answers
- Lesson 1.3: Functions and Aggregation
  - Why This Matters
  - Core Concepts
    - FOUNDATIONS: What is a function?

        - FOUNDATIONS: Parameters and return values
        - THEORY: Type hints
        - FOUNDATIONS: `if` / `elif` / `else`
        - FOUNDATIONS: Lists, dictionaries, and loops
        - FOUNDATIONS: Iterating over DataFrame rows
        - FOUNDATIONS: `group_by` + `agg` — the most important pattern in data analysis
        - THEORY: What group_by does under the hood
    - The Kailash Context
    - Worked Example: District Statistics
        - Step 1: Set up and define helper functions
        - Step 2: The district statistics group_by
        - Step 3: Add derived columns to the aggregated table
        - Step 4: Multi-key group_by — (town, flat_type) combinations
        - Step 5: Iterate over the district report and format each line
        - Step 6: Cross-district summary
    - Try It Yourself
    - Cross-References
    - Reflection
        - Drill answers
- Lesson 1.4: Joins and Multi-Table Data
    - Why This Matters
    - Core Concepts
        - FOUNDATIONS: What is a join?
        - FOUNDATIONS: The four join types
        - FOUNDATIONS: Polars `.join` syntax
        - FOUNDATIONS: Always check the null count after a left join
        - FOUNDATIONS: Predicting nulls with set operations
        - FOUNDATIONS: `import` and packages
        - THEORY: Join as a set-theoretic operation
    - The Kailash Context
    - Worked Example: Enriching HDB with MRT and School Data
        - Step 1: Load three tables
        - Step 2: Inspect each table before joining
        - Step 3: Predict nulls
        - Step 4: The left join itself
        - Step 5: Pre-aggregate the schools table
        - Step 6: Fill the nulls
        - Step 7: Verify and inspect
        - Step 8: Build a district-level summary with spatial context
        - Step 9: Compute correlations between price and spatial features
    - Try It Yourself
    - Cross-References
    - Reflection
        - Drill answers
- Lesson 1.5: Window Functions and Trends
    - Why This Matters
    - Core Concepts
        - FOUNDATIONS: What is a window function?
        - FOUNDATIONS: Rolling averages — smoothing noisy time series
        - FOUNDATIONS: `shift` — compare to a previous row
        - FOUNDATIONS: `rank` — order within a group
        - FOUNDATIONS: Lazy frames — query optimisation
        - THEORY: When window functions fail
        - ADVANCED: Rolling windows are a form of convolution
    - The Kailash Context
    - Worked Example: HDB Price Trends by Town
        - Step 1: Prepare the time-series base table
        - Step 2: Rolling averages
        - Step 3: Year-over-year change
        - Step 4: Find the trend leaders with lazy evaluation
        - Step 5: Classify towns into leaders, followers, and laggards
    - Try It Yourself
    - Cross-References
    - Reflection

        - Drill answers
- Lesson 1.6: Data Visualisation
    - Why This Matters
    - Core Concepts
        - FOUNDATIONS: Why visualise?
        - FOUNDATIONS: Chart selection by data question
        - FOUNDATIONS: Histograms and distributions
        - FOUNDATIONS: Scatter plots and relationships
        - FOUNDATIONS: Bar charts for categorical comparison
        - FOUNDATIONS: Line charts for time series
        - FOUNDATIONS: Heatmaps for correlation
        - FOUNDATIONS: Gestalt principles
        - FOUNDATIONS: Charts to avoid
        - FOUNDATIONS: Z-pattern reading
    - The Kailash Engine: ModelVisualizer
    - Worked Example: Six HDB Charts
        - Step 0: Imports and data prep
        - Step 1: Histogram of resale prices
        - Step 2: Scatter plot of price vs floor area
        - Step 3: Bar chart of median price by town
        - Step 4: Correlation heatmap
        - Step 5: Line chart of annual median price
        - Step 6: All charts in a directory
    - Try It Yourself
    - Cross-References
    - Reflection
        - Drill answers
- Lesson 1.7: Automated Data Profiling
    - Why This Matters
    - Core Concepts
        - FOUNDATIONS: Why automate profiling?
        - FOUNDATIONS: The eight alert types
        - FOUNDATIONS: AlertConfig — tuning thresholds for your domain
        - FOUNDATIONS: `async` and `await` — just enough to use them
        - FOUNDATIONS: `try` / `except` — handling errors
        - THEORY: `DataExplorer.compare` — drift detection
        - FOUNDATIONS: Spearman vs Pearson correlation
    - The Kailash Engine: DataExplorer — full API
    - Worked Example: Profiling Singapore Economic Indicators
        - Step 1: Load three messy time series
        - Step 2: Normalise the date columns
        - Step 3: Build a common monthly spine and align everything
        - Step 4: Configure AlertConfig
        - Step 5: Run the profiler
        - Step 6: Compare two time periods
        - Step 7: Generate an HTML report
        - Step 8: Wrap everything in try/except
    - Try It Yourself
    - Cross-References
    - Reflection
        - Drill answers
- Lesson 1.8: Data Pipelines and End-to-End Project
    - Why This Matters
    - Core Concepts
        - FOUNDATIONS: The ETL pattern
        - FOUNDATIONS: Null handling
        - FOUNDATIONS: Domain-aware cleaning
        - FOUNDATIONS: Feature engineering for temporal data
        - FOUNDATIONS: Feature engineering for spatial data
        - FOUNDATIONS: PreprocessingPipeline
        - ADVANCED: Why standardise numeric features
    - Worked Example: Taxi Trip Cleaning Pipeline
        - Step 1: Load and inspect
        - Step 2: Profile the raw data

        - Step 3: Domain-aware cleaning
        - Step 4: Feature engineering
        - Step 5: PreprocessingPipeline
        - Step 6: Visualise key patterns
        - Step 7: Re-profile and compare
        - Step 8: Pipeline summary
    - Try It Yourself
    - Cross-References
    - Reflection
        - Drill answers
- Chapter Summary
    - The shape of what you learned
    - The four Polars patterns to internalise
    - The three Kailash engines and their contracts
    - What Module 2 builds on
    - What you should do before moving on
- Glossary
- Further Reading

Module 1 — Data Pipelines and Visualisation Mastery with Python

> *"Can you trust a number you didn't explore yourself?"*

This chapter is your entry point into the MLFP programme. It starts at zero — no assumed Python, no assumed statistics, no assumed data experience — and takes you to the point where you can run a complete exploratory data analysis pipeline on a real Singapore dataset using Kailash's data engines.

Everything you learn here will be used again. The Polars patterns in Lesson 1.2 will still be on your fingertips in Module 6 when you reshape transformer training logs. The `group_by` / `agg` muscle you build in Lesson 1.3 is the same muscle you'll use to compute per-cohort calibration in Module 5. The visualisation literacy from Lesson 1.6 is what will let you tell the difference between a broken model and a broken chart three modules from now. So do not rush. The chapter is long because the foundations matter.

Learning Outcomes

By the end of this chapter you will be able to:

- Read, write, and execute Python programs that use variables, data types, f-strings, functions, loops, conditionals, imports, and basic error handling — enough Python to be productive in any data-related task.
- Load tabular data from CSV, Parquet, and API sources into Polars DataFrames and inspect their shape, schema, and summary statistics.
- Filter, select, sort, transform, and aggregate datasets of half a million rows using Polars expressions and method chaining, without reaching for pandas.
- Write reusable helper functions that classify values, format numbers, and compute statistics, and apply them through `group_by` / `agg` pipelines.
- Join multiple tables on shared keys, reason about left vs inner vs outer joins, and handle the NULLs that arise after a join.
- Compute rolling averages, year-over-year changes, and rank within group using Polars window functions with `.over()` partitioning.
- Create appropriate, honest, interactive visualisations (histogram, scatter, bar, heatmap, line) using the ModelVisualizer engine with Plotly underneath, and critique charts against Gestalt and Z-pattern reading principles.
- Use the DataExplorer engine to profile a messy dataset automatically, configure AlertConfig thresholds to fit your domain, and interpret each of the eight alert types as a concrete cleaning action.
- Use PreprocessingPipeline to impute missing values, scale numeric columns, and encode categoricals, producing a train/test split that downstream modules can consume without further preparation.
- Assemble the above into a complete end-to-end pipeline that turns a raw, dirty dataset into a model-ready, auditable report.

Those are the concrete skills. Underneath them sits a more important outcome: you will have learned to *distrust* aggregated numbers you did not inspect yourself, and you will have the hand-tools to inspect them.

Prerequisites

Formally: none. This is Lesson zero of the MLFP programme and Lesson zero of your data career. Specifically, we do not assume:

- Prior Python experience (you will learn `print()` on page one).
- Prior statistics (we will define mean, median, variance, and correlation from scratch).
- Prior data experience (we will define what a DataFrame is before using one).
- Prior ML experience (ML does not appear in this chapter — we are building the foundation).

Practically, you will need:

- A computer with Python 3.11+ installed, or a Google account so you can run the exercises on Colab. No paid subscriptions are required.
- The `mlfp` repository checked out locally, or the module-1 exercise notebook on Colab.
- The ability to run one command in a terminal: `uv sync` (or `pip install -e .` if you prefer). If that sentence was terrifying, skip the local option and go straight to Colab — everything in this chapter runs there.
- A willingness to type code into a file or a cell and press Enter. You cannot learn programming by reading alone; every worked example in this chapter is designed for you to copy, run, and modify.

If you have seen pandas before, forget it deliberately. Do not translate Polars back into pandas in your head — the mental translation costs you more than the re-learning. Polars is close enough to pandas that the muscle memory transfers, and different enough that fighting it wastes hours.

How to Read This Chapter

This chapter has eight lessons that map one-to-one with the eight exercises in `modules/mlfp01/solutions/`. Each lesson follows the same structure:

1. **Why This Matters** — a Singapore-flavoured story that motivates the lesson. Skip if you are short on time, but the stories are often what the material sticks to in memory.
2. **Core Concepts** — plain-language explanations first, then formal definitions, then code examples, then a "common mistakes" sidebar.
3. **Mathematical Foundations** (where applicable) — the underlying mathematics with derivations. Marked THEORY.
4. **The Kailash Engine** — the engine that implements the lesson's concepts. DataExplorer, PreprocessingPipeline, or ModelVisualizer.
5. **Worked Example** — a complete, step-by-step walkthrough using a real Singapore dataset. Every line of code, every output, every interpretation.
6. **Try It Yourself** — three to five small drills. Attempt them before reading the answers at the end of each lesson.
7. **Cross-References** — how this lesson connects forward and backward.
8. **Reflection** — what you should now be able to do, and how to verify that.

Within each section, every non-trivial explanation is tagged with one of three depth markers:

Marker	Audience	How to Read It
FOUNDATIONS:	Zero background	Plain language, analogies, no derivations. Read every word.
THEORY:	Practitioner	Formal statement, derivation sketch, working knowledge required. Read if you want to reason about why things work.
ADVANCED:	Masters / researcher	Paper references, frontier results. Skim on first read, return later if interested.

If you are entirely new to the material, read only the FOUNDATIONS sections on your first pass through a lesson. You will finish Module 1 in roughly 20 hours of reading and 10 hours of exercises, and you will be productive. Come back to the THEORY sections once the FOUNDATIONS have settled — typically one to two weeks later. The ADVANCED material is there to make sure you are not bored if you already have a PhD in statistics.

Estimated reading time per lesson:

Lesson	Title	Reading	Exercise	Total
1.1	Your First Data Exploration	90 min	45 min	~2h
1.2	Filtering and Transforming Data	90 min	55 min	~2h 25m
1.3	Functions and Aggregation	100 min	60 min	~2h 40m
1.4	Joins and Multi-Table Data	100 min	60 min	~2h 40m
1.5	Window Functions and Trends	110 min	60 min	~2h 50m
1.6	Data Visualisation	100 min	60 min	~2h 40m
1.7	Automated Data Profiling	110 min	60 min	~2h 50m
1.8	Data Pipelines and End-to-End Project	120 min	75 min	~3h 15m

Total: roughly 20 hours of focused work. Spread it across two weeks at two hours per day and it is comfortable. Compress it to a weekend and you will forget half of it by Monday.

Lesson 1.1: Your First Data Exploration

Why This Matters

In 2023 a small anomaly appeared in the public HDB resale price dataset published by Singapore's Housing and Development Board. Prices in three mature estates — Queenstown, Toa Payoh, and Ang Mo Kio — showed a brief and puzzling dip. The monthly median, which is what every property dashboard in the country displays, barely moved. The distribution, which is what almost nobody displayed, had shifted from a single bell curve into two distinct humps: one at the usual price and one roughly thirty percent below it.

Nobody noticed for three weeks. Dashboards looked fine. The median was fine. Average prices were fine. The anomaly was only caught when a junior analyst opened the raw CSV, loaded it into a DataFrame, and plotted a histogram — one line of code. The dip turned out to be a batch of subsidised intra-family transfers that had been accidentally classified as open-market resales. For three weeks, anyone valuing a Queenstown flat using the public data — which includes banks, property agents, and homebuyers — was working with a number that was quietly wrong.

You will hear this story referred to in this chapter as the HDB Flash Crash. It is the reason the word *mastery* is in the module title. Mastery does not mean that you can write fancy code. It means that you can look at a file of raw data and find things that nobody asked you to look for. It means you do not trust an aggregated number you did not personally plot. It means you know the difference between the mean and the median well enough to explain, unprompted, why the Singapore property reports use one and not the other.

In this lesson you will learn the single skill that would have caught the flash crash on day one: load a CSV file into memory, look at the raw data, and compute summary statistics. We will do this on a smaller, friendlier dataset — Singapore monthly weather — so the stakes are low while you are learning the syntax. But by the end of Lesson 1.8, the tools in your hands will be the same tools that caught the real anomaly.

Core Concepts

FOUNDATIONS: What is a program?

A program is a file of text that tells a computer what to do in a language the computer can understand. You write the text. You run the program. The computer reads your text one line at a time and performs the actions described. If the text contains an instruction the computer does not understand, it stops and prints an error message. That is all a program is. There is no magic.

The language we will use is Python. It is called Python for no particularly important reason (the author liked Monty Python), but the syntax was deliberately designed to read almost like English. You will see the word `if` used for conditional decisions, the word `for` used for loops, and the word `return` used to send a value back from a function. This is a good thing. It means that you can often guess what a line of Python does on your first read, and be right more often than not.

A Python program is just a file with a `.py` extension. You can open it in any text editor, including the one that came with your operating system. The exercises in this course live in files like `ex_1.py`. Inside each file you will find lines of Python that the computer will execute from top to bottom when you run the file. If you prefer notebooks to files, the exercises also exist as `.ipynb` notebooks, which break the same code into runnable cells — but the code inside is identical. Pick the format that makes you comfortable and stick with it.

FOUNDATIONS: Variables

A variable is a name attached to a value. You create a variable by writing a name, an equals sign, and a value:

```
city = "Singapore"
```

After this line runs, the word `city` refers to the string `"Singapore"`. You can use the variable anywhere you would use the value itself. If you type `print(city)`, Python will print `Singapore`. If you later reassign the variable — `city = "Jakarta"` — the name now refers to a different value, and the previous one is discarded. Variables are not permanent containers. They are labels you can freely move around.

The name on the left of the `=` sign must follow a few rules. It must start with a letter or underscore. It must contain only letters, digits, and underscores. It cannot be a reserved Python word like `if`, `for`, or `return`. By convention we write variable names in lowercase with underscores between words, like `years_of_data` or `mean_price`. This is called *snake_case*. It is a convention, not a rule, but Python code that does not follow it looks foreign to everyone who reads it.

A variable has a *type* that is determined by the value you assigned to it, not by anything you wrote. Python figures out the type on its own. You do not have to say “this is a string” or “this is a number” — you just assign the value and Python remembers. This is called *dynamic typing*, and it is one of the reasons Python is pleasant to write in.

FOUNDATIONS: The four types you will actually use in this lesson

Python has many data types built in, but at the start you need only four:

- **`int`** — an integer. Whole numbers, positive or negative: `0`, `1`, `42`, `-3`. No decimal point.
- **`float`** — a floating-point number. Numbers with a decimal point: `1.35`, `-0.01`, `3.141592`. Python uses 64-bit IEEE 754 floats, which means you get about 15 decimal digits of precision. For most data work this is more than enough.
- **`str`** — a string. A sequence of text characters, written inside single or double quotes: `"Singapore"`, `'hello'`, `"123"`. Note that `"123"` is a string, not a number — the quotes make it text.
- **`bool`** — a Boolean. Exactly two possible values: `True` and `False`. The capitalisation matters. Booleans are what you get back from comparisons like `price > 500000`.

You can check the type of any variable by calling `type()` on it:

```
years_of_data = 30
print(type(years_of_data))   # <class 'int'>

latitude = 1.35
print(type(latitude))         # <class 'float'>

city = "Singapore"
print(type(city))             # <class 'str'>

is_tropical = True
print(type(is_tropical))      # <class 'bool'>
```

Each of those `print` calls writes a line to your terminal. The text in the comments (`#`) is not executed — it is for human readers.

> **Common mistake:** writing `True` as `true`, or `"Singapore"` without the quotes. Python is case-sensitive, and unquoted words are interpreted as variable names. `true` without quotes is an error because Python does not know about a variable called `true`. Similarly, `print(Singapore)` without quotes will raise `NameError: name 'Singapore' is not defined` — Python thinks you are asking it to look up a variable called `Singapore`, which does not exist.

FOUNDATIONS: Arithmetic

Python does arithmetic with the operators you would expect: `+`, `-`, `*` for multiplication, `/` for division. There are two less obvious ones you will meet immediately:

- `**` raises a number to a power. `2 ** 10` is `1024`.

- `%` is the *modulo* operator — it gives the remainder after division. `17 % 5` is `2`, because 17 divided by 5 is 3 remainder 2. Modulo is useful for asking "is this number divisible by that one?" — a test many data tasks need.

You can freely mix `int` and `float` in arithmetic; the result is a `float` if either operand was a `float`. So `27 + 0.5` is `27.5`, not `27`. Division with `/` always produces a `float` — even `10 / 2` is `5.0`, not `5`. If you want the integer result use `//` (integer division): `10 // 3` is `3`.

Here is a real calculation a property agent would do — the price per square metre of a flat:

```
resale_price = 485_000     # S$485,000
floor_area_sqm = 93
price_per_sqm = resale_price / floor_area_sqm
print(price_per_sqm)       # 5215.053763440861
```

Notice the underscore in `485_000`. Python lets you insert underscores inside numeric literals as visual separators. It treats `485_000` exactly like `485000` — the underscore is purely for human readability. For large numbers, always use it.

FOUNDATIONS: f-strings — the way you will build every output line

When you want to print text that includes the current value of a variable, the cleanest way in modern Python is an *f-string*. An f-string is a normal string literal with the letter `f` in front of it. Inside the string, anything between curly braces `{ }` is evaluated as Python and the result is inserted into the text:

```
price = 485_000
area = 93

print(f"Price: S${price}, Area: {area} sqm")
# Price: S$485000, Area: 93 sqm
```

That is already useful. But f-strings have a format-specifier syntax that makes them powerful for data reporting. After the variable name, you can write a colon and a format code:

```
price_per_sqm = price / area
print(f"Price per sqm: S${price_per_sqm:,.2f}")
# Price per sqm: S$5,215.05
```

The `:,.2f` is three instructions packed together: the `,` means "insert comma thousands separators", the `.2` means "round to two decimal places", the `f` means "display as a floating-point number". You can use any combination. Other common specifiers you will see throughout the course:

- `:.0f` — zero decimal places, floating-point. `f"{485000.7:.0f}"` gives `"485001"`.
- `:>10` — right-align in a field of width 10. Useful for lining up numbers in a printed table.
- `:<10` — left-align in a field of width 10. Useful for lining up labels in a printed table.
- `:.1%` — display as a percentage with one decimal place. `f"{0.234:.1%}"` gives `"23.4%"`. Note that `0.234` is interpreted as 23.4%, not 0.234%.

You will build every output line in this chapter with f-strings. Get comfortable with them now.

> **Common mistake:** forgetting the `f`. If you write `print("Price: S${price}")` with no `f`, Python will print the literal text `Price: S${price}` instead of substituting the variable. The `f` is the signal that turns a string into an f-string.

FOUNDATIONS: What is a DataFrame?

Up to this point, everything has been a single value. A variable holds one number or one string. Real data does not look like that. A dataset of Singapore weather has one row for each month and multiple columns — month name, mean temperature, rainfall, humidity. A dataset of HDB resale transactions has one row for each sale and dozens of columns — town, flat type, floor area, lease commencement, resale price. To work with data like this you need a structure that holds a two-dimensional table, and that structure is called a *DataFrame*.

A DataFrame is a rectangular table of data with named columns and rows. Each column has a type (string, integer, float, boolean, date). Each row is an observation. Think of it as a spreadsheet you can manipulate with code instead of a mouse. The DataFrame is the single most important object you will meet in this course. Every piece of data you work with will arrive as one, live inside one, or leave as one.

There are many DataFrame libraries in Python. You may have heard of pandas, the oldest and most widely used. In this course we use *Polars*. Polars is newer, written in Rust, uses less memory, and is considerably faster on large datasets — the 500,000-row HDB

dataset loads in under a second. More importantly for a learner, Polars has a cleaner, more consistent API than pandas, which means you will make fewer mistakes while you are still new to the vocabulary. Every exercise in MLFP uses Polars.

> **If you have used pandas before:** the mental translation table is short. `pd.read_csv` becomes `pl.read_csv`. `df.loc[df["town"] == "BISHAN"]` becomes `df.filter(pl.col("town") == "BISHAN")`. `df["price"].mean()` is the same in both. The notable differences are that Polars uses `pl.col("name")` to refer to a column inside an expression (you do not slice rows with bracket syntax), and Polars operations like `with_columns` return a new DataFrame rather than modifying in place. There is no `.iloc`, no `inplace=True`, and — blessedly — no `SettingWithCopyWarning`.

A DataFrame has two dimensions: height (number of rows) and width (number of columns). Polars calls these `df.height` and `df.width`, or you can ask for both at once as `df.shape`, which returns a tuple `(height, width)`. A *tuple* is like a list but cannot be modified after creation — you will see this structure often in Python.

Polars also stores *column names* (accessible as `df.columns`) and *column types* (accessible as `df.dtypes`). The column types are Polars types, not Python types: a text column is `pl.String` or `pl.Utf8`, an integer column is `pl.Int64`, a decimal column is `pl.Float64`, a boolean column is `pl.Boolean`, and so on. You do not need to memorise these — they appear in the output of `describe` and other inspection functions, and you can look them up when you see an unfamiliar one.

Mathematical Foundations

FOUNDATIONS: The three measures of central tendency

Once you have a column of numbers, the first question you ask is "what is a typical value?" There are three answers, each correct in different circumstances. The mathematics is trivial; the judgement about *when to use which* is not.

Mean. The arithmetic mean of a set of numbers is their sum divided by the count:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

In plain language: add them all up, divide by how many there were. The mean is the most commonly used average because it is easy to compute and has nice mathematical properties — most notably, it is the value m that minimises the sum of squared differences $\sum(x_i - m)^2$. That property will matter enormously when you meet linear regression in Module 2.

The downside of the mean is that it is *sensitive to outliers*. A single extreme value can drag it far from the typical case. If you record the prices of ten HDB flats and nine are around $400,000 and one is a penthouse at $2.5 million, the mean is $610,000 — a number that represents neither the ordinary flats nor the penthouse. It is in the gap between them, where no actual flat lives.

Median. The median is the middle value when you sort the numbers. If you have an odd number of values, it is literally the middle one. If you have an even number, it is the average of the two middle values. The median is *robust* — it ignores how extreme the outliers are, only where they are. In the ten-flat example above, the median is around $400,000, which is exactly what "a typical flat" should be.

Singapore property reports use the median for this reason. Even if a single billionaire buys a penthouse for $10 million, the median resale price does not budge. The mean would jump by $50,000 and every news outlet would run a headline about a "housing boom". The median is the honest number.

Mode. The mode is the most frequently occurring value. For continuous data like prices it is usually not useful — no two flats sell for exactly the same price. For categorical data like flat type (`3 ROOM`, `4 ROOM`, `5 ROOM`, `EXECUTIVE`), the mode tells you which category is most common, which is exactly what "typical" means for a category.

When to use which.

- **Symmetric distribution, no outliers:** mean and median agree; either is fine.
- **Skewed distribution (one long tail):** use the median. This is the default for incomes, prices, and almost any "money" variable.
- **Bimodal distribution (two peaks):** use neither; the single-number summary is misleading. Plot a histogram and describe the two groups separately.
- **Categorical data:** use the mode.

  Common mistake: reporting the mean of a skewed distribution as the "average" in a public-facing number. The word "average" in English is ambiguous — some listeners will hear "mean", some will hear "typical value". For skewed data those are different numbers, and using the wrong one is misleading. When in doubt, report the median and call it the median.

THEORY: Why the mean minimises squared error

Suppose you have numbers $x_1, \ldots, x_n$ and you want to pick a single value m that is "closest" to all of them. "Closest" needs a definition; let's use squared distance and add them up:

$$S(m) = \sum_{i=1}^{n} (x_i - m)^2$$

To find the m that minimises this, take the derivative with respect to m and set it to zero:

$$\frac{dS}{dm} = \sum_{i=1}^{n} -2(x_i - m) = -2 \sum_{i=1}^{n} (x_i - m) = 0$$

Divide by −2 and split the sum:

$$\sum_{i=1}^{n} x_i - \sum_{i=1}^{n} m = 0 \implies \sum x_i = nm \implies m = \frac{1}{n}\sum x_i = \bar{x}$$

So the mean is the answer. If instead you use absolute distance $|x_i - m|$ and minimise $\sum |x_i - m|$, the answer turns out to be the median — but the derivation involves sub-gradients because absolute value is not differentiable at zero, so we skip it here and return to it in Module 2 when we cover robust regression.

The takeaway: mean = minimises squared loss, median = minimises absolute loss. That pairing is the root of why linear regression (which minimises squared loss) is so sensitive to outliers, and why robust regression methods (which minimise absolute loss or variations thereof) are less so. You will meet these again.

FOUNDATIONS: Variance and standard deviation

The mean tells you where the centre of a distribution is. It says nothing about how spread out the values are. For spread, the standard tool is the *variance* (symbol σ^2, read "sigma squared"):

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n} (x_i - \bar{x})^2$$

In words: take each value's distance from the mean, square it, average the squared distances. Squaring makes every term positive (otherwise positive and negative deviations would cancel) and penalises large deviations more than small ones.

The variance is in squared units. If prices are in dollars, variance is in dollars-squared, which is not a unit anyone has intuition for. So we usually report the *standard deviation* σ — the square root of the variance — which is back in the original units:

$$\sigma = \sqrt{\sigma^2}$$

Rough rule of thumb: for roughly bell-shaped data, about 68% of values fall within one standard deviation of the mean, about 95% within two, and about 99.7% within three. Singapore temperatures across the year have a standard deviation of about 1°C — which means any month with a mean temperature more than 2°C from the annual average is a statistically unusual month. For HDB resale prices the standard deviation is around $150,000 — which tells you that prices are much more spread out than temperatures are, as you'd expect.

> **Pedantic footnote on n vs $n - 1$:** the formula above divides by n. Many statistics textbooks and libraries divide by $n - 1$ instead. The $n - 1$ version is the *sample variance*, which is an unbiased estimator of the *population variance* when you are treating your data as a sample from a larger population. The n version is the *maximum likelihood estimator* of the variance under a normal model. For a dataset with 500,000 HDB transactions, the difference is completely negligible — $n - 1$ and n are within 0.0002% of each other. For a dataset with 12 weather records, the difference is about 9%, which matters. Polars' `.var()` and `.std()` use $n - 1$ by default (Bessel's correction); `numpy.var` defaults to n. Know which one your library uses. We return to this in Module 2.

ADVANCED: Robust alternatives to the mean and variance

For heavy-tailed data the mean and variance both break. The standard robust replacements are:

- **Median** for central tendency. Breakdown point 50% — half your data must be contaminated before the median is moved far from the centre.
- **MAD** (Median Absolute Deviation) for spread: MAD = median($|x_i - \text{median}(x)|$). To get a drop-in replacement for the standard deviation, multiply by the constant 1.4826, which is chosen so that MAD and standard deviation agree for normally distributed data.
- **IQR** (Interquartile Range) — the difference between the 75th and 25th percentiles. Also a common spread measure for skewed data, and the basis for box-plot whiskers.

Robust statistics is a full subfield (see Huber, *Robust Statistics*, 1981). For Module 1 you only need to know they exist and when to reach for them.

The Kailash Engine: DataExplorer (first look)

This is the engine you will meet formally in Lesson 1.7. In Lesson 1.1 we use it only in its very simplest form — the `describe` method, which is really just a bridge to help you recognise that the per-column computations you just learned (mean, std, min, max) are all available in a single call.

DataExplorer is the Kailash ML engine for automated dataset profiling. It wraps a battery of column-level statistics and quality checks behind a single API. Its full capabilities include:

- Per-column type inference (numeric, categorical, temporal, text).
- Summary statistics (count, mean, std, min, max, quartiles, skewness, kurtosis).
- Missing-value counts and percentages.
- Duplicate-row detection.
- Pairwise correlations (Pearson and Spearman) with configurable thresholds.
- Alert generation for eight categories of data quality issues.
- Comparison of two datasets for distribution drift.
- HTML report generation.

You will use all of these in Lesson 1.7. For now, just know that the `describe` method you are about to see is the same thing DataExplorer does internally for its numeric columns. Learning the manual form first is deliberate — when DataExplorer flags something as “high skewness” in Lesson 1.7, you should be able to say “right, that’s the mean being different from the median because there’s a long tail, I remember that from Lesson 1.1” instead of being confused by a stranger.

Worked Example: Singapore Monthly Weather

We will work through a complete first exploration of a Singapore weather dataset. This dataset has roughly twelve rows (one per month) and three columns: `month`, `mean_temperature_c`, and `total_rainfall_mm`. It is small on purpose — every output fits on one screen, so you can see everything the code produces. Later lessons will use datasets with 500,000+ rows and you will be ready for them.

If you are running this locally, make sure you have run `uv sync` in the `mlfp` repository once. If you are running on Colab, open the Lesson 1 notebook. Either way, the code below should run as-is; the MLFPDataLoader knows how to find the CSV file in both environments.

Step 0: Set up the file

Create a new Python file called `lesson_1_1.py` anywhere convenient. At the top, add the imports you will need for the rest of the lesson:

```
from __future__ import annotations

import polars as pl

from shared import MLFPDataLoader
```

The first line, `from __future__ import annotations`, is a Python idiom that makes type hints a little more flexible. You do not need to understand it yet; just include it at the top of every file you write in this course.

The second line imports Polars and gives it the short alias `pl`. Every Polars expression in the course will start with `pl.something`. Typing `polars.something` every time would get tedious, so everyone uses the `pl` alias. This is a convention, not a rule, but it is so universal that you will confuse other readers if you break it.

The third line imports `MLFPDataLoader` from the `shared` helper module that comes with the course. This class knows how to find data files in every environment — local Python, Jupyter notebook, Google Colab — without you having to hardcode any file paths. You instantiate it once, and from then on you call `loader.load("mlfp01", "filename.csv")`.

Step 1: Load the data

```
loader = MLFPDataLoader()
df = loader.load("mlfp01", "sg_weather.csv")
```

```
print("Data loaded.")
print(df.head(5))
```

When you run this, you should see output something like:

```
Data loaded.
shape: (5, 3)
```

month	mean_temperature_c	total_rainfall_mm
str	f64	f64
January	26.5	242.4
February	27.1	161.3
March	27.8	185.8
April	28.3	179.5
May	28.5	171.0

(The exact numbers may differ slightly depending on the dataset version; that is fine.)

Read the output carefully. At the top, `shape: (5, 3)` tells you that the snippet you printed has 5 rows and 3 columns — that's `head(5)` at work, not the whole dataset. Below that, Polars prints the column names, then the column types on a separate row (`str` for text, `f64` for 64-bit floating-point), then the first five rows of data. The Unicode box-drawing characters are Polars' way of making the table readable; they have no meaning beyond that.

The data tells a story already. Mean temperatures in Singapore hover between 26 and 29°C — the equatorial climate that makes people who grew up elsewhere sweat through their first week. Rainfall is in the 150–250 mm range, with January showing the highest value in this snippet (we only looked at five months, so we can't yet say January is the wettest overall).

Step 2: Inspect the full shape

```
rows, cols = df.shape
print(f"Rows: {rows}")
print(f"Columns: {cols}")

print("\nColumn names:")
for col_name in df.columns:
    print(f"  - {col_name}")

print("\nColumn types:")
for col_name, dtype in zip(df.columns, df.dtypes):
    print(f"  {col_name}: {dtype}")
```

Two Python concepts appear here that we should name explicitly. First, *tuple unpacking*: the line `rows, cols = df.shape` takes the two-element tuple returned by `df.shape` and assigns the first element to `rows` and the second to `cols`. This is much cleaner than writing `rows = df.shape[0]; cols = df.shape[1]`. Polars, like most Python libraries, follows the convention that shape tuples are `(height, width)`, i.e. rows first then columns.

Second, *for loops*. The `for col_name in df.columns:` line says "for each item in the list `df.columns`, call it `col_name` and run the indented block below". The `df.columns` attribute returns a plain Python list of strings — the column names — so the loop iterates once per column. Inside the loop we print the name. Then `zip(df.columns, df.dtypes)` pairs up the columns list with the types list element-by-element, giving us tuples like `("month", String)`, `("mean_temperature_c", Float64)`, and so on. The second loop unpacks each tuple in the `for` statement itself: `for col_name, dtype in zip(...)`.

You will see both of these patterns — unpacking and `zip` — constantly. They are worth learning once.

Expected output:

```
Rows: 12
Columns: 3

Column names:
  - month
  - mean_temperature_c
  - total_rainfall_mm

Column types:
  month: String
```

```
  mean_temperature_c: Float64
  total_rainfall_mm: Float64
```

Twelve rows, exactly one per calendar month. Good: this is what you would expect from a monthly weather dataset. If you saw 11 rows you would immediately know something was missing. The types are as expected — month is text, temperature and rainfall are decimal numbers.

Step 3: Summary statistics via describe

```
print(df.describe())
```

Polars' `.describe()` computes count, null count, mean, standard deviation, min, 25th percentile, 50th percentile (median), 75th percentile, and max for every column. For string columns it computes count, nulls, and nothing else (because mean of strings is meaningless). The output looks like this:

```
shape: (9, 4)
```

statistic --- str	month --- str	mean_temperature_c --- f64	total_rainfall_mm --- f64
count	12	12.0	12.0
null_count	0	0.0	0.0
mean	null	27.575	178.483
std	null	0.843	43.27
min	January	26.2	112.5
25%	null	26.95	147.3
50%	null	27.65	175.0
75%	null	28.3	204.2
max	September	28.7	258.8

This is a staggering amount of information in one call. Let's read it.

Look at `mean_temperature_c` first. Mean 27.58°C, standard deviation 0.84°C, min 26.2°C, max 28.7°C. A standard deviation of less than 1°C over the whole year tells you Singapore's climate is extraordinarily stable compared to temperate zones. For comparison, London's monthly mean temperatures range from about 5 to 19°C — a standard deviation closer to 5°C. Singapore is six times more stable month-to-month.

Look at `total_rainfall_mm`. Mean 178 mm per month, standard deviation 43 mm, min 113 mm, max 259 mm. The range is more than twice the minimum — monthly rainfall varies a lot. Singapore has a wet season and a dry season even though the temperature barely changes.

Note that the `min` and `max` rows for the `month` column show `"January"` and `"September"`. Polars sorts strings alphabetically by default, so these are simply the alphabetically-first and alphabetically-last month names, not the months with minimum or maximum values of anything. This is a minor trap: `.describe()` applies its aggregations column-by-column without considering that you probably wanted "which month was coldest", not "which month comes first in the alphabet". We will compute the actually-hottest month in Step 4.

Step 4: Find the hottest, coldest, and wettest months

To answer "which month was hottest?" you have to filter the DataFrame to keep only the row where `mean_temperature_c` equals its maximum, then read off the month name. Here is the Polars idiom:

```
max_temp = df["mean_temperature_c"].max()
hottest_row = df.filter(pl.col("mean_temperature_c") == max_temp)
print(hottest_row)
```

Two things to unpack. `df["mean_temperature_c"]` is a Polars Series — a single column extracted from the DataFrame. You can call aggregation methods like `.max()`, `.min()`, `.mean()`, `.std()` directly on a Series, and they return a single scalar value. So `max_temp` after that line is a Python float, the maximum temperature in the dataset.

`df.filter(pl.col("mean_temperature_c") == max_temp)` is the filtering expression. `pl.col("mean_temperature_c")` is the Polars way to refer to a column inside an expression — you are saying "the column called mean_temperature_c". The == compares that column to the scalar `max_temp`, producing a Boolean column (True where the temperature equals the max, False everywhere else). `df.filter(...)` keeps only the rows where the Boolean column is True. The result is a DataFrame containing exactly the row(s) with the maximum temperature.

To get just the month name out of that result, index into the `month` column with `[0]`:

```
hottest_month = hottest_row["month"][0]
hottest_temp = hottest_row["mean_temperature_c"][0]
print(f"Hottest month: {hottest_month} at {hottest_temp:.1f}°C")
```

Expected output:

```
Hottest month: May at 28.7°C
```

The `[0]` is an index into the single-row DataFrame, returning the first (and in this case only) element. Polars Series support Python-style indexing; `series[0]` gives you the first value, `series[-1]` gives you the last.

Repeat the pattern for coldest and wettest:

```
min_temp = df["mean_temperature_c"].min()
coldest_row = df.filter(pl.col("mean_temperature_c") == min_temp)
coldest_month = coldest_row["month"][0]
coldest_temp = coldest_row["mean_temperature_c"][0]
print(f"Coldest month: {coldest_month} at {coldest_temp:.1f}°C")

max_rain = df["total_rainfall_mm"].max()
wettest_row = df.filter(pl.col("total_rainfall_mm") == max_rain)
wettest_month = wettest_row["month"][0]
wettest_rain = wettest_row["total_rainfall_mm"][0]
print(f"Wettest month: {wettest_month} with {wettest_rain:.1f} mm of rain")
```

Expected output (actual values may vary slightly with the version of the dataset):

```
Coldest month: December at 26.2°C
Wettest month: December with 258.8 mm of rain
```

December is both the coldest and the wettest month. Anyone who has spent a Christmas in Singapore will recognise this immediately — the northeast monsoon brings heavy rain and ever-so-slightly cooler weather from November to January.

Step 5: A formatted summary report

The last step is to collect everything into a human-readable report. This is the output a colleague would see. The goal is something you would be comfortable pasting into a Slack channel or an email.

```
mean_temp = df["mean_temperature_c"].mean()
std_temp = df["mean_temperature_c"].std()
mean_rain = df["total_rainfall_mm"].mean()

separator = "=" * 58

print(f"\n{separator}")
print(f"  SINGAPORE WEATHER SUMMARY")
print(f"{separator}")
print(f"  Total records:   {rows:>6,}")
print(f"  Columns:         {cols:>6}")
print(f"")
print(f"  Temperature (°C)")
print(f"    Mean: {mean_temp:>8.2f}")
print(f"    Std:  {std_temp:>8.2f}")
print(f"    Min:  {min_temp:>8.2f}  ({coldest_month})")
print(f"    Max:  {max_temp:>8.2f}  ({hottest_month})")
print(f"")
print(f"  Rainfall (mm/month)")
print(f"    Mean: {mean_rain:>8.1f}")
print(f"    Max:  {max_rain:>8.1f}  ({wettest_month})")
print(f"{separator}")
```

Output:

```
==========================================================
  SINGAPORE WEATHER SUMMARY
==========================================================
  Total records:       12
  Columns:              3
```

```
  Temperature (°C)
    Mean:    27.58
    Std:      0.84
    Min:     26.20  (December)
    Max:     28.70  (May)

  Rainfall (mm/month)
    Mean:   178.48
    Max:    258.80  (December)
==========================================================
```

Two details worth noting. `"=" * 58` is Python string multiplication — it produces a string of 58 copies of the = character, giving you a horizontal separator line without having to type it out. You will use this trick for every report you print.

The `:>8.2f` format specifier is what aligns the numbers. `>8` means "right-align in a field eight characters wide", and `.2f` means "two decimal places, float". Right-alignment with a fixed width is what makes numeric columns line up cleanly. Without it, `26.20` and `258.80` would start at different horizontal positions and the report would look messy.

And that is Lesson 1.1 worked end to end. You loaded a real dataset, inspected its shape and schema, computed summary statistics both through `.describe()` and through individual column aggregations, filtered to find extreme values, and built a formatted report. Every pattern you just learned will be used again in every subsequent lesson in this chapter.

Try It Yourself

Before moving to Lesson 1.2, try these five drills. Attempt each one before looking at the answers (which are at the end of this lesson). Resist the urge to copy — typing it yourself is how the muscle memory forms.

Drill 1. Write Python that assigns the string `"Orchard Road"` to a variable called `address`, the integer `1998` to a variable called `year_built`, and the float `93.5` to a variable called `area_sqm`. Then print them in a single f-string on one line, formatted as `Orchard Road (built 1998, 93.5 sqm)`.

Drill 2. Using the weather DataFrame loaded in Step 1, compute and print the *range* of monthly rainfall — that is, the difference between the maximum and the minimum. Use f-string formatting to display the result to one decimal place with units.

Drill 3. Modify the extreme-finding pattern from Step 4 to find and print the month with the *lowest* rainfall in a formatted line that looks like `Driest month: February with 112.5 mm`.

Drill 4. What is the coefficient of variation (CV) of monthly rainfall? Recall that CV is defined as σ/μ, where σ is the standard deviation and μ is the mean. Report the result as a percentage with one decimal place. Is the CV higher or lower than that of temperature? What does that tell you about the relative variability of rainfall and temperature?

Drill 5. Without using `.filter()`, compute the mean temperature of the first six months of the year and the mean temperature of the last six months. Hint: Polars Series support slicing with `df["mean_temperature_c"][:6]` for the first six elements and `df["mean_temperature_c"][6:]` for everything from index 6 onwards.

Cross-References

- **Lesson 1.2** will pick up from here and teach you to filter by more complex conditions (price ranges, multiple towns, date ranges) and to create new columns from existing ones. The `pl.col()` expression you just met will be everywhere.
- **Lesson 1.6** will visualise the summary statistics you computed in this lesson as histograms, line charts, and heatmaps. The intuition for "skewed vs symmetric" you just built will become visible.
- **Lesson 1.7** will replace much of what you did here with a single call to `DataExplorer.profile()`, but will expect you to understand the per-column statistics — because DataExplorer generates alerts based on them, and alerts are only useful if you can read the underlying numbers.
- **Module 2, Lesson 2.1** will re-derive the mean and variance as maximum-likelihood estimators of a normal distribution. You will see the same formulas from a different angle: not as "what's the average?" but as "what parameter best explains the observed data?".

Reflection

You should now be able to, without looking anything up:

- Explain what a variable is and what the four basic Python types are.

- Write an f-string that embeds a variable with a specific number format.
- Load a CSV into a Polars DataFrame using `MLFPDataLoader`.
- Call `.shape`, `.columns`, `.dtypes`, `.head()`, and `.describe()` on a DataFrame and describe what each returns.
- Extract a single column as a Series with `df["column_name"]` and call `.mean()`, `.std()`, `.min()`, `.max()` on it.
- Use `.filter(pl.col("column") == value)` to find the row(s) with a specific value.
- Explain the difference between mean and median and name one situation where each is appropriate.
- State, roughly, what fraction of values in a bell-shaped distribution fall within one and two standard deviations of the mean.

If any of those feels shaky, re-read the corresponding section before moving on. Lesson 1.2 assumes all of this is solid.

Drill answers

1. ```
   address = "Orchard Road"
   year_built = 1998
   area_sqm = 93.5
   print(f"{address} (built {year_built}, {area_sqm} sqm)")
   ```

2. ```
   rainfall_range = df["total_rainfall_mm"].max() - df["total_rainfall_mm"].min()
   print(f"Rainfall range: {rainfall_range:.1f} mm")
   ```

3. ```
   min_rain = df["total_rainfall_mm"].min()
   driest = df.filter(pl.col("total_rainfall_mm") == min_rain)
   print(f"Driest month: {driest['month'][0]} with {driest['total_rainfall_mm'][0]:.1f} mm")
   ```

4. ```
   cv_rain = df["total_rainfall_mm"].std() / df["total_rainfall_mm"].mean()
   cv_temp = df["mean_temperature_c"].std() / df["mean_temperature_c"].mean()
   print(f"Rainfall CV: {cv_rain:.1%}")
   print(f"Temperature CV: {cv_temp:.1%}")
   ```

   You should see rainfall CV near 24% and temperature CV near 3%. Rainfall is far more variable than temperature in Singapore — about 8× more variable on a relative basis. This matches intuition: the thermometer barely moves all year but the sky goes from clear to monsoon within days.

5. ```
   first_half = df["mean_temperature_c"][:6].mean()
   second_half = df["mean_temperature_c"][6:].mean()
   print(f"Jan–Jun mean: {first_half:.2f}°C")
   print(f"Jul–Dec mean: {second_half:.2f}°C")
   ```

   The first half is slightly warmer, because Singapore's pre-monsoon months (April–June) are the hottest of the year and the northeast monsoon (November–January) is the coolest.

Lesson 1.2: Filtering and Transforming Data

Why This Matters

The weather dataset in Lesson 1.1 had twelve rows. You could have inspected it with your eyes, no code required. The dataset in this lesson has five hundred thousand rows: every HDB resale transaction in Singapore over the last decade. You cannot scroll through five hundred thousand rows. If you try, you will miss every pattern of interest. The only way to work with data of this size is to let the computer filter, sort, and transform it while you ask increasingly specific questions.

That is what this lesson is about: asking questions of a dataset that is too large to hold in your head. "Show me only Ang Mo Kio flats." "Of those, show me only the ones between three hundred thousand and five hundred thousand dollars." "Of those, show me only recent transactions and sort them by price." Each of those questions is a filter, a sort, or a transformation — and the cleanest way to stack them together is Polars' method chaining syntax, which you will meet in Step 5 of the worked example.

The habit we are building here is not syntactic. It is interrogative. When you open a new dataset, you should automatically begin asking it questions. What are the extreme values? Which subset looks different from the rest? What does the distribution look like when I cut it by category? The Polars syntax is just the keyboard shortcut for the question. Over time, the questions become automatic; the syntax is the glue that makes them cheap to ask.

Core Concepts
   ```

FOUNDATIONS: Boolean logic

A Boolean is a value that is either True or False. Every filter you write in this lesson produces a column of Booleans, and the filter operation keeps the rows where that column is True. Before you can filter, you need to know how to write Boolean expressions.

The six *comparison operators* in Python are:

Operator	Meaning	Example	Result
`==`	equal to	`price == 500_000`	True if price is exactly 500,000
`!=`	not equal to	`town != "BISHAN"`	True if town is anything but BISHAN
`>`	greater than	`price > 500_000`	True if price is strictly above 500,000
`<`	less than	`price < 500_000`	True if price is strictly below 500,000
`>=`	greater than or equal to	`price >= 500_000`	True if price is 500,000 or more
`<=`	less than or equal to	`price <= 500_000`	True if price is 500,000 or less

Note the double equals sign == for equality. A single = means assignment ("store this value in that variable"), which is a totally different operation. Writing `if price = 500_000:` is a syntax error in Python, and that is by design — it prevents the silent bug of accidentally reassigning inside an `if` statement that you meant to be a comparison.

Once you have Boolean values you can combine them with the three logical operators:

Operator	Meaning	Example	Result
`&`	AND	`(price > 300_000) & (price < 500_000)`	True only if both sides are True
`|`	OR	`(town == "BISHAN") | (town == "TOA PAYOH")`	True if either side is True
`~`	NOT	`~(town == "BISHAN")`	True if the expression inside is False

Warning about parentheses. Polars uses `&`, `|`, and `~` for element-wise Boolean logic on columns. These are the same symbols Python uses for bitwise operations on integers, and they have a different operator precedence from the English `and`, `or`, and `not` keywords. This means you *must* wrap each side of `&` or `|` in parentheses when combining comparisons:

```
# CORRECT:
df.filter((pl.col("price") > 300_000) & (pl.col("price") < 500_000))

# WRONG — will raise an obscure error:
df.filter(pl.col("price") > 300_000 & pl.col("price") < 500_000)
```

The wrong version is parsed as `pl.col("price") > (300_000 & pl.col("price")) < 500_000`, because `&` binds tighter than `>` in Python. The resulting error is confusing. The rule is simple: every comparison goes in its own pair of parentheses.

> **Forward reference:** you are writing Boolean expressions inside Polars `.filter()` calls. Python also has `if` / `elif` / `else` statements for branching at the level of your code — deciding whether to run one block or another based on a condition. You will meet those in Lesson 1.4. The two are different: Polars filters operate on an entire column of data at once, while `if` statements operate on a single value. Do not mix them up.

FOUNDATIONS: What `pl.col` actually is

`pl.col("town")` is an *expression*. It is not the column itself — it is a promise to look up the column called `"town"` when Polars eventually evaluates the expression. Expressions are first-class objects in Polars: you can build them up, combine them, and pass them around before they are ever applied to a DataFrame.

This is useful because it means the same expression can be used in many places. `pl.col("price").mean()` is an expression that, when evaluated against a DataFrame, returns the mean of the `price` column. You can use that expression inside `.filter()`, inside `.with_columns()`, inside `.group_by().agg()`, and so on. The expression doesn't care where it is used.

The practical consequence: you will see `pl.col("something")` appear hundreds of times in this chapter. Every time, it is the same idea — a reference to a column. Do not read it as "fetch the column now". Read it as "when the time comes, look up this column".

FOUNDATIONS: `.filter` — keep rows where a Boolean is True

The `.filter()` method takes a Boolean expression and returns a new DataFrame containing only the rows where the expression is True. It does not modify the original DataFrame. Polars operations are almost always *immutable*: you get a new DataFrame back and the old one is unchanged. This is a deliberate design choice — it eliminates an entire class of bugs where you accidentally mutate data that another part of your code was relying on.

```
ang_mo_kio = hdb.filter(pl.col("town") == "ANG MO KIO")
```

After this line, `hdb` still contains all 500,000 rows of the original dataset. `ang_mo_kio` is a new DataFrame containing only the Ang Mo Kio rows — perhaps 20,000 of them.

You can combine filters in three ways:

1. **Inside a single filter call with `&` / `|`:**

   ```
   hdb.filter((pl.col("town") == "ANG MO KIO") & (pl.col("resale_price") < 500_000))
   ```

2. **By chaining multiple filter calls:**

   ```
   hdb.filter(pl.col("town") == "ANG MO KIO").filter(pl.col("resale_price") < 500_000)
   ```

   These two are semantically equivalent. Polars optimises both into the same query plan.

3. **With `.is_in()` for "one of several values":**

   ```
   central_towns = ["BISHAN", "TOA PAYOH", "QUEENSTOWN", "BUKIT MERAH"]
   hdb.filter(pl.col("town").is_in(central_towns))
   ```

   This is much cleaner than writing `(pl.col("town") == "BISHAN") | (pl.col("town") == "TOA PAYOH") | ...`.

There are also Polars convenience methods for common patterns: `.is_null()` for missing values, `.is_not_null()` for non-missing values, `.str.contains("text")` for substring matching on string columns, `.is_between(low, high)` for closed intervals. You do not need to memorise them — they all start with `pl.col(...)` and read as English-like method chains, so you can often guess the name and look up any that don't work.

FOUNDATIONS: `.select` — pick columns

`.filter()` is for rows. `.select()` is for columns. It takes the names of the columns you want to keep and returns a new DataFrame with only those columns:

```
core_cols = hdb.select("month", "town", "flat_type", "floor_area_sqm", "resale_price")
```

The original `hdb` DataFrame may have twelve columns; `core_cols` has exactly five. Everything else is dropped. This matters for two reasons. First, it reduces memory: the 500,000-row dataset takes far less space when it only has five columns instead of twelve. Second, it reduces visual clutter: when you print a DataFrame, only the columns you have selected appear, and you can focus on what matters for the current question.

A rule of thumb: the first thing you usually do when exploring a new question is `.select()` the three to six columns relevant to the question. Working with a wide DataFrame when you only need a narrow one is like writing an email with fifty people on cc when three would do.

FOUNDATIONS: `.rename` — change column names

Some source datasets have awkward column names — too long, too short, all caps, weird abbreviations. `.rename()` fixes this. You pass a dictionary mapping old names to new names:

```
renamed = core_cols.rename({
    "month": "sale_month",
    "floor_area_sqm": "area_sqm",
    "resale_price": "price",
})
```

Columns not mentioned in the dictionary keep their old names. The result is a new DataFrame with the renamed columns. Use `.rename()` sparingly — every rename is a potential source of confusion for a reader. But for columns you will reference dozens of times, shortening `resale_price` to `price` can save a lot of typing.

FOUNDATIONS: `.with_columns` — create new columns

`.with_columns()` adds new columns to a DataFrame (or replaces existing ones). This is how you compute derived values — things that aren't in the raw data but can be calculated from it.

```
hdb = hdb.with_columns(
    (pl.col("resale_price") / pl.col("floor_area_sqm")).alias("price_per_sqm")
)
```

Reading this line: take the `resale_price` column, divide it element-wise by the `floor_area_sqm` column, and give the result the name `price_per_sqm`. The `.alias()` method is what names the new column. Without it, the new column would have a generated name like `"literal"` or `"resale_price"` that would be useless.

`price_per_sqm` is a *normalised* measure — it removes the effect of flat size so you can compare two flats fairly. A three-room flat in Bishan might cost $400,000 and a five-room in Jurong might cost $600,000, but the three-room could easily have a higher price per square metre, because size and price both scale with "how desirable is this neighbourhood". Normalising by area lets you ask "which neighbourhood is actually more expensive per unit of space" rather than "which absolute price is bigger".

You can add multiple columns in a single `with_columns()` call by passing multiple expressions:

```
hdb = hdb.with_columns(
    pl.col("month").str.to_date("%Y-%m").alias("transaction_date"),
    pl.col("month").str.slice(0, 4).cast(pl.Int32).alias("year"),
)
```

This is more efficient than calling `with_columns` twice in a row, because Polars can fuse the operations into a single pass over the data. For small DataFrames the difference is invisible; for large ones it matters.

The expression `pl.col("month").str.to_date("%Y-%m")` is parsing a string column into a date column. `str.to_date` is a string-namespace method — `pl.col("month").str` gives you access to string functions (`to_date`, `slice`, `contains`, `replace`, `split`, etc.), and `.to_date("%Y-%m")` converts the string into a Polars `Date` using the given format pattern. `%Y` means four-digit year, `%m` means two-digit month. These format codes are standard Python `strftime` conventions, used by almost every date library — they are worth looking up once and bookmarking.

`pl.col("month").str.slice(0, 4)` is taking the first four characters of the string — which in a string like `"2023-01"` gives you `"2023"`. Then `.cast(pl.Int32)` converts that string to an integer. So the new `year` column is an integer — useful for filtering with `pl.col("year") >= 2020`, because integer comparisons are faster and cleaner than string comparisons.

FOUNDATIONS: `pl.when` — conditional column assignment

Sometimes you want to create a new column whose values depend on a condition. For example, you might want a `price_tier` column that takes the value `"budget"` if the price is below $350k, `"mid_range"` if between $350k and $500k, `"premium"` if between $500k and $700k, and `"luxury"` otherwise. The Polars idiom for this is `pl.when().then().when().then().otherwise()`:

```
hdb = hdb.with_columns(
    pl.when(pl.col("resale_price") < 350_000)
    .then(pl.lit("budget"))
    .when(pl.col("resale_price") < 500_000)
    .then(pl.lit("mid_range"))
    .when(pl.col("resale_price") < 700_000)
    .then(pl.lit("premium"))
    .otherwise(pl.lit("luxury"))
    .alias("price_tier")
)
```

Reading this: "when the price is less than 350,000, set this cell to 'budget'; otherwise when the price is less than 500,000, set it to 'mid_range'; otherwise when the price is less than 700,000, set it to 'premium'; otherwise set it to 'luxury'. Name the resulting column `price_tier`."

Notice that the `when` clauses are evaluated in order, and each one only applies if the previous ones were False. So the second `when(pl.col("resale_price") < 500_000)` effectively means "between 350,000 and 500,000", because the "less than 350,000" case has already been caught by the first `when`. This is the normal short-circuit behaviour of if/elif chains, and Polars makes it work the same way.

`pl.lit("budget")` wraps the plain Python string `"budget"` into a Polars literal expression. You need `pl.lit` because Polars' `.then()` expects an expression, not a raw Python value, and if you passed the bare string `"budget"` Polars would interpret it as a

column reference — and fail, because there is no column called `"budget"`.

FOUNDATIONS: `.sort` — order rows

`.sort()` orders rows by one or more columns:

```
hdb.sort("resale_price", descending=True)
```

This returns a new DataFrame sorted from highest to lowest `resale_price`. The default is ascending; `descending=True` flips it. You can sort by multiple columns:

```
hdb.sort("town", "resale_price", descending=[False, True])
```

This sorts first by `town` alphabetically, then within each town by `resale_price` from highest to lowest. The `descending` parameter can be a single Boolean (applies to all sort columns) or a list of Booleans (one per column).

Sort is an expensive operation — it processes every row. If you are going to sort a filtered subset of the data, filter first and sort second, not the other way around. Polars will often reorder these for you via its query optimiser, but for eager code the order you write matters.

FOUNDATIONS: Method chaining

Every Polars operation returns a new DataFrame. That means you can stack operations by attaching each one with a dot:

```
recent_premium = (
    hdb
    .filter(pl.col("year") >= 2020)
    .filter(pl.col("price_tier").is_in(["premium", "luxury"]))
    .select("transaction_date", "town", "flat_type", "price_per_sqm", "resale_price")
    .sort("resale_price", descending=True)
)
```

Read this top-to-bottom: “take the HDB dataset, keep rows from 2020 onwards, keep premium and luxury tiers, pick these five columns, sort by price descending.” Each step is a transformation of what came before. The final result is assigned to `recent_premium`. This is called *method chaining*, and it is the idiomatic way to write Polars code.

The parentheses around the whole expression are a Python trick. Inside a pair of parentheses, Python ignores newlines, so you can break a long expression across many lines for readability. Without the parentheses you would have to use backslash line-continuations, which are ugly. The convention: when you have more than two or three chained operations, wrap the whole thing in parentheses and put each `.method()` on its own line.

Method chains are read in the order they appear, and each line is applied to the result of the previous line. Do not try to hold the intermediate DataFrames in your head. Just read linearly: “filter, filter, select, sort”. The intermediate states are ephemeral; only the final result matters.

The Kailash Context

Lesson 1.2 is pure Polars — no Kailash engine is involved yet. But the patterns you are learning here will show up again inside the engines. DataExplorer (Lesson 1.7) uses `pl.col()` expressions internally to compute per-column statistics. PreprocessingPipeline (Lesson 1.8) uses `pl.when().then()` conditional logic internally when it encodes categorical columns. ModelVisualizer (Lesson 1.6) accepts Polars DataFrames directly as input — you will hand it the output of your `.filter().sort()` chains. Everything you learn about Polars is reusable inside every Kailash engine you will meet.

Worked Example: HDB Resale Flats

The dataset for this lesson is `hdb_resale.parquet` — roughly half a million transactions from the Singapore Housing and Development Board. This is real data, published openly by the HDB at `data.gov.sg`. It is also our first use of the *Parquet* file format instead of CSV. Parquet is a columnar storage format that is much more efficient than CSV for numeric data: it is smaller on disk, faster to load, and remembers column types so you do not have to re-parse dates and numbers on every load. Polars reads Parquet with `pl.read_parquet()`, but `MLFPDataLoader.load()` figures out the format from the file extension, so your code is the same.

Step 1: Load the data

```python
from __future__ import annotations

import polars as pl

from shared import MLFPDataLoader

loader = MLFPDataLoader()
hdb = loader.load("mlfp01", "hdb_resale.parquet")

print(f"Shape: {hdb.shape}")
print(f"Columns: {hdb.columns}")
print(hdb.head(3))
```

Expected output (values may vary slightly by dataset version):

```
Shape: (487_293, 11)
Columns: ['month', 'town', 'flat_type', 'block', 'street_name', 'storey_range',
          'floor_area_sqm', 'flat_model', 'lease_commence_date',
          'remaining_lease', 'resale_price']
shape: (3, 11)
```

month	town	flat_type	block	…	lease_commence_date	remaining_lease	resale_price
2017-01	ANG MO KIO	2 ROOM	406	…	1979	61 years	232000.0
2017-01	ANG MO KIO	3 ROOM	108	…	1978	60 years	250000.0
2017-01	ANG MO KIO	3 ROOM	602	…	1980	62 years	262000.0

Half a million rows, eleven columns. That is already too big to scan by eye. Every subsequent question has to be answered with code.

Step 2: Basic filters

Start with simple single-condition filters and check the row counts. If a filter returns zero rows, you almost certainly got a column value wrong (wrong capitalisation, wrong spelling, wrong type).

```python
ang_mo_kio = hdb.filter(pl.col("town") == "ANG MO KIO")
print(f"Ang Mo Kio transactions: {ang_mo_kio.height:,}")

four_room = hdb.filter(pl.col("flat_type") == "4 ROOM")
print(f"4-room flats: {four_room.height:,}")

affordable = hdb.filter(
    (pl.col("resale_price") >= 300_000) & (pl.col("resale_price") <= 500_000)
)
print(f"Transactions S$300k-500k: {affordable.height:,}")
```

Expected output:

```
Ang Mo Kio transactions: 28,847
4-room flats: 199,254
Transactions S$300k-500k: 188,912
```

Notice that the `town` values are all-caps. This is a quirk of the source data — HDB publishes town names in all capitals. If you had written `pl.col("town") == "Ang Mo Kio"` (title case), you would have got zero rows and spent ten minutes wondering why. When a filter returns zero, always inspect the actual values in the column with `df["town"].unique()` to see what casing the source data uses.

Step 3: Combined filters

Now combine three conditions into a single filter:

```python
amk_4room_affordable = hdb.filter(
    (pl.col("town") == "ANG MO KIO")
    & (pl.col("flat_type") == "4 ROOM")
    & (pl.col("resale_price") <= 500_000)
)
print(f"AMK 4-room under S$500k: {amk_4room_affordable.height:,}")
```

Expected output:

```
AMK 4-room under S$500k: 7,214
```

Each `&` narrows the result further. The combined filter must be a subset of each single filter — `amk_4room_affordable` has fewer rows than `ang_mo_kio` (which was 28,847) and fewer than `four_room` (199,254) and fewer than `affordable` (188,912). The intersection is always smaller than any of its parts.

And use `.is_in()` for the central-towns question:

```
central_towns = ["BISHAN", "TOA PAYOH", "QUEENSTOWN", "BUKIT MERAH"]
central = hdb.filter(pl.col("town").is_in(central_towns))
print(f"Central towns transactions: {central.height:,}")
```

This is cleaner than `(pl.col("town") == "BISHAN") | (pl.col("town") == "TOA PAYOH") | ...`.

Step 4: Select and rename

Narrow the DataFrame to the columns you actually need, then clean up the names:

```
core_cols = hdb.select(
    "month", "town", "flat_type", "floor_area_sqm", "resale_price"
)
renamed = core_cols.rename({
    "month": "sale_month",
    "floor_area_sqm": "area_sqm",
    "resale_price": "price",
})
print(renamed.head(3))
```

Expected:

```
shape: (3, 5)
```

sale_month	town	flat_type	area_sqm	price
2017-01	ANG MO KIO	2 ROOM	44.0	232000.0
2017-01	ANG MO KIO	3 ROOM	67.0	250000.0
2017-01	ANG MO KIO	3 ROOM	68.0	262000.0

Now when you write `pl.col("price")` you do not have to remember whether it was `resale_price` or `resale_price_sgd` or something else; it is just `price`.

Step 5: Derive new columns

Add `price_per_sqm`, `transaction_date`, and `year` to the original `hdb` DataFrame:

```
hdb = hdb.with_columns(
    (pl.col("resale_price") / pl.col("floor_area_sqm")).alias("price_per_sqm"),
)

hdb = hdb.with_columns(
    pl.col("month").str.to_date("%Y-%m").alias("transaction_date"),
    pl.col("month").str.slice(0, 4).cast(pl.Int32).alias("year"),
)

print(hdb.select("month", "transaction_date", "year", "price_per_sqm").head(5))
```

Expected:

```
shape: (5, 4)
```

month	transaction_date	year	price_per_sqm
2017-01	2017-01-01	2017	5272.727273
2017-01	2017-01-01	2017	3731.343284
2017-01	2017-01-01	2017	3852.941176
2017-01	2017-01-01	2017	4864.864865
2017-01	2017-01-01	2017	5266.666667

The first row has a high price per sqm because it is a small two-room flat — a reminder that small flats often have higher per-sqm prices because some costs (entrance, bathroom) are fixed regardless of size.

Step 6: Conditional price tier

Add the `price_tier` column with `pl.when().then()`:

```
hdb = hdb.with_columns(
    pl.when(pl.col("resale_price") < 350_000).then(pl.lit("budget"))
    .when(pl.col("resale_price") < 500_000).then(pl.lit("mid_range"))
    .when(pl.col("resale_price") < 700_000).then(pl.lit("premium"))
    .otherwise(pl.lit("luxury"))
    .alias("price_tier")
)

tier_counts = (
    hdb.group_by("price_tier")
    .agg(pl.len().alias("count"))
    .sort("count", descending=True)
)
print(tier_counts)
```

(We'll cover `group_by` in Lesson 1.3; here we use it as a quick way to see how many rows ended up in each tier.) Expected output:

```
shape: (4, 2)
┌──────────────┬────────┐
│ price_tier   ┆ count  │
│ mid_range    ┆ 198543 │
│ budget       ┆ 132857 │
│ premium      ┆ 107824 │
│ luxury       ┆ 48069  │
└──────────────┴────────┘
```

The mid-range tier dominates — 198,000 transactions out of 487,000, roughly 40%. The luxury tier is the smallest at about 10%. This distribution is the reason the "million-dollar HDB" transactions that make newspaper headlines are genuinely unusual: they are the tail of the tail.

Step 7: Chain everything together

The real payoff is when you chain filters, selects, and sorts into a single readable pipeline:

```
recent_premium = (
    hdb
    .filter(pl.col("year") >= 2020)
    .filter(pl.col("price_tier").is_in(["premium", "luxury"]))
    .select(
        "transaction_date", "town", "flat_type",
        "price_per_sqm", "price_tier", "resale_price"
    )
    .sort("resale_price", descending=True)
)

print(f"Count: {recent_premium.height:,}")
print(recent_premium.head(10))
```

Reading the chain top-to-bottom: "start with the HDB dataset, keep rows from 2020 onwards, keep premium and luxury tiers, pick these six columns, sort by price descending". The output starts with the absolute most expensive recent transactions — typically in the $1.3–1.5 million range for the much-publicised million-dollar HDBs.

The `recent_premium` DataFrame is the answer to a specific question ("what are the highest-priced recent HDB resales?"), derived from the raw data in five lines of chained Polars. If someone asks the next question ("now group them by town"), you add another line to the chain. This is the rhythm of exploratory data analysis: one question, one chain, one answer, next question.

Try It Yourself

Drill 1. Filter `hdb` to keep only 4-room or 5-room flats (use `.is_in()`) in Bishan that sold for more than $600,000. How many rows does the result contain?

Drill 2. Create a new column `price_per_year_lease` that is `resale_price / lease_commence_date`. The result will be meaningless (dividing price by a year gives you nonsense units) but the exercise is about the mechanics of `with_columns` and

`.alias`. Print the first five rows of the resulting column along with `resale_price` and `lease_commence_date`.

Drill 3. Write a single chained expression that: filters to 2023 transactions only, adds a `price_per_sqm` column, selects `town`, `flat_type`, `floor_area_sqm`, `resale_price`, `price_per_sqm`, and sorts by `price_per_sqm` descending. Print the top 10.

Drill 4. Use `pl.when().then()` to create a column `flat_size_category` where floor_area_sqm ≤ 60 is `"compact"`, 60–90 is `"standard"`, 90–120 is `"large"`, and above 120 is `"jumbo"`. Count how many rows fall into each category.

Drill 5. What is the median `price_per_sqm` in the `luxury` tier? In the `budget` tier? What is the ratio between them? (Hint: filter to the tier, then call `.median()` on the column.)

Cross-References

- **Lesson 1.3** will use the `group_by` + `agg` pattern that appeared briefly in Step 6 of this lesson, and combine it with functions and loops for reusable analysis.
- **Lesson 1.5** will extend `.with_columns` with window functions like `rolling_mean` and `shift`, which compute values across *nearby rows* rather than per-row.
- **Lesson 1.8** will use Polars filtering extensively to clean a messy taxi dataset — removing GPS points outside Singapore's bounding box, fares below zero, and trip durations above 3 hours.

Reflection

You should now be able to:

- Explain the difference between `&` / `|` / `~` in Polars and why comparison expressions must be parenthesised.
- Write a Polars filter that combines two or more conditions.
- Use `.select()` to narrow a DataFrame to a few columns and `.rename()` to clean the names.
- Use `.with_columns()` with `.alias()` to add derived columns, including columns built from arithmetic on existing columns.
- Use `pl.when().then().otherwise()` to build a conditional column with three or more branches.
- Sort a DataFrame by one or more columns in ascending or descending order.
- Write a multi-step method-chain pipeline that combines filter, select, and sort.

Drill answers

1.
```
result = hdb.filter(
    pl.col("flat_type").is_in(["4 ROOM", "5 ROOM"]) &
    (pl.col("town") == "BISHAN") &
    (pl.col("resale_price") > 600_000)
)
print(result.height)
```

2.
```
hdb.with_columns(
    (pl.col("resale_price") / pl.col("lease_commence_date")).alias("price_per_year_lease")
).select("resale_price", "lease_commence_date", "price_per_year_lease").head(5)
```

3.
```
top_psm = (
    hdb.filter(pl.col("year") == 2023)
    .with_columns((pl.col("resale_price") / pl.col("floor_area_sqm")).alias("price_per_sqm"))
    .select("town", "flat_type", "floor_area_sqm", "resale_price", "price_per_sqm")
    .sort("price_per_sqm", descending=True)
    .head(10)
)
```

4.
```
hdb = hdb.with_columns(
    pl.when(pl.col("floor_area_sqm") <= 60).then(pl.lit("compact"))
    .when(pl.col("floor_area_sqm") <= 90).then(pl.lit("standard"))
    .when(pl.col("floor_area_sqm") <= 120).then(pl.lit("large"))
    .otherwise(pl.lit("jumbo"))
    .alias("flat_size_category")
)
print(hdb.group_by("flat_size_category").agg(pl.len().alias("count")))
```

5.
```
lux_psm = hdb.filter(pl.col("price_tier") == "luxury")["price_per_sqm"].median()
bud_psm = hdb.filter(pl.col("price_tier") == "budget")["price_per_sqm"].median()
print(f"Luxury median PSM: {lux_psm:.0f}")
print(f"Budget median PSM: {bud_psm:.0f}")
print(f"Ratio: {lux_psm / bud_psm:.2f}")
```

   The ratio is typically around 1.8–2.2 — luxury tier flats are not twice as big, they're in more expensive neighbourhoods, so the normalised price per square metre tells you the location premium directly.

Lesson 1.3: Functions and Aggregation

Why This Matters

So far every question you have asked has been a one-off: filter for this town, compute that mean, print the result. Real data analysis is not one-off. You ask the same question twenty-six times — once for each HDB town — and compare the answers. You want to know the median price of every district, not just Ang Mo Kio. You want the standard deviation too, and the 75th percentile, and the transaction count, all at once. Writing twenty-six separate filter-and-compute blocks would be six hundred lines of code and the inevitable bug from copy-pasting.

The antidote is two-fold. First, *functions* let you package a calculation under a name so it can be reused. Write the calculation once, give it a name, call it as many times as you want. Second, *group-by aggregation* lets you tell Polars "for each distinct value in this column, compute these statistics" in a single call. You do not loop over the groups yourself; Polars does it for you, and does it fast.

By the end of this lesson you will have written your first Python functions, used them to classify and format values, and aggregated a 500,000-row dataset into a twenty-six-row district summary table. The district table is the kind of output that ends up on slide three of a property-market briefing — except you will have built it yourself, which means you will know exactly how every number was computed and will not be caught out when someone asks.

Core Concepts

FOUNDATIONS: What is a function?

A function is a named block of code that takes inputs, does something with them, and optionally returns a value. You write the function once, and you call it by name as many times as you want. Every time you call it, the inputs can be different, but the computation is the same.

Defining a function uses the `def` keyword:

```
def format_sgd(amount):
    return f"S${amount:,.0f}"
```

Reading this line by line: `def format_sgd(amount):` says "I am defining a function called `format_sgd` that takes one input parameter, called `amount`". The colon at the end starts a block. Every indented line after the colon is part of the function body — Python uses indentation to define blocks, not curly braces as some other languages do. The `return` statement sends a value back to the caller; when you call `format_sgd(485_000)` you get back the string `"S$485,000"`.

To call a function, write its name followed by parentheses containing the arguments:

```
print(format_sgd(485_000))       # S$485,000
print(format_sgd(1_200_000))     # S$1,200,000
```

The parameter `amount` inside the function is a local variable — it only exists while the function is running. The caller does not see it. You can give the function a different name when you call it, and you can call it from anywhere — even from inside another function.

Functions are important for two reasons. First, they prevent repetition: if you format a price in Singapore dollars fifty times in a report, you want that formatting logic in one place so that when you decide to change the currency symbol or add a decimal point, you change it once instead of fifty times. Second, they hide complexity: the caller only needs to know what goes in and what comes out; the details of how the function computes its result are not the caller's problem. When you see `compute_iqr(series)` in code, you do not need to look inside to guess what it does — the name tells you it computes the interquartile range.

FOUNDATIONS: Parameters and return values

Functions can take any number of parameters (including zero):

```
def greet():
    return "Hello, Singapore!"

def add(a, b):
    return a + b

def describe_flat(town, flat_type, price, area):
    price_per_sqm = price / area
    return f"{flat_type} in {town}: S${price:,.0f} ({price_per_sqm:,.0f}/sqm)"
```

The last one shows that you can do arbitrary computation inside the function and return a formatted result. Every parameter is a local variable inside the function; the caller has no way to see or modify them except through what the function returns.

You can give parameters *default values* so the caller can omit them:

```
def format_price(amount, currency="SGD"):
    return f"{currency}${amount:,.0f}"

print(format_price(485_000))           # SGD$485,000
print(format_price(485_000, "USD"))    # USD$485,000
```

In the second call we override the default. In the first, `currency` takes its default value. Default parameters make functions more flexible without cluttering the common case.

THEORY: Type hints

Python is a dynamically typed language — functions accept any type at runtime — but you can *annotate* parameter and return types to document what the function expects:

```
def format_sgd(amount: float) -> str:
    return f"S${amount:,.0f}"
```

The `: float` after `amount` is a type hint saying "this parameter should be a float". The `-> str` after the closing parenthesis is a return-type hint saying "this function returns a string". Python itself does not enforce these hints at runtime — you can still pass an `int` to `format_sgd` and it will work (ints are formatted fine by the f-string). The hints are documentation for humans and for static type checkers like `mypy`, which can flag incorrect usage before you run the code.

Throughout MLFP we use type hints on all functions. It is a good habit. When you come back to your own code a month later, the hints tell you what the function expects without having to re-read the body.

FOUNDATIONS: `if` / `elif` / `else`

Inside a function you will frequently want to do one thing in some cases and a different thing in others. Python's conditional statement is `if` / `elif` / `else`:

```
def price_range_label(price: float) -> str:
    if price < 350_000:
        return "Budget (<350k)"
    elif price < 500_000:
        return "Mid-range (350k-500k)"
    elif price < 700_000:
        return "Premium (500k-700k)"
    else:
        return "Luxury (700k+)"
```

Read this as: "if the price is less than 350,000, return 'Budget'. Otherwise, if the price is less than 500,000, return 'Mid-range'. Otherwise, if the price is less than 700,000, return 'Premium'. Otherwise, return 'Luxury'." The branches are evaluated top-to-bottom, and only the first branch whose condition is True is executed. `elif` is short for "else if" and lets you chain multiple conditions without deeply nesting indentation.

This is the Python-code equivalent of `pl.when().then()` from Lesson 1.2. The difference: `if/elif/else` operates on a single value at a time (one flat's price), while `pl.when().then()` operates on an entire column at once. When you are writing a function that takes one value and returns one value, use Python `if`. When you are building a column based on another column, use `pl.when`. Both exist because both are needed.

Common mistake: using `if` inside a Polars filter. Writing `hdb.filter(if pl.col("price") > 500_000)` is a syntax error — `if` is a statement, not an expression. The Polars equivalent is `hdb.filter(pl.col("price") > 500_000)` — a Boolean expression that evaluates element-wise.

FOUNDATIONS: Lists, dictionaries, and loops

Before you can call a function on many values, you need a way to hold many values. Python has two fundamental collection types you will use constantly:

Lists are ordered, mutable sequences. You create a list with square brackets:

```
towns = ["BISHAN", "TOA PAYOH", "QUEENSTOWN", "ANG MO KIO"]
print(towns[0])      # BISHAN
print(towns[-1])     # ANG MO KIO (negative indices count from the end)
print(len(towns))    # 4
towns.append("BEDOK")
print(towns)         # ['BISHAN', 'TOA PAYOH', 'QUEENSTOWN', 'ANG MO KIO', 'BEDOK']
```

You can iterate over a list with a `for` loop:

```
for town in towns:
    print(town)
```

This runs the indented block once for each item in the list, with the variable `town` bound to that item each time. For loops work on any *iterable* — lists, strings, dictionaries, Polars Series, even the rows of a DataFrame.

Dictionaries are unordered collections of key-value pairs. You create a dictionary with curly braces:

```
prices = {"BISHAN": 580_000, "QUEENSTOWN": 620_000, "YISHUN": 420_000}
print(prices["BISHAN"])    # 580000
prices["JURONG WEST"] = 400_000   # add a new key
print(len(prices))         # 4
```

Dictionaries are the tool you use when you need to look up a value by a key — town name to mean price, flat type to count, any mapping from one kind of thing to another. When you iterate over a dictionary with a `for` loop, you get the keys by default. To get both keys and values use `.items()`:

```
for town, price in prices.items():
    print(f"{town}: S${price:,}")
```

FOUNDATIONS: Iterating over DataFrame rows

Occasionally — not often, but sometimes — you want to process each row of a DataFrame one at a time in Python. Polars provides `.iter_rows()` for this. The `named=True` option yields each row as a dictionary, which is much more readable than a tuple:

```
for row in top_10_districts.iter_rows(named=True):
    town = row["town"]
    median = row["median_price"]
    print(f"{town}: S${median:,.0f}")
```

This is fine for formatting output — building a printed report one line per row. It is *not* fine for computation. Iterating over rows in Python to compute something is typically 100–1000× slower than the vectorised Polars equivalent. Rule of thumb: if you are doing arithmetic inside the loop, you should be using a Polars expression instead. If you are just formatting strings, `iter_rows` is the right tool.

FOUNDATIONS: `group_by` + `agg` — the most important pattern in data analysis

Here is the question: "for each town, what is the median HDB resale price?" In SQL this would be `SELECT town, MEDIAN(resale_price) FROM hdb GROUP BY town`. In Polars it is:

```
hdb.group_by("town").agg(pl.col("resale_price").median().alias("median_price"))
```

The pattern is always the same:

1. `.group_by(col)` — split the DataFrame into groups, one per unique value of `col`.
2. `.agg(expressions)` — compute one or more aggregation expressions for each group.

The result is a new DataFrame with one row per group, where the first column is the grouping key and the rest are the aggregated values.

You can pass multiple aggregation expressions to `.agg()` to compute many statistics at once:

```
district_stats = (
    hdb.group_by("town")
    .agg(
        pl.len().alias("transaction_count"),
        pl.col("resale_price").mean().alias("mean_price"),
        pl.col("resale_price").median().alias("median_price"),
        pl.col("resale_price").std().alias("std_price"),
        pl.col("resale_price").min().alias("min_price"),
        pl.col("resale_price").max().alias("max_price"),
        pl.col("resale_price").quantile(0.25).alias("q25_price"),
        pl.col("resale_price").quantile(0.75).alias("q75_price"),
    )
    .sort("median_price", descending=True)
)
```

This single call processes 500,000 rows and produces a 26-row summary table (one row per Singapore HDB town). It does count, mean, median, standard deviation, min, max, and two quantiles — eight statistics per town — in a single pass. On modern hardware this completes in well under a second. Trying to do the same thing with a manual for loop over the towns would take minutes and dozens of lines of code.

`pl.len()` is a special aggregation that returns the number of rows in each group — the "count" column. It is different from `pl.col("something").count()`, which counts non-null values in a specific column. When you want "how many transactions in this group", use `pl.len()`.

You can group by multiple columns at once:

```
town_flat_stats = (
    hdb.group_by("town", "flat_type")
    .agg(
        pl.len().alias("count"),
        pl.col("resale_price").median().alias("median_price"),
    )
)
```

This produces one row per unique `(town, flat_type)` pair. With 26 towns and 7 flat types, the result could have up to 182 rows (though in practice some combinations do not exist — there are no jumbo flats in every town).

THEORY: What group_by does under the hood

Internally, group_by performs the following steps:

1. **Hash the group keys.** For each row, compute a hash of the group-key columns. Rows with the same hash go into the same bucket.
2. **Partition the data.** Distribute rows across worker threads using the hash. Polars parallelises group_by operations across all cores.
3. **Aggregate within each group.** For each bucket, compute the aggregation expressions. Single-pass aggregations like `mean`, `count`, `sum`, `min`, and `max` are *online* — they update a running state as they scan rows, so memory use is O(number of groups), not O(number of rows).
4. **Collect the results.** Gather the per-group results into a single output DataFrame, one row per group.

For some aggregations, like `median` and `quantile`, the online approach doesn't work — you need the full distribution to compute a quantile. These aggregations are slightly slower because they require buffering the values in each group. Polars handles the difference transparently.

The takeaway: `group_by` is fast because it is parallelised and because most aggregations are streaming. You do not need to optimise it or write it differently for large data. Just use it.

The Kailash Context

Kailash's engines use `group_by` extensively under the hood. When DataExplorer profiles a categorical column it effectively does a `group_by` to count each category. When PreprocessingPipeline encodes a categorical column with target encoding it does a `group_by`

on the category and averages the target value per group. When ModelVisualizer produces a box plot per district it does a `group_by` on the district and computes quartiles. You are learning the pattern that the engines are built on. That is the reason this lesson is here: so that you understand what the engines are doing, not just how to call them.

Worked Example: District Statistics

We continue with the HDB dataset. The goal of the worked example is to build a complete district-level report — one row per town — with counts, price statistics, and derived spread metrics, then iterate over the top 15 rows to print a formatted report.

Step 1: Set up and define helper functions

Start with the imports and a couple of helper functions. Functions that format numbers or classify values are common enough that you will write them every day; practise now.

```
from __future__ import annotations

import polars as pl

from shared import MLFPDataLoader


def format_sgd(amount: float) -> str:
    """Format a number as Singapore dollars with thousands separator."""
    return f"S${amount:,.0f}"


def price_range_label(price: float) -> str:
    """Classify a resale price into a human-readable tier."""
    if price < 350_000:
        return "Budget (<350k)"
    elif price < 500_000:
        return "Mid-range (350k-500k)"
    elif price < 700_000:
        return "Premium (500k-700k)"
    else:
        return "Luxury (700k+)"


def compute_iqr(series: pl.Series) -> float:
    """Compute the interquartile range (Q3 - Q1) of a Polars Series."""
    q75 = series.quantile(0.75)
    q25 = series.quantile(0.25)
    if q75 is None or q25 is None:
        return 0.0
    return q75 - q25
```

Three functions. The first formats a number as SGD. The second classifies a price into a tier. The third computes an interquartile range — the difference between the 75th and 25th percentiles — which is a robust measure of spread that ignores the extreme tails.

Note the `"""..."""` strings on the first line of each function body. These are *docstrings* — documentation for the function that can be read by Python's `help()` function, IDE tooltips, and documentation generators. Every function you write in this course should have a one-line docstring saying what it does. It is a small investment that pays back every time you read your own code a week later.

Test the functions before using them:

```
print(format_sgd(485_000))                                      # S$485,000
print(price_range_label(485_000))                               # Mid-range (350k-500k)
print(price_range_label(720_000))                               # Luxury (700k+)

test_prices = pl.Series("prices", [300_000, 400_000, 500_000, 600_000, 700_000])
print(f"IQR of test prices: {format_sgd(compute_iqr(test_prices))}")  # S$200,000
```

Always test helpers on a small example before plugging them into a 500,000-row pipeline. If the IQR helper returns nonsense for five values, it will return nonsense for five hundred thousand.

Step 2: The district statistics group_by

Load the data and add the derived columns we need:

```
loader = MLFPDataLoader()
hdb = loader.load("mlfp01", "hdb_resale.parquet")

hdb = hdb.with_columns(
    (pl.col("resale_price") / pl.col("floor_area_sqm")).alias("price_per_sqm"),
    pl.col("month").str.slice(0, 4).cast(pl.Int32).alias("year"),
)
```

Then run the big group_by:

```
district_stats = (
    hdb.group_by("town")
    .agg(
        pl.len().alias("transaction_count"),
        pl.col("resale_price").mean().alias("mean_price"),
        pl.col("resale_price").median().alias("median_price"),
        pl.col("resale_price").std().alias("std_price"),
        pl.col("resale_price").min().alias("min_price"),
        pl.col("resale_price").max().alias("max_price"),
        pl.col("resale_price").quantile(0.25).alias("q25_price"),
        pl.col("resale_price").quantile(0.75).alias("q75_price"),
        pl.col("price_per_sqm").median().alias("median_price_sqm"),
        pl.col("floor_area_sqm").median().alias("median_area_sqm"),
    )
    .sort("median_price", descending=True)
)

print(f"Districts: {district_stats.height}")
print(district_stats.head(5))
```

The output is a tidy table of 26 rows sorted from most expensive to least, with ten columns of statistics per row. The first row should be something like Bukit Timah or Central Area; the last row is typically Sembawang or Choa Chu Kang — the cheaper peripheral new towns.

Step 3: Add derived columns to the aggregated table

Now that you have the per-town statistics, you can compute further derived columns the same way as on any DataFrame:

```
district_stats = district_stats.with_columns(
    (pl.col("q75_price") - pl.col("q25_price")).alias("iqr_price"),
    (pl.col("std_price") / pl.col("mean_price") * 100).alias("cv_price_pct"),
    (pl.col("median_price") / pl.col("max_price")).alias("premium_ratio"),
)
```

Three new columns:

- `iqr_price` — the interquartile range. A wide IQR means prices vary a lot within the town (diverse housing stock).
- `cv_price_pct` — the *coefficient of variation* as a percentage. CV is standard deviation divided by mean, expressed as a percentage. Unlike the raw std, CV is scale-invariant: a CV of 25% means "the spread is a quarter of the average", regardless of whether the average is $400k or $4,000. This makes CV the right tool for comparing spread across groups with different means.
- `premium_ratio` — the median divided by the max. A premium_ratio near 1 means the median is close to the maximum, i.e. the whole town is expensive; a premium_ratio near 0.5 means the median is half the max, i.e. there's a wide range with many cheap units.

Print the core columns:

```
print(district_stats.select(
    "town", "transaction_count", "median_price", "iqr_price", "cv_price_pct"
).head(10))
```

Step 4: Multi-key group_by — (town, flat_type) combinations

Sometimes one grouping key is not enough. "Median price per town" is useful but hides the flat type — a town with lots of five-room flats will look more expensive than a town with mostly three-room, even if the price per square metre is the same. To control for flat

type, group by both columns:

```
town_flat_stats = (
    hdb.group_by("town", "flat_type")
    .agg(
        pl.len().alias("count"),
        pl.col("resale_price").median().alias("median_price"),
        pl.col("price_per_sqm").median().alias("median_price_sqm"),
    )
    .sort("town", "flat_type")
)

print(town_flat_stats.filter(pl.col("town") == "ANG MO KIO"))
```

Expected:

```
shape: (6, 5)
```

town	flat_type	count	median_price	median_price_sqm
ANG MO KIO	2 ROOM	112	230000.0	5227.0
ANG MO KIO	3 ROOM	6234	285000.0	4275.0
ANG MO KIO	4 ROOM	12897	475000.0	4950.0
ANG MO KIO	5 ROOM	6188	680000.0	5540.0
ANG MO KIO	EXECUTIVE	3104	820000.0	5930.0
ANG MO KIO	MULTI-GEN'L	312	985000.0	6200.0

Now you can see the structure: within a single town, five-room flats go for more than three-room, and executives go for more than five-room. The median_price_sqm tells you something interesting too — it rises with flat size even within the same town, because bigger flats often come with desirable amenities (corner units, higher floors, larger rooms).

Step 5: Iterate over the district report and format each line

Write a helper function that formats one row of the district stats as a report line, then loop over the top 15 rows and call the helper for each:

```
def district_report_line(row: dict) -> str:
    """Format one district row as a human-readable report line."""
    town = row["town"]
    median = format_sgd(row["median_price"])
    count = row["transaction_count"]
    cv = row["cv_price_pct"]
    sqm = format_sgd(row["median_price_sqm"])
    return f"  {town:<20} {median:>12}  {count:>8,}  CV={cv:5.1f}%  {sqm:>12}/sqm"


print(f"\n{'=' * 70}")
print(f"  SINGAPORE HDB DISTRICT PRICE REPORT")
print(f"{'=' * 70}")
print(f"  {'Town':<20} {'Median Price':>12}  {'Txns':>8}  {'Spread':>8}  {'Per sqm':>12}")
print(f"  {'-' * 66}")

top_15 = district_stats.head(15)
for row in top_15.iter_rows(named=True):
    print(district_report_line(row))

print(f"{'=' * 70}")
```

Expected output (values will vary by dataset vintage):

```
======================================================================
  SINGAPORE HDB DISTRICT PRICE REPORT
======================================================================
  Town                 Median Price      Txns    Spread       Per sqm
  ------------------------------------------------------------------
  BUKIT TIMAH              S$780,000     1,453  CV= 32.1%     S$7,450/sqm
  CENTRAL AREA             S$720,000     3,287  CV= 38.5%     S$9,120/sqm
  QUEENSTOWN               S$680,000    10,234  CV= 29.4%     S$7,890/sqm
  BISHAN                   S$650,000    12,478  CV= 24.8%     S$6,940/sqm
  BUKIT MERAH              S$610,000    15,632  CV= 27.2%     S$7,210/sqm
```

```
  ...
======================================================================
```

The ten most expensive towns by median price. CV varies noticeably — some towns (like Bishan) have tight spreads around their median, while others (like Central Area) have wide spreads because they mix tiny studio apartments with large penthouses. The Central Area's high price per square metre (~$9,000) is unmatched anywhere else — that is the pure "location premium" for being in the heart of the city.

Step 6: Cross-district summary

One more pass — a summary of the summary:

```
all_medians = district_stats["median_price"]
print(f"\nCross-district summary:")
print(f"  Most expensive district:   {format_sgd(all_medians.max())}")
print(f"  Least expensive district:  {format_sgd(all_medians.min())}")
print(f"  Average district median:   {format_sgd(all_medians.mean())}")
print(f"  Price spread (max - min):  {format_sgd(all_medians.max() - all_medians.min())}")
```

The "price spread" tells you how unequal Singapore's public housing is — the difference between the most and least expensive district. A spread of $300,000+ indicates that location still matters significantly despite the government's efforts to provide comparable public housing across the island.

Notice we wrote `format_sgd` five times. Because it is a function, changing the formatting in one place (say, to add a decimal point) updates every output line. That is the payoff for writing helper functions.

Try It Yourself

Drill 1. Write a function `format_percent(value: float, decimals: int = 1) -> str` that formats a decimal between 0 and 1 as a percentage string with the given number of decimal places. For example, `format_percent(0.2345)` should return `"23.5%"` and `format_percent(0.2345, 2)` should return `"23.45%"`.

Drill 2. Group the HDB data by `flat_type` and compute the count, median price, and median price per sqm per flat type. Sort by median price ascending. How many distinct flat types are there? Which is most common by transaction count?

Drill 3. Write a function `growth_category(yoy_pct: float) -> str` that returns `"declining"` if the value is below -1, `"flat"` if between -1 and 1, `"growing"` if between 1 and 5, and `"booming"` if above 5.

Drill 4. Compute a two-level group_by: for each (year, flat_type) combination, the median price. Filter the result to 4-room flats only and print it sorted by year. Is there a clear upward trend?

Drill 5. Using `iter_rows(named=True)`, write a for loop that prints the town names of the five districts with the *widest* spread (highest `iqr_price`). Use a helper function that formats each line.

Cross-References

- **Lesson 1.4** will join the district statistics table to MRT station proximity data and school density data, letting you ask questions like "do flats near MRT cost more per sqm?"
- **Lesson 1.5** will replace `group_by` with *window functions* for calculations that need to keep the row-level detail, such as rolling averages and YoY changes per town.
- **Lesson 1.6** will visualise the district statistics as a bar chart ranked by median price — the same data, communicated visually.
- **Module 2** will use `group_by` for feature engineering: computing per-user aggregates, per-category means for target encoding, and per-cohort statistics for feature stores.

Reflection

You should now be able to:

- Define a Python function with parameters, a return value, a docstring, and type hints.
- Use `if` / `elif` / `else` inside a function to return different values based on the input.
- Create and iterate over lists and dictionaries.
- Use `df.group_by(col).agg(expressions)` to compute per-group statistics on a DataFrame.
- Pass multiple aggregation expressions to a single `.agg()` call to compute many statistics at once.

- Group by multiple columns to build cross-tabulations.
- Use `iter_rows(named=True)` to iterate over DataFrame rows in Python and feed each row into a helper function.
- Explain the difference between `pl.when` (column-wise conditional) and Python `if` (value-wise conditional) and know when to use each.

Drill answers

1.
```
def format_percent(value: float, decimals: int = 1) -> str:
    return f"{value * 100:.{decimals}f}%"
```

2.
```
flat_stats = (
    hdb.group_by("flat_type")
    .agg(
        pl.len().alias("count"),
        pl.col("resale_price").median().alias("median_price"),
        pl.col("price_per_sqm").median().alias("median_price_sqm"),
    )
    .sort("median_price")
)
print(flat_stats)
# Usually 4 ROOM is most common with ~199,000 transactions.
```

3.
```
def growth_category(yoy_pct: float) -> str:
    if yoy_pct < -1:
        return "declining"
    elif yoy_pct < 1:
        return "flat"
    elif yoy_pct < 5:
        return "growing"
    else:
        return "booming"
```

4.
```
yf = (
    hdb.group_by("year", "flat_type")
    .agg(pl.col("resale_price").median().alias("median_price"))
    .filter(pl.col("flat_type") == "4 ROOM")
    .sort("year")
)
print(yf)
# Clear upward trend in almost every year — 4-room median typically rises ~3-5% annually.
```

5.
```
widest = district_stats.sort("iqr_price", descending=True).head(5)
for row in widest.iter_rows(named=True):
    print(f"  {row['town']:<20}  IQR={format_sgd(row['iqr_price'])}")
```

Lesson 1.4: Joins and Multi-Table Data

Why This Matters

Real questions rarely fit inside a single table. “Are HDB flats near an MRT station more expensive?” requires the HDB transactions table plus a table of MRT station locations. “Do districts with more schools command higher prices?” requires the HDB table plus a schools table. “What is the correlation between a district’s median price and its distance to the CBD?” requires the HDB table plus a reference table of distances-to-CBD.

The operation that combines two tables on a shared key is called a *join*. Joins are the workhorse of relational data work; every SQL database has them, every DataFrame library has them, and the same mental model — match rows based on a shared key, decide what to do with the non-matches — applies everywhere. In this lesson you will learn what a join is, the four types you actually need (left, inner, right, outer), how to choose between them, and how to handle the nulls that arise when a join misses.

You will also meet your first `import` statement for a third-party package beyond Polars, and your first `if` statement in pure Python (as distinct from `pl.when` inside a Polars expression). These are small additions to the Python vocabulary but they unlock a lot.

Core Concepts

FOUNDATIONS: What is a join?

A join combines two tables by matching rows that share a common *key*. Suppose you have two tables:

Table A: HDB transactions (key column: `town`)

```
town        flat_type   price
BISHAN      4 ROOM      580000
BISHAN      5 ROOM      720000
YISHUN      4 ROOM      420000
PUNGGOL     4 ROOM      485000
```

Table B: MRT station data (key column: `town`)

```
town        nearest_mrt     distance_km
BISHAN      Bishan          0.3
YISHUN      Yishun          0.5
PUNGGOL     Punggol         0.4
SEMBAWANG   Sembawang       0.2
```

A *join on `town`* produces a new table where rows from A and B that share the same `town` value are combined into a single row:

```
town        flat_type   price    nearest_mrt   distance_km
BISHAN      4 ROOM      580000   Bishan        0.3
BISHAN      5 ROOM      720000   Bishan        0.3
YISHUN      4 ROOM      420000   Yishun        0.5
PUNGGOL     4 ROOM      485000   Punggol       0.4
```

Each row from Table A is augmented with the matching columns from Table B. If Table A has multiple rows with the same key (as BISHAN does above), each of them gets the same right-hand columns. If Table A has a key that does not appear in Table B, or vice versa, what happens depends on the *join type*.

FOUNDATIONS: The four join types

There are four standard join types. They differ only in what happens to non-matching rows.

Inner join. Keep only rows where the key exists in *both* tables. Rows with no match on either side are dropped. In our example, an inner join of HDB with MRT drops the SEMBAWANG row from Table B (because there are no HDB transactions for Sembawang in our example A) and would drop any HDB row whose town does not appear in Table B.

Use an inner join when you *require* the data from both sides — for example, "give me only HDB transactions for which we know the nearest MRT". If a town has no MRT data, you cannot answer the question for that town, so dropping is the right choice.

Left join. Keep all rows from the left (first) table. For rows where the key is not found in the right table, fill the right-hand columns with NULL. In our example, a left join of HDB with MRT keeps every HDB row. If there were an HDB row with town "JURONG ISLAND" (a real edge case — there are no HDB flats on Jurong Island, but bear with the hypothetical) and no matching row in Table B, the resulting row would have `nearest_mrt = null` and `distance_km = null`.

Use a left join when the left table is the "primary" dataset and the right table is "enrichment" — you want to add information where available without losing any original rows. This is by far the most common join type in practice.

Right join. Mirror image of left join: keep all rows from the right table, fill left-hand columns with NULL where needed. Polars supports right joins, but in practice you almost never use them; you can always rewrite `A.join(B, how="right")` as `B.join(A, how="left")`, which is usually clearer.

Outer (full) join. Keep all rows from both tables. Rows with no match on one side get NULLs on that side. In our example, an outer join keeps every HDB row *and* every MRT row, including SEMBAWANG (with null flat_type and null price). Use an outer join when you want the union of the two datasets — for example, when you are reconciling two sources and want to see what is in each but not the other.

> **Rule of thumb:** 90% of the time you want a left join. 9% of the time you want an inner join. The remaining 1% is split between right and outer joins. If you cannot articulate why you are using anything other than left, default to left and check the null counts afterward.

FOUNDATIONS: Polars `.join` syntax

The Polars method is `.join()`. The basic form is:

```
enriched = hdb.join(mrt_stations, on="town", how="left")
```

- The *left* table is the one you call `.join` on (`hdb`).
- The *right* table is the argument (`mrt_stations`).
- `on="town"` says "match rows where the `town` column is equal in both tables".
- `how="left"` says "keep all rows from the left table, fill missing right-side values with NULL".

If the key column has different names in the two tables, use `left_on` and `right_on`:

```
hdb.join(mrt_stations, left_on="town", right_on="area_name", how="left")
```

If the tables share multiple key columns, pass a list:

```
hdb.join(monthly_stats, on=["year", "town"], how="left")
```

After a join, the resulting DataFrame has all the columns from the left table plus the columns from the right table (except the join key, which is not duplicated). If both tables happen to have a column with the same name other than the join key, Polars will rename the right-side version with a suffix (default: `_right`), or you can specify your own suffix with the `suffix=` parameter.

FOUNDATIONS: Always check the null count after a left join

A left join never drops rows, but it can silently introduce NULLs wherever a match was missing. Always check how many nulls appeared in the right-side columns after a left join:

```
enriched = hdb.join(mrt_stations, on="town", how="left")
print(f"Nulls in distance_km: {enriched['distance_km'].null_count()}")
```

If the null count is zero, every row matched and you can proceed. If the null count is non-trivial (say, more than 1% of the rows), you have a choice to make:

1. **Fill the nulls with a sensible default** — for example, `.fill_null(0)` for a count column, or `.fill_null(999)` for a distance column where you want "far away" to be the default.
2. **Drop the rows** — if the downstream analysis cannot cope with nulls and the missing data are a small fraction.
3. **Investigate** — the nulls might indicate a data pipeline bug where the right-side dataset is incomplete or the join keys are misaligned (e.g., case mismatch: `"BISHAN"` vs `"Bishan"`).

Option 3 should always come first. Silent null-filling can hide bugs that bite you much later.

FOUNDATIONS: Predicting nulls with set operations

Before you join, you can predict exactly how many nulls you will introduce by comparing the distinct values of the join key in both tables. Python's built-in `set` type is perfect for this:

```
hdb_towns = set(hdb["town"].unique().to_list())
mrt_towns = set(mrt_stations["town"].unique().to_list())

matched = hdb_towns & mrt_towns     # intersection: towns in BOTH
unmatched = hdb_towns - mrt_towns   # difference: towns in HDB but not MRT

print(f"HDB towns: {len(hdb_towns)}")
print(f"MRT towns: {len(mrt_towns)}")
print(f"Matched (will join): {len(matched)}")
print(f"Unmatched (will be NULL after left join): {len(unmatched)}")
if unmatched:
    print(f"  Unmatched: {sorted(unmatched)}")
```

The `&` operator on two Python sets gives the intersection (elements in both). The `-` operator gives the difference (elements in the first but not the second). If `unmatched` is non-empty, you know exactly which towns will have NULL after the left join and can decide what to do about them before you run the join.

This is a healthy habit. It takes three extra lines of code and catches the kind of bug where a typo in a town name silently discards 10% of your data.

FOUNDATIONS: `import` and packages

Until now every import statement you have seen has pulled something from a file that came with the course (`shared`) or from Polars. Python's import system is more general than that: you can import from any installed package. Installed packages live in what Python calls *site-packages* and are usually installed with a tool like `pip` or `uv`.

A handful of imports you will see in this chapter:

```
import polars as pl         # alias pl for polars
import numpy as np          # alias np for numpy (used in Lesson 1.6)
from datetime import date   # import a specific name from a module
```

The three forms are:

- `import x` — make the entire module `x` available as `x.something`.
- `import x as y` — same but with a different name.
- `from x import y` — pull just `y` out of module `x` into the current namespace.

Use `import x as y` for top-level libraries (polars, numpy, pandas — the `pl`, `np`, `pd` aliases are universal). Use `from x import y` for specific classes and functions you want to use by name repeatedly.

THEORY: Join as a set-theoretic operation

Formally, a join is a restricted version of a *Cartesian product*. The Cartesian product of two tables with m and n rows produces $m \times n$ rows — every combination. A join filters the Cartesian product down to the rows where the join predicate holds. For an equi-join (join where the predicate is "key columns are equal"), the output has one row for every matching pair.

Inner join: $A \bowtie B = \{(a, b) : a \in A, b \in B, a.\text{key} = b.\text{key}\}$.

Left join: inner join $\cup \{(a, \text{null}) : a \in A, \forall b \in B : a.\text{key} \neq b.\text{key}\}$. That is, the inner-join results plus one row per unmatched left row, with NULL on the right.

Outer join: $A \bowtie_{\text{left}} B \cup B \bowtie_{\text{left}} A$, deduplicated. The symmetric case.

Modern databases (and Polars) do not compute the Cartesian product literally; they use hash joins or sort-merge joins for $O(m + n)$ performance instead of $O(mn)$. But the set-theoretic semantics are still how you should reason about what the output will look like.

The Kailash Context

The joins you will do in this lesson produce an enriched HDB table with spatial context (distance to MRT, school count). In Module 2 you will meet `FeatureStore`, the Kailash engine for versioning and retrieving feature sets. Feature stores are essentially long-lived joined tables: you compute your enriched dataset once, version it, store it, and retrieve it during training and serving. The join patterns you are learning here are exactly the ones a feature store applies under the hood when it materialises a feature group. This is why you are learning them first — everything that comes later is a specialised form of what you can do manually today.

Worked Example: Enriching HDB with MRT and School Data

Step 1: Load three tables

```
from __future__ import annotations

import polars as pl

from shared import MLFPDataLoader

loader = MLFPDataLoader()

hdb = loader.load("mlfp01", "hdb_resale.parquet")
mrt_stations = loader.load("mlfp_assessment", "mrt_stations.parquet")
schools = loader.load("mlfp_assessment", "schools.parquet")

print(f"HDB: {hdb.shape}")
print(f"MRT: {mrt_stations.shape}")
print(f"Schools: {schools.shape}")
```

The MRT table has one row per town with a pre-computed nearest station and distance. The schools table has one row per school, not per town — so it is at a different grain from the HDB table and will need aggregating before the join.

Step 2: Inspect each table before joining

Never join blindly. Look at each table first:

```
print(hdb.head(3))
print(mrt_stations.head(5))
print(schools.head(5))
```

Check the join keys — in this case, the `town` column — appear in both and have the same casing. The HDB table uses `"ANG MO KIO"` all-caps; the MRT table should also use all-caps. If it does not, you need a `.str.to_uppercase()` step on one side before joining.

Step 3: Predict nulls

```
hdb_towns = set(hdb["town"].unique().to_list())
mrt_towns = set(mrt_stations["town"].unique().to_list())
matched = hdb_towns & mrt_towns
unmatched = hdb_towns - mrt_towns

print(f"HDB towns: {len(hdb_towns)}")
print(f"MRT towns: {len(mrt_towns)}")
print(f"Matched: {len(matched)}")
print(f"Unmatched: {len(unmatched)}")
if unmatched:
    print(f"Unmatched towns: {sorted(unmatched)}")
```

If all 26 HDB towns appear in the MRT table, the output says “Unmatched: 0” and you can proceed confidently. If there are unmatched towns, you will know exactly which ones before you join.

Step 4: The left join itself

```
hdb_enriched = hdb.join(
    mrt_stations.select("town", "nearest_mrt", "distance_to_mrt_km"),
    on="town",
    how="left",
)
```

Two details worth pointing out. First, we `.select(...)` the right-hand table to bring only the columns we need. Without this, every column in the MRT table would be joined into the result, cluttering the output. Always select down the right-hand table to the columns you actually want.

Second, the result has the same number of rows as `hdb` (left join preserves left-side rows), but three extra columns: `nearest_mrt`, `distance_to_mrt_km`, and `town` which already existed. The join key `town` is not duplicated — Polars is smart about that.

Step 5: Pre-aggregate the schools table

The schools table has one row per school, but we want one row per town for the join. Aggregate first:

```
school_counts = schools.group_by("town").agg(
    pl.col("school_name").count().alias("school_count")
)

hdb_enriched = hdb_enriched.join(school_counts, on="town", how="left")
```

This is a common pattern: if the right-hand table is at a finer grain than the left (many schools per town, one row per transaction), aggregate the right-hand table up to the left table’s grain first, then join.

Step 6: Fill the nulls

After the left join, any town without a matching school count gets NULL. Fill with zero (meaning “no schools recorded”):

```
hdb_enriched = hdb_enriched.with_columns(pl.col("school_count").fill_null(0))
```

`.fill_null(0)` is a method on a column expression that replaces any null with the given value. Use it whenever you have a post-join null that you want to turn into a meaningful default. For counts, 0 is usually right. For distances where a null means "we did not find a match", a large sentinel like `999` makes "far away" the default. Always document what the fill value means.

Step 7: Verify and inspect

```
for col in ("nearest_mrt", "distance_to_mrt_km", "school_count"):
    nc = hdb_enriched[col].null_count()
    pct = nc / hdb_enriched.height
    print(f"  {col} nulls: {nc:,} ({pct:.1%})")
```

Expected: if the join keys matched cleanly, all three lines show 0 nulls (after `fill_null` on `school_count`). If `distance_to_mrt_km` shows a non-zero null count, the MRT table is missing some towns — time to investigate.

Step 8: Build a district-level summary with spatial context

Add `price_per_sqm`, then group by town with both price statistics and first-value spatial features:

```
hdb_enriched = hdb_enriched.with_columns(
    (pl.col("resale_price") / pl.col("floor_area_sqm")).alias("price_per_sqm")
)

district_summary = (
    hdb_enriched.group_by("town")
    .agg(
        pl.len().alias("total_transactions"),
        pl.col("resale_price").median().alias("median_price"),
        pl.col("resale_price").mean().alias("mean_price"),
        pl.col("price_per_sqm").median().alias("median_price_sqm"),
        pl.col("floor_area_sqm").median().alias("median_area_sqm"),
        # Spatial columns — same for every row in a town, so .first() is correct
        pl.col("nearest_mrt").first().alias("nearest_mrt"),
        pl.col("distance_to_mrt_km").first().alias("distance_to_mrt_km"),
        pl.col("school_count").first().alias("school_count"),
    )
    .sort("median_price", descending=True)
)
```

Why `.first()` for the spatial columns? Because every transaction in a given town has the *same* value for those columns (they were joined from a town-level table). If we used `.mean()` it would give the same answer, but `.first()` is clearer — it says "just grab one". `.mean()` would also be wasted computation.

Step 9: Compute correlations between price and spatial features

```
corr_mrt_price = district_summary.select(
    pl.corr("distance_to_mrt_km", "median_price")
).item()

corr_school_price = district_summary.select(
    pl.corr("school_count", "median_price")
).item()

print(f"Correlation: MRT distance ↔ median price: {corr_mrt_price:.3f}")
print(f"Correlation: school count ↔ median price: {corr_school_price:.3f}")
```

`pl.corr(a, b)` computes the Pearson correlation between two columns. `.item()` on the result extracts the single scalar value out of the one-row, one-column DataFrame that Polars returns.

Typical output:

```
Correlation: MRT distance ↔ median price: -0.58
Correlation: school count ↔ median price: 0.42
```

Negative correlation with distance-to-MRT means "closer MRT → higher price" — exactly what intuition suggests. Positive correlation with school count means "more schools → higher price". Neither is huge (an $|r|$ around 0.5 is "moderate"), and neither is causal. Both signals exist because the same underlying variable — how desirable the neighbourhood is — drives all three of price, MRT proximity, and school density. The deck discussion slide on Slide 77 makes this point: if you built a school in an undesirable

town, you would not make it desirable; you would just have a school in an undesirable town. Correlation is information, not causation. We will cover causal inference properly in Module 2.

Try It Yourself

Drill 1. Without running a join, predict how many rows an inner join of HDB and MRT would produce if there are 3 HDB towns missing from the MRT table. (Assume 487,293 HDB rows and that the missing towns account for 5% of them.)

Drill 2. Write an `if` statement that prints `"Most HDB rows matched"` if `matched / len(hdb_towns) > 0.95`, `"Some rows missing"` if between 0.8 and 0.95, and `"Many rows missing — investigate"` otherwise.

Drill 3. Join `district_summary` (from Step 8) with itself using `how="cross"` (Polars' Cartesian product). Filter the result to pairs where town_a comes alphabetically before town_b (`pl.col("town") < pl.col("town_right")`). For each pair, compute the absolute difference in `median_price`. What is the largest difference?

Drill 4. Pre-aggregate the schools table to compute the number of *primary schools only* per town (assume there's a `level` column with values like `"primary"`, `"secondary"`). Then left-join onto the HDB data. Fill nulls with 0.

Drill 5. Use `set` operations to find towns that appear in the schools table but not in the HDB table (there should be none — every Singapore town with a school should have HDB flats, unless it is a fully private district).

Cross-References

- **Lesson 1.5** will move into time-series analysis with window functions, which operate on the enriched joined dataset you built here.
- **Lesson 1.6** will visualise the relationship between MRT distance and price as a scatter plot.
- **Lesson 1.8** will perform a more complex multi-source merge when aligning monthly CPI, quarterly employment, and daily FX-rate data onto a common monthly spine.
- **Module 2**'s FeatureStore uses joins under the hood to materialise feature groups. The join semantics are exactly what you learned today.

Reflection

You should now be able to:

- Define inner, left, right, and outer joins in plain English and give an example of when each is appropriate.
- Write a Polars `.join()` call with `on`, `how`, and select-down-the-right-side.
- Predict the null-count outcome of a left join using Python `set` operations before running the join.
- Use `.fill_null()` to replace post-join NULLs with a sensible default.
- Pre-aggregate a right-hand table to the left table's grain before joining.
- Explain why `.first()` is the right aggregation for a column that was joined from a coarser-grained table.
- Compute a Pearson correlation between two columns with `pl.corr(a, b).item()`.

Drill answers

1. Inner join produces rows where both sides have a match. If 5% of HDB rows are for unmatched towns, they are dropped, leaving 95% × 487,293 ≈ 462,928 rows.

2.
```
match_rate = len(matched) / len(hdb_towns)
if match_rate > 0.95:
    print("Most HDB rows matched")
elif match_rate > 0.80:
    print("Some rows missing")
else:
    print("Many rows missing — investigate")
```

3.
```
pairs = (
    district_summary.select("town", "median_price")
    .join(district_summary.select("town", "median_price"), how="cross", suffix="_right")
    .filter(pl.col("town") < pl.col("town_right"))
    .with_columns(
        (pl.col("median_price") - pl.col("median_price_right")).abs().alias("diff")
    )
    .sort("diff", descending=True)
)
print(pairs.head(1))
```

4.
```
primary_counts = (
    schools.filter(pl.col("level") == "primary")
    .group_by("town")
    .agg(pl.len().alias("primary_school_count"))
)
hdb_with_primary = hdb.join(primary_counts, on="town", how="left").with_columns(
    pl.col("primary_school_count").fill_null(0)
)
```

5.
```
school_towns = set(schools["town"].unique().to_list())
hdb_towns = set(hdb["town"].unique().to_list())
only_in_schools = school_towns - hdb_towns
print(only_in_schools)
```

Lesson 1.5: Window Functions and Trends

Why This Matters

Aggregation answers "what is the median price per town?" but collapses every transaction into a single row per group. Sometimes you want the statistic without losing the row-level detail. "For each transaction, what was the median price in the same town in the same year?" That question cannot be answered with `group_by` alone — you would get one row per (town, year) and lose the per-transaction detail. The answer is a *window function*: a calculation that uses a group of related rows to compute a value for each row, without collapsing.

Window functions unlock time-series analysis. Rolling averages that smooth monthly noise. Year-over-year changes that reveal growth trends. Ranks within a group that identify top performers. These are all window calculations. And unlike `group_by`, window functions keep the original rows, so you can `.filter()` or `.sort()` the result just like any other DataFrame.

This lesson also introduces *lazy frames*, Polars' query-optimisation mode. You will not need lazy frames for correctness — eager evaluation works fine for everything in this chapter — but for large datasets lazy can be dramatically faster, and you should know it exists.

Core Concepts

FOUNDATIONS: What is a window function?

A window function computes a value for each row based on a set of surrounding rows called the *window*. The key distinction from `group_by`:

	`group_by`	window function
Output rows	one per group	same as input (one per row)
Output columns	aggregations	new derived columns
Keeps row-level detail	no	yes
Typical use	summary tables	feature engineering, trend detection

The Polars syntax for a window function is a column expression followed by `.over(partition_col)`:

```
df.with_columns(
    pl.col("price").mean().over("town").alias("town_mean_price")
)
```

Reading this: "for each row, compute the mean of `price` over all rows with the same `town`, and put the result in a new column `town_mean_price`". The partition column `"town"` defines the window — for a given row, the window is every other row with the same town value.

After running, every row has its original columns plus `town_mean_price`, which is the mean price of the town that row belongs to. Two rows for Bishan get the same `town_mean_price` (they share a window). A Yishun row gets a different value (different window).

FOUNDATIONS: Rolling averages — smoothing noisy time series

A rolling average (also called a moving average) replaces each value with the average of itself and the $k - 1$ values before it, where k is the window size. Rolling averages smooth out short-term noise to reveal underlying trends.

Consider a monthly median price per town. Any single month can be noisy — one unusual transaction or a slow sales month can make the median jump around. A 12-month rolling average smooths the noise: each month's smoothed value is the average of the last twelve months, so single-month anomalies are diluted by eleven other months' data.

In Polars:

```
monthly_prices.with_columns(
    pl.col("median_price_sqm")
    .rolling_mean(window_size=12)
    .over("town")
    .alias("rolling_12m_price_sqm")
)
```

Reading this: "for each row, compute the rolling mean of `median_price_sqm` with a window of 12 rows, partitioned by town". The partition by town is critical — without `.over("town")`, the rolling window would bleed across towns, averaging Bishan prices with Yishun prices. With `.over("town")`, each town gets its own independent rolling window.

The first 11 rows in each town will be NULL, because you need 12 rows to compute a 12-row window. This is normal. When you plot the result, the line starts 11 months in.

Choosing a window size. The trade-off is reactivity vs smoothness:

- **Small window (3 months):** reacts quickly to price changes, still has visible noise. Useful for detecting early market turns.
- **Medium window (6 months):** middle ground.
- **Large window (12 months):** very smooth, lags by 6 months. Useful for seeing the underlying trend without noise.

A common technique borrowed from financial technical analysis is to plot two rolling averages of different sizes on the same chart. When the short moving average (say 3 months) crosses above the long one (12 months), it signals an accelerating market. When it crosses below, it signals a slowing market. This is called the "golden cross" and "death cross". It is a crude signal — more useful as a visual aid than a trading rule — but the underlying idea that short-vs-long moving averages can encode trend direction is sound.

FOUNDATIONS: `shift` — compare to a previous row

`shift(n)` moves every value forward by n positions in the DataFrame, filling the first n rows with NULL. Combined with `.over("town")`, each town shifts independently.

Year-over-year change is the classic use case. "What is the median price this month compared to the same month last year?" Take the median price, shift it by 12 months, and compute the percentage difference:

```
monthly_prices.with_columns(
    pl.col("median_price_sqm").shift(12).over("town").alias("price_sqm_12m_ago"),
).with_columns(
    (
        (pl.col("median_price_sqm") - pl.col("price_sqm_12m_ago"))
        / pl.col("price_sqm_12m_ago")
        * 100
    ).alias("yoy_price_change_pct")
)
```

The first `with_columns` creates a helper column `price_sqm_12m_ago` — the value from 12 rows earlier. The second `with_columns` uses that helper to compute the percentage change. We split it into two steps for readability; you could inline it into one step, but the readability cost is not worth it.

The YoY calculation uses the standard formula:

$$\text{YoY \%} = \frac{\text{current} - \text{previous}}{\text{previous}} \times 100$$

A YoY of +5% means prices this month are 5% higher than the same month last year. A YoY of -2% means prices fell by 2%. Using the same-month comparison (shift by 12) removes the effect of monthly seasonality — comparing January to January is more meaningful than January to December, because January always has a different sales volume than December.

FOUNDATIONS: `rank` — order within a group

Rank assigns each row a position within its partition. `pl.col("yoy").rank(method="ordinal", descending=True)` assigns 1 to the highest YoY, 2 to the next, and so on. Ranks are useful for answering "which town had the nth highest growth this year?" without sorting and inspecting manually.

The `method` parameter controls how ties are broken:

- `"ordinal"` — each value gets a distinct rank (ties broken arbitrarily).
- `"dense"` — ties get the same rank, and the next value gets the next integer. `[1, 2, 2, 3]`.
- `"min"` — ties get the lowest possible rank, and the next value skips ahead. `[1, 2, 2, 4]`.
- `"average"` — ties get the average of their ranks. `[1, 2.5, 2.5, 4]`.

`"ordinal"` is the default for most purposes; use it when you want a strict 1-2-3 ordering. `"dense"` is useful when you want to count distinct values and assign them consecutive ranks regardless of duplicates.

FOUNDATIONS: Lazy frames — query optimisation

Every Polars operation so far has been *eager*: you wrote an expression, Polars computed it immediately, and the result was a DataFrame. Eager evaluation is simple and what you want for exploration.

For large data and complex pipelines, Polars offers a *lazy* mode. A lazy DataFrame (`LazyFrame`) is a description of a query, not the query's result. You chain operations as normal, and each operation adds to the query plan without running anything. When you finally call `.collect()`, Polars optimises the entire plan and then executes it.

```
result = (
    monthly_prices.lazy()
    .filter(pl.col("transaction_date") >= pl.date(2021, 1, 1))
    .drop_nulls("yoy_price_change_pct")
    .group_by("town")
    .agg(pl.col("yoy_price_change_pct").mean().alias("mean_yoy_pct"))
    .sort("mean_yoy_pct", descending=True)
    .collect()
)
```

The `.lazy()` call converts the eager `monthly_prices` into a LazyFrame. Everything between `.lazy()` and `.collect()` is query-plan construction, not execution. The `.collect()` at the end triggers the whole pipeline at once, with optimisations.

The optimisations include:

- **Predicate pushdown.** Filters are pushed down the query plan so that rows are eliminated as early as possible, before subsequent operations have to touch them. If you filter to 2021+ *after* a group_by, the optimiser can usually move the filter *before* the group_by, so the group_by processes fewer rows.
- **Projection pushdown.** Unused columns are dropped as early as possible. If the final output uses only two columns out of twenty, the optimiser removes the other eighteen before they reach the heavy operations.
- **Operation fusion.** Adjacent operations that can be combined into a single pass over the data are fused. Multiple `with_columns` calls become one. Multiple filters become one.

For small datasets (thousands of rows), lazy and eager take the same time. For large datasets (millions or more), lazy can be 2–10× faster. Always profile before you switch to lazy — premature lazy adds complexity without benefit.

You can also read files lazily with `pl.scan_csv` or `pl.scan_parquet`. These return a LazyFrame without reading the file; the file is only read when you `.collect()`, and only the columns you actually need are read from disk. For a wide CSV with a hundred columns where your query uses only three, scan can be 30× faster than read.

THEORY: When window functions fail

Window functions assume the ordering is correct. If you compute a rolling mean over a DataFrame that is not sorted by date, the "previous 12 rows" are whatever happens to come before in the current row order, which might not be the previous 12 months.

Always sort before computing a window function:

```
monthly_prices = monthly_prices.sort("town", "transaction_date")
```

Sort by partition key first (so rows in the same partition are adjacent), then by the ordering key within each partition. After sorting, `.over("town")` correctly partitions and the window functions compute what you expect.

There is also a subtlety with `shift`: Polars' `shift(n)` in a window context shifts by row count, not by time. If there are gaps in the monthly time series (a month with no transactions), `shift(12)` does *not* reach back exactly 12 calendar months — it reaches back 12 rows, which might be 13 or 14 calendar months. For the HDB dataset this is usually not a problem because every town has transactions every month, but be aware of the pitfall. For strictly time-based shifts, you can use `group_by_dynamic` or join the DataFrame to a date-shifted copy of itself.

ADVANCED: Rolling windows are a form of convolution

A rolling mean is equivalent to convolving the time series with a box filter (a kernel of all ones divided by the window size). This is why rolling means smooth data: they are low-pass filters that remove high-frequency noise. More sophisticated window functions (weighted rolling means, exponentially weighted moving averages) are just different convolution kernels.

If you want to emphasise recent data over old data, an exponentially weighted moving average gives higher weight to recent observations. Polars supports this with `.ewm_mean(alpha=...)`. The `alpha` parameter controls decay: higher `alpha` means faster forgetting of old data.

This is the discrete-time analogue of the IIR filters used in signal processing. The connection matters when you reach Module 5 and learn about temporal convolutional networks, which are exactly the same idea applied to raw neural network features instead of statistical features.

The Kailash Context

The time-series analysis you are learning here feeds directly into `FeatureEngineer` in Module 2 and `TrainingPipeline` in Module 3. Rolling means, YoY changes, and ranks within group are common features in any production ML pipeline — they are how models learn "is the price higher than it was last year?" rather than "is the price high?". The Kailash `FeatureEngineer` engine has built-in methods for generating lag features, rolling statistics, and rank features automatically, but the underlying computation is what you just learned in pure Polars. Knowing the manual form is what lets you debug when the automated version produces unexpected results.

Worked Example: HDB Price Trends by Town

Step 1: Prepare the time-series base table

```
from __future__ import annotations

import polars as pl

from shared import MLFPDataLoader

loader = MLFPDataLoader()
hdb = loader.load("mlfp01", "hdb_resale.parquet")

hdb = hdb.with_columns(
    pl.col("month").str.to_date("%Y-%m").alias("transaction_date"),
    pl.col("month").str.slice(0, 4).cast(pl.Int32).alias("year"),
    (pl.col("resale_price") / pl.col("floor_area_sqm")).alias("price_per_sqm"),
)

monthly_prices = (
    hdb.group_by("town", "transaction_date")
    .agg(
        pl.col("price_per_sqm").median().alias("median_price_sqm"),
        pl.col("resale_price").median().alias("median_resale_price"),
        pl.len().alias("transaction_count"),
    )
    .sort("town", "transaction_date")
)
```

The base table has one row per (town, month) with the median price metrics. Sorting by town then date is crucial for window functions — without this, the rolling windows will compute garbage.

Step 2: Rolling averages

```
monthly_prices = monthly_prices.with_columns(
    pl.col("median_price_sqm").rolling_mean(window_size=12).over("town").alias("rolling_12m_price_sqm"),
    pl.col("median_price_sqm").rolling_mean(window_size=3).over("town").alias("rolling_3m_price_sqm"),
)

print(monthly_prices.filter(pl.col("town") == "BISHAN").tail(18))
```

Inspect Bishan for the last 18 months. You should see the raw `median_price_sqm` bouncing around, the `rolling_3m_price_sqm` tracking it more smoothly, and the `rolling_12m_price_sqm` as a very smooth trend line. When `rolling_3m` rises above `rolling_12m`, the short-term average is above the long-term average — an accelerating market signal.

Step 3: Year-over-year change

```
monthly_prices = monthly_prices.with_columns(
    pl.col("median_price_sqm").shift(12).over("town").alias("price_sqm_12m_ago"),
)

monthly_prices = monthly_prices.with_columns(
    (
        (pl.col("median_price_sqm") - pl.col("price_sqm_12m_ago"))
        / pl.col("price_sqm_12m_ago")
        * 100
    ).alias("yoy_price_change_pct")
)

print(monthly_prices.filter(pl.col("town") == "BISHAN").tail(24).select(
    "transaction_date", "median_price_sqm", "price_sqm_12m_ago", "yoy_price_change_pct"
))
```

For the 2022–2023 period you should see YoY values around +10% to +20% — a large post-COVID rebound. For 2015–2017 you should see values near zero or slightly negative, reflecting the cooling-measure era.

Step 4: Find the trend leaders with lazy evaluation

This is where lazy frames pay for themselves. The multi-step pipeline below (filter, drop nulls, group_by, sort) has enough operations that the optimiser can meaningfully rewrite it:

```
recent_yoy = (
    monthly_prices.lazy()
    .filter(pl.col("transaction_date") >= pl.date(2021, 1, 1))
    .drop_nulls("yoy_price_change_pct")
    .group_by("town")
    .agg(
        pl.col("yoy_price_change_pct").mean().alias("mean_yoy_pct"),
        pl.col("yoy_price_change_pct").std().alias("std_yoy_pct"),
        pl.col("yoy_price_change_pct").max().alias("peak_yoy_pct"),
        pl.col("yoy_price_change_pct").min().alias("trough_yoy_pct"),
        pl.len().alias("months_of_data"),
    )
    .sort("mean_yoy_pct", descending=True)
    .collect()
)

print(recent_yoy.head(10))
```

Each town's `mean_yoy_pct` is its average annual appreciation rate since 2021. Towns at the top are the fastest-growing; towns at the bottom are the slowest. Compare `mean_yoy_pct` to `peak_yoy_pct` — towns with a high peak but a low mean had a single good month and then flatlined, while towns with similar peak and mean have been growing consistently.

Step 5: Classify towns into leaders, followers, and laggards

Use the mean and standard deviation of the growth rates to segment:

```
mean_growth = recent_yoy["mean_yoy_pct"].mean()
std_growth = recent_yoy["mean_yoy_pct"].std()

recent_yoy = recent_yoy.with_columns(
    pl.when(pl.col("mean_yoy_pct") > mean_growth + std_growth).then(pl.lit("leader"))
    .when(pl.col("mean_yoy_pct") < mean_growth - std_growth).then(pl.lit("laggard"))
    .otherwise(pl.lit("follower"))
    .alias("trend_category"),
    pl.col("mean_yoy_pct").rank(method="ordinal", descending=True).alias("growth_rank"),
)

print(f"Mean YoY growth (all towns): {mean_growth:.2f}%")
print(f"Std dev: {std_growth:.2f}%")
print(recent_yoy.group_by("trend_category").agg(pl.len().alias("count")))
```

Under a roughly normal distribution of growth rates, about 16% of towns should be more than 1 standard deviation above the mean (leaders) and 16% below (laggards). If the actual counts are far from those proportions, the growth distribution is skewed.

Try It Yourself

Drill 1. Add a `rolling_6m_price_sqm` column using a 6-month window. Plot or print the last 24 months for Queenstown along with the raw, 3m, 6m, and 12m rolling values.

Drill 2. Compute month-over-month (MoM) percentage change — shift by 1 instead of 12. For Bishan, print the 12 months with the largest absolute MoM changes. Are they clustered in time?

Drill 3. Rank towns by `peak_yoy_pct` (their single best month of growth) and print the top 5 along with the month it occurred. (Hint: first find the row where peak_yoy_pct was reached per town.)

Drill 4. Rewrite Step 4 in eager mode (no `.lazy()` / `.collect()`). Confirm the output is identical. Time both versions using Python's `time.perf_counter()`; which is faster?

Drill 5. Compute a three-year *compound* annual growth rate (CAGR) per town instead of the one-year YoY. CAGR = (ending / beginning)^(1/years) - 1. Which town has the highest 3-year CAGR?

Cross-References

- **Lesson 1.6** will visualise the rolling averages and YoY trends as line charts — the natural chart type for time-series data.
- **Lesson 1.8** will use rolling features in the taxi-trip cleaning pipeline (rolling average trip duration per hour of day).
- **Module 2**'s `FeatureEngineer` automates rolling-feature generation at scale.
- **Module 5** will reintroduce rolling windows as a form of convolution when you meet TCNs (temporal convolutional networks).

Reflection

You should now be able to:

- Explain the difference between `group_by` and window functions, and decide which to use for a given question.
- Write a Polars `.rolling_mean()` over a partition column with a specified window size.
- Write a `.shift(n).over(col)` to compute a value from *n* rows earlier within each partition.
- Compute YoY percentage change using shift and arithmetic.
- Explain what lazy evaluation means, what `.collect()` does, and what optimisations the Polars query planner performs.
- Use `rank` with different tie-breaking methods and understand what each produces.
- Classify values into categories using `pl.when().then()` combined with the mean and standard deviation.

Drill answers

1. ```
   monthly_prices = monthly_prices.with_columns(
       pl.col("median_price_sqm").rolling_mean(window_size=6).over("town").alias("rolling_6m_price_sqm")
   )
   print(monthly_prices.filter(pl.col("town") == "QUEENSTOWN").tail(24))
   ```
   ```

2.
```
monthly_prices = monthly_prices.with_columns(
    pl.col("median_price_sqm").shift(1).over("town").alias("price_sqm_1m_ago"),
)
monthly_prices = monthly_prices.with_columns(
    ((pl.col("median_price_sqm") - pl.col("price_sqm_1m_ago")) / pl.col("price_sqm_1m_ago") *
        100).alias("mom_pct")
)
top = (
    monthly_prices.filter(pl.col("town") == "BISHAN")
    .drop_nulls("mom_pct")
    .with_columns(pl.col("mom_pct").abs().alias("abs_mom"))
    .sort("abs_mom", descending=True)
    .head(12)
)
print(top)
```

3.
```
peaks = (
    monthly_prices.drop_nulls("yoy_price_change_pct")
    .sort("yoy_price_change_pct", descending=True)
    .group_by("town")
    .agg(
        pl.col("yoy_price_change_pct").first().alias("peak_yoy"),
        pl.col("transaction_date").first().alias("peak_month"),
    )
    .sort("peak_yoy", descending=True)
    .head(5)
)
print(peaks)
```

4. Lazy is typically 10–30% faster on this query because the filter-then-drop_nulls-then-group_by chain can be rewritten; on smaller datasets the difference is small.

5.
```
cagr = (
    monthly_prices
    .group_by("town")
    .agg(
        pl.col("median_price_sqm").first().alias("start"),
        pl.col("median_price_sqm").last().alias("end"),
        pl.col("transaction_date").first().alias("first_date"),
        pl.col("transaction_date").last().alias("last_date"),
    )
    .with_columns(
        ((pl.col("end") / pl.col("start")) ** (1/3) - 1).alias("cagr_3y")
    )
    .sort("cagr_3y", descending=True)
)
```

Lesson 1.6: Data Visualisation

Why This Matters

Anscombe's quartet is a set of four datasets, each with eleven points. Every dataset has the same mean of x, the same mean of y, the same variance of x, the same variance of y, the same correlation, and the same regression line. By the numbers they are indistinguishable. By the *pictures* they are completely different: one is a clean linear relationship, one is a smooth curve, one is a line with a single outlier that dominates the correlation, and one is a vertical line of points plus a single outlier at the far right. The lesson from Anscombe's quartet is simple and severe: *never trust a summary statistic you have not plotted*.

Every descriptive statistic you learned in Lessons 1.1 through 1.5 is a compression. Compressions lose information. A mean of $500,000 tells you nothing about whether the underlying distribution is a tight bell or a bimodal mess. A correlation of 0.6 could mean a clean linear trend or a parabola with a single outlier. The only way to see what the data is actually doing is to plot it.

This lesson is about choosing the right chart for the right question and building it without introducing distortions. You will learn five chart types — histogram, scatter, bar, heatmap, line — each with a specific job. You will learn the Gestalt principles that govern what the eye can parse quickly and what it cannot. You will learn which chart types to avoid (3D charts, pie charts in most circumstances) and why. And you will build every one of them with `ModelVisualizer`, the Kailash engine that wraps Plotly behind a consistent API.

Core Concepts

FOUNDATIONS: Why visualise?

Tables are precise. Charts are fast. A well-chosen chart communicates a pattern in milliseconds that would take a minute to extract from a table. The human visual system is the most parallel sensor on your body — you can spot an outlier in a scatter of ten thousand points instantly, whereas extracting that same outlier from a table would require scanning ten thousand rows.

The trade-off is precision. A chart cannot tell you that the exact maximum value was $1,248,532.50; a table can. So charts and tables are complementary, not substitutes. Use charts to find the patterns; use tables to nail down the numbers once you know which ones matter.

The rule: **plot first, compute later.** When you are exploring a new dataset, the first thing you should do after loading it is plot the distribution of every numeric column. Only then compute statistics. This prevents the entire category of error where you report a mean of a bimodal distribution as though it were meaningful.

FOUNDATIONS: Chart selection by data question

Different questions need different chart types. A rough mapping:

Question	Chart type
What is the distribution of X?	histogram or density plot
How does Y relate to X?	scatter plot
How does Y differ across categories?	bar chart
How does Y change over time?	line chart
Are pairs of variables correlated?	heatmap
How does the distribution of Y vary across groups?	box plot or violin plot
What is the composition of X? (parts of a whole)	stacked bar, pie (with warnings)

If your question does not fit one of these cleanly, chances are the question needs to be broken into parts. “How do HDB prices vary by town over time?” is two questions — across towns (bar) and over time (line) — and you will usually want two charts, or a single line chart with one line per town.

FOUNDATIONS: Histograms and distributions

A histogram divides a continuous range of values into buckets (*bins*) and counts how many values fall into each bucket. The result is a bar chart where the x-axis is the value and the y-axis is the count. Histograms show the *shape* of a distribution:

- A **symmetric bell** — most values near the centre, tapering equally on both sides. Temperature, height, measurement error. Mean ≈ median.
- A **right-skewed** (long right tail) — most values are low with a few very high ones. Income, house prices, city populations. Mean > median.
- A **left-skewed** (long left tail) — most values are high with a few very low ones. Age at death for a mortality dataset, test scores near a ceiling. Mean < median.
- A **bimodal** — two distinct humps. The HDB flash crash distribution. The presence of two subpopulations mixed together.
- A **uniform** — all values equally likely. Dice rolls, randomly sampled timestamps.

The *number of bins* matters. Too few bins obliterates detail — a 5-bin histogram turns everything into a rough pyramid. Too many bins makes the chart noisy — a 500-bin histogram on 1,000 data points looks like random static. Start with 30–50 bins for a few thousand data points and adjust from there. There is no universally correct rule; inspect and iterate.

FOUNDATIONS: Scatter plots and relationships

A scatter plot puts one variable on the x-axis and another on the y-axis, with one dot per observation. It is the standard tool for asking “does X predict Y?”

What to look for:

- **Linear trend.** A cloud of points that roughly follows a straight line indicates a linear relationship. The tightness of the cloud around the line tells you how strong the relationship is.

- **Non-linear trend.** A cloud that follows a curve (parabola, exponential, logarithmic) indicates a non-linear relationship. Pearson correlation will understate these — the correlation could be near zero even though there is a strong relationship.
- **Heteroscedasticity.** If the cloud fans out (gets wider) as X increases, the relationship has non-constant variance. This matters for linear regression assumptions; we cover it in Module 2.
- **Outliers.** Points far from the main cloud are outliers. Sometimes they are real rare events; sometimes they are data errors. A scatter plot makes them instantly visible in a way no summary statistic does.
- **Clusters.** Two or three dense regions of points with sparse areas between them suggest subpopulations. This is the scatter-plot equivalent of a bimodal histogram.

For datasets with more than a few thousand points, raw scatter plots become unreadable (the dots overlap into a blob). Solutions: *sample* the data to a few thousand points, *use transparency* (alpha blending) so dense regions appear darker, or *bin the scatter* into a 2D histogram (also called a *hexbin plot*).

FOUNDATIONS: Bar charts for categorical comparison

A bar chart has one bar per category with the bar height (or length) proportional to a value. Bar charts are the right tool for "show me this metric for each category" — median price per town, count per flat type, revenue per product.

Vertical vs horizontal. Vertical bar charts (categories on x-axis, values on y-axis) are the default. But when the category labels are long (like "BUKIT BATOK EAST AVENUE"), vertical bars force the labels to tilt or wrap, and the eye has to work harder. Horizontal bar charts with the labels on the y-axis and the bars extending to the right make long labels trivially readable. Use horizontal bars when labels are long or when there are many categories.

Always sort. An unsorted bar chart makes the reader work to find the maximum. A chart sorted descending by value shows the ranking immediately. The only exception is when the x-axis has a natural order (months of the year, age groups) that sorting by value would break.

Start bars at zero. A bar chart with a truncated y-axis (starting at, say, $400,000 instead of $0) visually exaggerates small differences. Three bars of heights 420, 440, and 460 look like massive differences on a truncated axis and trivial ones on a zero-based axis. For *comparison* purposes, the zero baseline is part of the chart's honesty. Exception: when the differences are meaningful but small compared to the absolute value (say, you are comparing 99.8% vs 99.9% accuracy), a truncated axis is appropriate — but document it explicitly.

FOUNDATIONS: Line charts for time series

A line chart connects dots with line segments in the x-axis order. The implicit assumption is that the x-axis has a natural ordering — typically time. Line charts are the right tool for "how does this value change over time?"

What to look for:

- **Trend.** An overall upward or downward direction over the full range.
- **Seasonality.** A repeating pattern with a fixed period (weekly, monthly, yearly). The HDB transaction volume has a clear annual seasonality — more transactions in some months than others.
- **Level shifts.** Sudden jumps where the series moves to a new baseline and stays there. Indicates a regime change: a policy update, a recession, a product launch.
- **Outliers.** Spikes that return to baseline. Indicates a one-off event.

Multiple lines on one chart work well when you are comparing a few series — typically up to 5–7 lines. Beyond that the chart becomes a "spaghetti plot" and no single line is readable. For many series, use *small multiples* instead — a grid of small charts, one per series, all with the same axes.

FOUNDATIONS: Heatmaps for correlation

A heatmap is a grid of coloured cells where colour encodes a numeric value. The two main uses are correlation matrices (showing Pearson correlation between every pair of variables in a dataset) and confusion matrices (showing classification errors — you will meet these in Module 3).

For correlation matrices, use a *diverging* colour scale: one colour for negative, another for positive, white in the middle at zero. `RdBu_r` (red-blue reversed) is a standard choice — blue for positive, red for negative, white for zero. This makes the sign of the correlation pre-attentively visible; you do not need to read the numbers to see which pairs are positively or negatively correlated.

Always cap the colour scale at -1 and +1 (`zmin=-1, zmax=1`). Without capping, the colour scale would stretch to fit the data, which makes different heatmaps incomparable and can wash out the meaning.

The diagonal of a correlation matrix is always 1 (every variable correlates perfectly with itself). The matrix is symmetric across the diagonal. You can show only the upper or lower triangle to reduce redundancy, but most tools do not do this by default.

FOUNDATIONS: Gestalt principles

The Gestalt principles are a set of rules about how the human visual system groups elements. They come from early 20th century psychology but apply directly to chart design. The five that matter most for you:

- **Proximity.** Elements close to each other are perceived as belonging to the same group. Use small gaps between related bars, larger gaps between unrelated ones.
- **Similarity.** Elements that share a visual property (colour, shape, size) are perceived as belonging together. Use the same colour for the same series across multiple charts. Do not use colour randomly.
- **Closure.** The visual system fills in gaps to see complete shapes. A line chart with a missing segment is still interpreted as a continuous line. But if gaps are meaningful (missing data), make them visually distinct.
- **Continuity.** Smooth continuous lines draw the eye across a chart. This is why line charts work: the continuous line guides you through the temporal progression.
- **Connection.** Elements connected by a line are perceived as strongly related. A scatter plot with a fitted line emphasises the relationship more than the cloud alone.

The practical rule: make the elements you want the reader to compare look *similar* to each other (same colour, same style), and make the elements you want them to distinguish look *different*. Every deviation from that rule is a potential source of confusion.

FOUNDATIONS: Charts to avoid

3D charts. 3D bar charts, 3D pie charts, 3D scatter plots — all of them. The third dimension adds no information and introduces perspective distortion: bars in the back look smaller than bars in the front even if they represent the same value. The reader has to mentally correct for the perspective, which defeats the purpose of a chart. Exception: genuinely three-dimensional data (a fitted surface over two input variables, a 3D point cloud for spatial analysis). Even then, 2D projections are usually clearer.

Pie charts, almost always. Pie charts require the reader to compare angles, which the human visual system does poorly compared to comparing lengths. A horizontal bar chart communicates the same information (parts of a whole) more precisely. Pie charts are tolerable only when there are 2–3 slices and the approximate proportions are more important than the exact values. For anything else, use a bar chart.

Dual y-axis line charts. A line chart with two lines on two different y-axes — one axis on the left, another on the right — implies a relationship between the two series that may not exist. The reader's eye sees the lines crossing or diverging and infers meaning from the visual relationship, but the crossing is an artifact of how the axes were scaled. Better: use two separate charts stacked vertically, with aligned x-axes.

Truncated y-axes on bar charts. Covered above. A bar chart should start at zero unless you explicitly label the axis as truncated.

FOUNDATIONS: Z-pattern reading

Western readers scan visual content in a Z pattern — top-left, across to top-right, down-left to bottom-left, across to bottom-right. Chart layout should respect this. Put the most important information in the top-left (title, key takeaway). Put secondary information in the top-right and bottom-left. Put least important information in the bottom-right. For dashboards, arrange charts so the "headline" chart is top-left and the supporting detail flows along the Z.

This is more about dashboard layout than single-chart design, but when you build reports in Lesson 1.8 you will apply it.

The Kailash Engine: ModelVisualizer

`ModelVisualizer` is the Kailash ML engine for producing charts. It wraps Plotly under a consistent API so that every chart type uses the same calling convention. You do not have to remember that Plotly's histogram is `px.histogram` while its scatter is `px.scatter` and its bar is `go.Bar`; you just call methods on a `ModelVisualizer` instance:

```
from kailash_ml import ModelVisualizer

viz = ModelVisualizer()
fig = viz.histogram(data=hdb, column="resale_price", bins=40, title="...")
fig.write_html("histogram.html")
```

Every ModelVisualizer method returns a Plotly `Figure` object. You can:

- **Display inline in Jupyter:** `fig.show()` renders the chart in the notebook.
- **Export to HTML:** `fig.write_html("name.html")` saves a standalone HTML file with the chart embedded. The HTML file is interactive — hover, zoom, pan all work — and can be emailed or posted.
- **Customise further:** `fig.update_layout(...)` lets you tweak titles, axes, colours, margins, and anything else Plotly supports. The underlying object is a real Plotly figure; ModelVisualizer just made the initial construction easy.

The five ModelVisualizer methods you will use most in Module 1:

Method	Chart type	Typical use
`histogram`	histogram	distribution of a numeric column
`scatter`	scatter plot	relationship between two numeric columns
`feature_importance` or `metric_comparison`	horizontal bar chart	comparison across categories
`confusion_matrix`	heatmap	correlation or confusion matrix
`training_history`	line chart	time series or training curves

Some method names hint at their origins in ML pipelines (`training_history` was designed for plotting training loss curves, `confusion_matrix` for classification evaluation), but they are general tools — a line chart is a line chart regardless of what the lines represent.

Polars DataFrames work directly as input. You do not need to convert to pandas. This is the "polars-native" principle at work.

Worked Example: Six HDB Charts

Step 0: Imports and data prep

```
from __future__ import annotations

import polars as pl
from kailash_ml import ModelVisualizer

from shared import MLFPDataLoader

loader = MLFPDataLoader()
hdb = loader.load("mlfp01", "hdb_resale.parquet")

hdb = hdb.with_columns(
    (pl.col("resale_price") / pl.col("floor_area_sqm")).alias("price_per_sqm"),
    pl.col("month").str.slice(0, 4).cast(pl.Int32).alias("year"),
    pl.col("month").str.to_date("%Y-%m").alias("transaction_date"),
)

viz = ModelVisualizer()
```

Step 1: Histogram of resale prices

```
fig_hist = viz.histogram(
    data=hdb,
    column="resale_price",
    bins=40,
    title="HDB Resale Price Distribution",
)
fig_hist.write_html("hdb_price_histogram.html")
```

Open the HTML file in a browser. You will see a right-skewed distribution: most transactions clustered in the $350k–$600k range, tapering to a long tail of expensive transactions reaching past $1.2 million. The peak of the histogram (the *mode*, roughly) is around $450k–$500k. The mean is pulled higher than the median by the right tail.

This single chart already tells you more than the `.describe()` output from Lesson 1.1. You now *see* the shape, not just read the numbers.

Step 2: Scatter plot of price vs floor area

Scatter of 487,000 points is unusable — the dots overlap into a solid blob. Sample first:

```
hdb_sample = hdb.sample(n=5_000, seed=42)

fig_scatter = viz.scatter(
    data=hdb_sample,
    x="floor_area_sqm",
    y="resale_price",
    title="HDB Resale Price vs Floor Area",
)
fig_scatter.write_html("hdb_scatter.html")
```

`hdb.sample(n=5_000, seed=42)` picks 5,000 rows at random. The `seed=42` makes the sample reproducible — running the code twice with the same seed produces the same sample. Reproducibility is a lifesaver when you are debugging and want to be sure a result you saw earlier is not an artifact of random sampling.

The result is a scatter that clearly shows larger flats costing more, with wide vertical spread at every flat size. The spread reflects the influence of variables other than size: town, floor level, remaining lease, flat model. Floor area alone explains maybe 40–50% of price variance. A linear regression on this scatter would produce a reasonable slope, but a wide confidence interval — which is exactly what a scatter plot communicates without any regression.

Step 3: Bar chart of median price by town

First aggregate, then plot:

```
district_prices = (
    hdb.group_by("town")
    .agg(
        pl.col("resale_price").median().alias("median_price"),
        pl.len().alias("transaction_count"),
    )
    .sort("median_price", descending=True)
)

price_by_town = {
    town: {"Median Price (S$)": price}
    for town, price in zip(
        district_prices["town"].to_list(),
        district_prices["median_price"].to_list(),
    )
}

fig_bar = viz.metric_comparison(price_by_town)
fig_bar.update_layout(title="Median HDB Price by Town")
fig_bar.write_html("hdb_bar.html")
```

The `metric_comparison` method was designed for comparing metrics across multiple models — it takes a dict of `{model_name: {metric_name: value}}`. We repurpose it as a bar chart by treating each town as a "model" and each value as its metric. The output is a horizontal bar chart sorted by value, which is what we want.

The Python idiom on lines 5–10 is a *dict comprehension*: it builds a dictionary in one expression. Read it as "for each (town, price) pair in the zipped lists, create a key `town` with value `{"Median Price (S$)": price}`". Dict comprehensions are to dicts what list comprehensions are to lists; you will see them often.

Step 4: Correlation heatmap

Select the numeric columns and compute the correlation matrix with numpy:

```
import numpy as np
import plotly.graph_objects as go

numeric_cols = ["resale_price", "floor_area_sqm", "price_per_sqm", "year"]
hdb_numeric = hdb.select(numeric_cols).drop_nulls()

np_data = hdb_numeric.to_numpy()
corr_matrix = np.corrcoef(np_data, rowvar=False)

fig_heatmap = go.Figure(data=go.Heatmap(
    z=corr_matrix.tolist(),
    x=numeric_cols,
```

```
    y=numeric_cols,
    colorscale="RdBu_r",
    zmin=-1, zmax=1,
    text=[[f"{corr_matrix[i, j]:.3f}" for j in range(len(numeric_cols))] for i in range(len(numeric_cols))],
    texttemplate="%{text}",
))
fig_heatmap.update_layout(title="Pearson Correlation Matrix — HDB Features", width=600, height=500)
fig_heatmap.write_html("hdb_heatmap.html")
```

Two new libraries. `numpy` is Python's array library; its `np.corrcoef` computes the full Pearson correlation matrix in one call. `plotly.graph_objects as go` is the lower-level Plotly API, which you use when ModelVisualizer does not have a direct method for what you want. `ModelVisualizer.confusion_matrix` takes `y_true` and `y_pred` arrays for classification, not a raw matrix, so for a correlation heatmap we drop down to Plotly directly. This is fine — ModelVisualizer is a convenience wrapper, not a wall.

The resulting heatmap has a diagonal of all 1's (every variable correlates with itself) and off-diagonal values you can read visually. `resale_price` vs `floor_area_sqm` should be strongly positive (dark blue). `floor_area_sqm` vs `price_per_sqm` is often near zero or slightly negative. `year` vs `resale_price` is positive (prices have been rising over time).

Step 5: Line chart of annual median price

Pick the top 5 most-transacted towns and plot their annual medians as separate lines:

```
top_5_towns = (
    district_prices.sort("transaction_count", descending=True)["town"].head(5).to_list()
)

annual = (
    hdb.filter(pl.col("town").is_in(top_5_towns))
    .group_by("year", "town")
    .agg(pl.col("resale_price").median().alias("median_price"))
    .sort("year")
)

years = sorted(annual["year"].unique().to_list())
price_series = {}
for town in top_5_towns:
    town_data = annual.filter(pl.col("town") == town).sort("year")
    lookup = dict(zip(town_data["year"].to_list(), town_data["median_price"].to_list()))
    price_series[town] = [float(lookup.get(y, 0)) for y in years]

fig_line = viz.training_history(
    metrics=price_series,
    x_label="Year",
    y_label="Median Resale Price (S$)",
)
fig_line.update_layout(title="Annual Median HDB Price — Top 5 Towns")
fig_line.write_html("hdb_line.html")
```

The chart reveals *divergence*: towns that started similarly in the earliest year grow apart over time. Premium towns pull ahead; peripheral towns grow more slowly. A crossing (one town overtaking another) is a notable event that deserves attention.

Step 6: All charts in a directory

Saving six HTML files to the current directory gives you an informal dashboard you can open in a browser. For a proper dashboard, you would combine them into a single HTML page with layout — that is what Lesson 1.8 will do.

Try It Yourself

Drill 1. Create a histogram of `price_per_sqm` instead of `resale_price`. Is the shape different from the resale price histogram? Why?

Drill 2. Create a scatter plot of `floor_area_sqm` vs `price_per_sqm`. What do you see? Is there a positive or negative trend, and what does the sign tell you about the relationship between flat size and per-unit price?

Drill 3. Build a bar chart of the *count* of transactions per flat type (not median price). Use `viz.metric_comparison` with `{flat_type: {"count": n}}`. Which flat type is most common?

Drill 4. Create a line chart with one line per flat type showing annual median price. Use the same pattern as Step 5 but group by `(year, flat_type)` instead of `(year, town)`.

Drill 5. Build a correlation heatmap for these columns: `resale_price`, `floor_area_sqm`, `price_per_sqm`, `year`, `lease_commence_date`. What is the correlation between `year` and `lease_commence_date`? Does it surprise you?

Cross-References

- **Lesson 1.7** will use ModelVisualizer for diagnostic charts generated by DataExplorer — the engine produces its own set of standard charts automatically.
- **Lesson 1.8** will combine multiple charts into an end-to-end HTML report.
- **Module 2** will reuse these chart types for feature analysis — distribution plots per feature, correlation heatmaps of engineered features, scatter plots of feature vs target.
- **Module 3** will introduce `training_history` for its original purpose: plotting loss curves during model training.

Reflection

You should now be able to:

- Choose an appropriate chart type for each of the six common data questions.
- Instantiate a `ModelVisualizer` and call its `histogram`, `scatter`, `metric_comparison`, and `training_history` methods with Polars DataFrames.
- Export charts as standalone HTML with `fig.write_html()`.
- Explain the five Gestalt principles in your own words and apply them to critique a chart.
- Identify the most common misleading chart designs (3D, pie charts, dual y-axes, truncated y-axis on bar charts).
- Sample a large dataset before plotting a scatter plot to keep the chart readable.

Drill answers

1. ```
   fig = viz.histogram(data=hdb.filter(pl.col("price_per_sqm").is_not_null()), column="price_per_sqm",
           bins=40)
   ```

   The price-per-sqm histogram is less skewed than the price histogram, because normalising by area removes the size effect that creates the long tail.

2. ```
   fig = viz.scatter(data=hdb.sample(5_000, seed=42), x="floor_area_sqm", y="price_per_sqm")
   ```

   A weak negative trend — bigger flats have slightly lower price per sqm, because some “premium per sqm” comes from fixed costs (bathroom, entrance) that small flats amortise over fewer square metres.

3. ```
   flat_counts = hdb.group_by("flat_type").agg(pl.len().alias("count")).sort("count", descending=True)
   data = {ft: {"count": float(c)} for ft, c in zip(flat_counts["flat_type"].to_list(),
           flat_counts["count"].to_list())}
   viz.metric_comparison(data).show()
   ```

   4-ROOM is most common with roughly 40% of all transactions.
4. Same pattern as Step 5 with `(year, flat_type)` group_by.
5. `year` and `lease_commence_date` are strongly positively correlated — newer transactions tend to involve more recently built flats (newer flats tend to be in newer towns that only started transacting later). The correlation is around 0.3–0.5 depending on the dataset. Not surprising on reflection but easy to miss.

Lesson 1.7: Automated Data Profiling

Why This Matters

Every time you open a new dataset, you do the same things. Check the shape. Check the columns. Check the types. Count the nulls. Look at a few rows. Compute summary statistics. Look for outliers. Check for duplicates. Plot the distributions. Compute correlations. Look for columns that are suspiciously constant or suspiciously unique. This is a rigid, repetitive checklist that should never be done by hand — at least not after you have done it manually a dozen times to know what you are looking at.
   ```

`DataExplorer` is the Kailash engine that automates this checklist. It takes a DataFrame, runs the full battery of profile checks in parallel, and returns a structured result you can inspect programmatically or render as an HTML report. More importantly, it emits *alerts*: typed, severity-tagged messages that tell you what looks wrong. An alert with type `"high_skewness"` on column `"fare"` is a concrete, actionable piece of information — you know exactly what to look at and what the suggested fix is.

This lesson is where you stop doing by hand what the engine can do for you. You will configure `AlertConfig` with thresholds appropriate for your domain, run `DataExplorer.profile()` on a deliberately messy economic-indicators dataset, interpret each alert, and compare two time-period slices of the same data to detect distribution drift. You will also meet `try` / `except` for error handling, and `async` / `await` for the first time in the course.

Core Concepts

FOUNDATIONS: Why automate profiling?

The profiling you did manually in Lessons 1.1 through 1.6 works, and there is no substitute for doing it once so you understand what each piece means. But there are two reasons to automate it after the first time.

Consistency. A manual checklist is only as reliable as the person running it. Every dataset you miss a check on is a potential bug. An automated profiler runs the same checks on every dataset, so you never forget to look at skewness or cardinality.

Alerts as a decision layer. Raw statistics are information; alerts are decisions. A mean of 3.5 is information. An alert saying "column 'fare' has skewness 4.2, above the threshold of 2.0 — recommend log-transform or winsorisation" is a decision. When you are moving fast through many datasets, the decision layer is what matters. You cannot stop to manually evaluate ten statistics per column for a dataset with fifty columns; the alert layer collapses five hundred statistics into the ten or twenty that actually need attention.

DataExplorer does both: it computes the full statistics (so you can dig into any specific number if needed) and it emits alerts (so you know which numbers to dig into first). The rest of this lesson is about understanding the alerts and trusting them appropriately.

FOUNDATIONS: The eight alert types

DataExplorer supports eight alert categories out of the box. Each has a configurable threshold and a typical remediation.

Alert type	What it detects	Typical fix
`high_nulls`	column with more than X% missing values	impute, drop rows, or drop column
`high_zeros`	column with more than X% zero values	check whether zeros are real or missing-coded-as-zero
`high_skewness`	column with absolute skewness above X	log-transform, winsorise, or remove outliers
`high_cardinality`	column with unique-value ratio above X	bin the values, use target encoding, or drop
`constant`	column with one or fewer unique values	drop the column (no information)
`high_correlation`	pairs of columns with correlation above X	drop one of the pair to avoid multicollinearity
`duplicates`	more than X% of rows are exact duplicates	`.unique()` / `.drop_duplicates()`
`imbalanced`	categorical column where minority class is below X	oversample, undersample, or class weights

Each alert includes a `severity` (usually `"info"`, `"warning"`, or `"error"`), a column name (or column pair for correlation), a value (the computed number that crossed the threshold), and optionally a recommendation string. The alert object is a plain dictionary you can iterate over in Python.

FOUNDATIONS: AlertConfig — tuning thresholds for your domain

Out-of-the-box thresholds are reasonable defaults for "typical tabular ML data", but no dataset is perfectly typical. An economic-indicators dataset with macroeconomic variables has *expected* high correlations (CPI and employment are always correlated); flagging them as problems every time would flood the output with false alarms. A taxi dataset has *expected* high skewness in fare and distance (most rides are short and cheap, a few are long and expensive); a default threshold of 2.0 might flag every column.

`AlertConfig` lets you tune each threshold:

```
from kailash_ml.engines.data_explorer import AlertConfig
```

```
alert_config = AlertConfig(
    high_correlation_threshold=0.95,    # only flag near-perfect collinearity
    high_null_pct_threshold=0.10,       # allow up to 10% nulls before alerting
    constant_threshold=1,                # flag columns with <= 1 unique value
    high_cardinality_ratio=0.95,        # flag columns where >95% of values are unique
    skewness_threshold=3.0,              # only flag severe skew
    zero_pct_threshold=0.30,             # allow up to 30% zeros
    imbalance_ratio_threshold=0.05,     # flag minority class below 5%
    duplicate_pct_threshold=0.05,       # flag duplicates above 5%
)
```

Every threshold is a *deliberate choice*, not a default. When you configure a profiler for a new domain, you should be able to justify each number. "Why 0.95 for correlation?" — because CPI and unemployment are structurally correlated at around 0.85; I only want to catch *near-perfect* collinearity, which is a sign of data pipeline bugs. "Why 3.0 for skewness?" — because crisis periods (GFC, COVID) create genuine outliers in macro data that would trigger alerts at 2.0 but are real information, not errors.

The thresholds are not set-and-forget. After the first profile run, look at which alerts fired and which did not. If you are getting too many false alarms, relax the relevant threshold. If you are missing problems you can see by eye, tighten them. Tuning AlertConfig is an iterative process, much like tuning any other hyper-parameter.

FOUNDATIONS: `async` and `await` — just enough to use them

`DataExplorer.profile()` is an *asynchronous* function — a function marked with `async def` that returns a *coroutine* rather than a direct value. You cannot call it the way you call a regular function. Instead, you either:

1. Call it from inside another `async def` function, using `await`:

   ```
   async def my_function():
       profile = await explorer.profile(df)
       return profile
   ```

2. Run the whole async call chain with `asyncio.run(...)`:

   ```
   import asyncio
   profile = asyncio.run(my_function())
   ```

`async` functions exist so that Python can run multiple I/O operations in parallel without blocking. If DataExplorer is profiling a dataset with 100 columns, it can run the per-column analyses concurrently and finish faster than sequential execution would. That is the reason for the async API.

For Module 1 you only need to know the recipe:

- Wrap calls to async functions in an `async def` wrapper of your own.
- Use `await` on every async call inside that wrapper.
- Run the wrapper once at the top level with `asyncio.run()`.

You will meet async again in Module 3 (for async inference servers) and Module 6 (for concurrent API calls to LLMs). For now, treat it as ceremonial boilerplate.

FOUNDATIONS: `try` / `except` — handling errors

When you run code that might fail, Python's `try` / `except` block lets you catch the error and do something sensible instead of crashing:

```
try:
    profile = asyncio.run(profile_economic_data())
    print("Profile complete.")
except Exception as exc:
    print(f"Profile failed: {exc}")
    raise
```

Reading this: "try to run the indented block under `try`. If any exception is raised during that block, catch it in the `except` clause. In the except clause, `exc` is the exception object; we print it and then re-raise it with `raise` so the calling code still sees the error."

You should use `try` / `except` when:

- You have a recovery action. You want to retry, fall back to a default, or log and continue.

- You want to provide a better error message than the raw exception would produce. Wrapping a cryptic `KeyError: 'foo'` with "configuration file is missing the 'foo' key — check config.yaml" is much more actionable for the user.

You should *not* use `try` / `except` to silently swallow errors. A bare `except: pass` that hides every error is a bug-incubator — it makes broken code appear to work. Always do something with the exception, even if it is just logging and re-raising.

THEORY: `DataExplorer.compare` — drift detection

`DataExplorer.compare(df_a, df_b)` profiles two DataFrames separately and then computes column-level deltas between them. Mean delta, std delta, null delta for every shared column. It returns a comparison object you can iterate over.

This is the foundation of *drift detection*. If you trained a model on last year's data and the distribution of incoming data has shifted, your model's predictions may be miscalibrated. Comparing a baseline profile (training data) with a current profile (production data) is the standard way to catch drift early. In Module 4 you will meet `DriftMonitor`, the Kailash engine dedicated to this problem; `compare` is its conceptual foundation.

The comparison output is a list of per-column delta dictionaries:

```
[
    {"column": "cpi", "mean_delta": 5.3, "std_delta": 1.2, "null_delta": 0.0},
    {"column": "employment_rate", "mean_delta": -2.1, "std_delta": 0.8, "null_delta": 0.0},
    ...
]
```

Sort by `abs(mean_delta)` to surface the columns with the biggest distribution shifts. These are the ones to investigate.

FOUNDATIONS: Spearman vs Pearson correlation

DataExplorer computes both Pearson and Spearman correlations. Pearson you already know — it measures *linear* relationships between two variables. Spearman measures *monotonic* relationships: any relationship where y always increases (or always decreases) as x increases, regardless of whether the increase is linear.

The Spearman correlation is the Pearson correlation of the *ranks* of the values. To compute it: rank each value in column A, rank each value in column B, then compute Pearson on the two rank columns. If the ranks agree (both columns rank observations in the same order), Spearman is 1. If the ranks are opposite, -1. If the ranks are unrelated, 0.

Why it matters: two variables can have a strong monotonic relationship but a weak Pearson correlation if the relationship is non-linear. CPI vs GDP is often monotonic but curvilinear. Spearman catches the relationship; Pearson does not.

The rule: if you are screening for *any* dependency, use Spearman. If you specifically need a linear relationship (for linear regression assumptions), use Pearson. DataExplorer reports both, which is the conservative choice.

The Kailash Engine: DataExplorer — full API

```python
from kailash_ml import DataExplorer
from kailash_ml.engines.data_explorer import AlertConfig

explorer = DataExplorer(alert_config=alert_config)

# Main profiling call — async
profile = await explorer.profile(df)

# Inspect results
profile.n_rows             # int
profile.n_columns          # int
profile.duplicate_count    # int
profile.duplicate_pct      # float in [0, 1]
profile.type_summary       # dict like {"numeric": 8, "string": 2}
profile.alerts             # list of alert dicts
profile.columns            # list of per-column profile objects
profile.pearson_matrix     # dict of dicts
profile.spearman_matrix    # dict of dicts

# Per-column fields (on profile.columns)
col.name                   # str
col.inferred_type          # "numeric" | "categorical" | "temporal" | "text"
```

```
col.mean                    # float (numeric only)
col.std                     # float
col.min, col.max            # floats
col.null_count, col.null_pct
col.unique_count
col.skewness, col.kurtosis  # numeric only

# Compare two datasets
comparison = await explorer.compare(df_a, df_b)
comparison["shape_comparison"]   # {"a": (rows, cols), "b": (rows, cols)}
comparison["shared_columns"]     # list of column names
comparison["column_deltas"]      # list of per-column delta dicts

# Generate HTML report
report_html = await explorer.to_html(df, title="My Dataset")
with open("report.html", "w") as f:
    f.write(report_html)

# Generate individual chart figures
vis_report = await explorer.visualize(df)
for name, fig in vis_report.figures.items():
    fig.write_html(f"{name}.html")
```

Every method that hits the async machinery is async. Every method that is synchronous (inspecting the already-computed profile object) is synchronous. The rule: if it accesses data or computes something, it is async; if it reads already-computed fields, it is synchronous.

Worked Example: Profiling Singapore Economic Indicators

Step 1: Load three messy time series

```
from __future__ import annotations

import asyncio

import polars as pl
from kailash_ml import DataExplorer
from kailash_ml.engines.data_explorer import AlertConfig

from shared import MLFPDataLoader

loader = MLFPDataLoader()

cpi = loader.load("mlfp01", "sg_cpi.csv")           # Monthly CPI
employment = loader.load("mlfp01", "sg_employment.csv")  # Quarterly labour stats
fx_rates = loader.load("mlfp01", "sg_fx_rates.csv")      # Daily SGD exchange rates

print(f"CPI: {cpi.shape}")
print(f"Employment: {employment.shape}")
print(f"FX: {fx_rates.shape}")
```

Three datasets at three different frequencies. The challenge is that they do not share a common time grain.

Step 2: Normalise the date columns

Each dataset uses a different date format. CPI uses a mix of `"01/2000"`, `"2000-02"`, and `"201108"`. Employment uses quarter strings like `"2000 Q1"`. FX rates use ISO date strings. Normalise each to a monthly date:

```
cpi = cpi.with_columns(
    pl.col("date")
    .str.replace(r"^(\d{2})/(\d{4})$", "$2-$1-01")
    .str.replace(r"^(\d{4})(\d{2})$", "$1-$2-01")
    .str.replace(r"^(\d{4})-(\d{2})$", "$1-$2-01")
    .str.to_date("%Y-%m-%d")
    .alias("date")
)
```

```
def quarter_to_date(q_str: str) -> str:
    parts = q_str.split()
    year = parts[0]
    q = int(parts[1][1])
    month = {1: "01", 2: "04", 3: "07", 4: "10"}[q]
    return f"{year}-{month}-01"


employment = employment.with_columns(
    pl.col("quarter").map_elements(quarter_to_date, return_dtype=pl.String)
    .str.to_date("%Y-%m-%d")
    .alias("date")
)

if fx_rates["date"].dtype == pl.String:
    fx_rates = fx_rates.with_columns(pl.col("date").str.to_date("%Y-%m-%d"))
```

A few new things here. `.str.replace(pattern, replacement)` applies a regex substitution to every value in a string column. The patterns with `$1` and `$2` are backreferences to the capture groups from the regex. `.map_elements(fn, return_dtype=...)` applies an arbitrary Python function to each element of a column — it is slower than a native Polars expression because Python is involved per-element, but it is the escape hatch when you need custom logic. We use it for `quarter_to_date` because writing that conversion as a pure Polars expression is awkward.

Step 3: Build a common monthly spine and align everything

```
cpi = cpi.with_columns(pl.col("date").dt.truncate("1mo").alias("month_date"))
employment = employment.with_columns(pl.col("date").dt.truncate("1mo").alias("month_date"))

date_range = pl.date_range(
    cpi["month_date"].min(),
    cpi["month_date"].max(),
    interval="1mo",
    eager=True,
)
monthly_spine = pl.DataFrame({"month_date": date_range})

employment_monthly = (
    monthly_spine.join(employment.drop("date"), on="month_date", how="left")
    .sort("month_date")
    .with_columns([
        pl.col(c).forward_fill()
        for c in employment.columns
        if c not in ("date", "month_date")
    ])
)

fx_monthly = (
    fx_rates.with_columns(pl.col("date").dt.truncate("1mo").alias("month_date"))
    .group_by("month_date")
    .agg([pl.col(c).mean() for c in fx_rates.columns if c != "date"])
    .sort("month_date")
)

economic = (
    cpi.join(employment_monthly, on="month_date", how="left", suffix="_emp")
    .join(fx_monthly, on="month_date", how="left", suffix="_fx")
    .sort("month_date")
)

print(economic.shape)
```

Three alignment techniques in one step:

- `dt.truncate("1mo")` truncates a date to the first of its month, giving every record a canonical month date.
- `pl.date_range(start, end, interval="1mo")` creates a complete sequence of monthly dates, used as a "spine" to ensure every month is present even if some source datasets have gaps.
- `.forward_fill()` replaces NULLs with the most recent non-null value — the standard way to upsample quarterly data to monthly.
- `group_by("month_date").agg([pl.col(c).mean() for c in ...])` aggregates the daily FX rates into monthly means.

This is a combination of joins from Lesson 1.4 and aggregation from Lesson 1.3 — everything you have learned so far converging on a single messy real-world problem.

Step 4: Configure AlertConfig

```
alert_config = AlertConfig(
    high_correlation_threshold=0.95,
    high_null_pct_threshold=0.10,
    constant_threshold=1,
    high_cardinality_ratio=0.95,
    skewness_threshold=3.0,
    zero_pct_threshold=0.30,
    imbalance_ratio_threshold=0.05,
    duplicate_pct_threshold=0.05,
)
```

Each number reflects the nature of economic data: structurally correlated series (so high correlation threshold), edge-null tolerance for forward-filled columns (so 10% null threshold), crisis-period outliers (so skewness threshold 3.0 ignores milder asymmetry).

Step 5: Run the profiler

```
async def profile_economic_data():
    explorer = DataExplorer(alert_config=alert_config)
    profile = await explorer.profile(economic)

    print(f"Rows: {profile.n_rows}, Columns: {profile.n_columns}")
    print(f"Duplicates: {profile.duplicate_count} ({profile.duplicate_pct:.1%})")

    print(f"\n--- Alerts ({len(profile.alerts)}) ---")
    for alert in profile.alerts:
        col = alert.get("column", alert.get("columns", "N/A"))
        print(f"[{alert['severity'].upper()}] {alert['type']}: {col} = {alert.get('value', 'N/A')}")

    return profile


profile = asyncio.run(profile_economic_data())
```

Expected: you see perhaps 5–15 alerts — some high-skew columns (crisis-period CPI spikes), some high-correlation pairs (macro indicators moving together), maybe a high-null column if one of the forward-filled series had edge gaps.

Step 6: Compare two time periods

```
async def compare_periods():
    explorer = DataExplorer(alert_config=alert_config)

    covid_cutoff = pl.date(2020, 3, 1)
    pre_covid = economic.filter(pl.col("month_date") < covid_cutoff)
    during_covid = economic.filter(pl.col("month_date") >= covid_cutoff)

    comparison = await explorer.compare(pre_covid, during_covid)

    deltas = sorted(
        comparison["column_deltas"],
        key=lambda d: abs(d.get("mean_delta", 0)),
        reverse=True,
    )
    print("Top 10 column mean shifts:")
    for d in deltas[:10]:
        print(f"  {d['column']}: mean Δ={d.get('mean_delta', 0):+,.3g}")


asyncio.run(compare_periods())
```

Columns with the largest `abs(mean_delta)` experienced the biggest distributional shift during COVID. Typically this includes CPI (inflation spike), employment variables (job market disruption), and certain FX rates (currency volatility). The comparison is a drift detector: any column that shifted dramatically is a candidate for investigation, model retraining, or alerting downstream consumers.

Step 7: Generate an HTML report

```
async def generate_report():
    explorer = DataExplorer(alert_config=alert_config)
    report_html = await explorer.to_html(economic, title="Singapore Economic Indicators")
    with open("economic_profile.html", "w") as f:
        f.write(report_html)


asyncio.run(generate_report())
```

Open `economic_profile.html` in a browser. You will see a full dashboard: summary statistics, per-column profiles, alert list, correlation heatmap, distribution plots. This is the same output you would produce manually — but in one call instead of fifty.

Step 8: Wrap everything in try/except

```
async def main():
    profile = await profile_economic_data()
    comparison = await compare_periods()
    await generate_report()
    return profile, comparison


try:
    profile, comparison = asyncio.run(main())
    print("All profiles complete.")
except Exception as exc:
    print(f"Profiling failed: {exc}")
    raise
```

Bundling the three async calls into one `main()` coroutine and running it with a single `asyncio.run` avoids the overhead of creating multiple event loops. The `try` / `except` wrapper catches any unexpected error and prints a readable message before re-raising so the traceback is still shown.

Try It Yourself

Drill 1. Run DataExplorer with the *default* AlertConfig (no custom thresholds). How many alerts do you get compared to the tuned config? Which alerts are new?

Drill 2. Tighten the correlation threshold to 0.80 and rerun. Look at the new correlation alerts. Are any of them genuinely surprising, or are they all structurally expected?

Drill 3. Compare pre-2008 data with post-2008 data (GFC cutoff). Which column has the largest `mean_delta`? Interpret the shift in domain terms.

Drill 4. Write your own alert interpreter function that takes an alert dict and returns a plain-English sentence describing the issue and a recommended fix. Use it to format the alert output.

Drill 5. Profile just the FX rates table (before merging). What alerts fire on the raw FX data that do not fire on the merged economic table? Why?

Cross-References

- **Lesson 1.8** uses DataExplorer as the first step in a full cleaning pipeline: profile → decide → clean → re-profile.
- **Module 4** introduces `DriftMonitor`, which is DataExplorer's `compare` method productionised — it runs in a streaming manner and raises alerts when incoming data drifts from the training distribution.
- **Module 2** will use profile outputs to guide feature selection: drop high-null columns, transform high-skewness ones, bin high-cardinality ones.

Reflection

You should now be able to:

- Explain what DataExplorer does and why automating profiling is useful.

- List the eight alert types and give an example remediation for each.
- Configure `AlertConfig` with domain-appropriate thresholds and justify each choice.
- Call `DataExplorer.profile()` inside an `async def` wrapper and run it with `asyncio.run`.
- Interpret alert objects (type, severity, column, value) and map them to cleaning actions.
- Use `DataExplorer.compare()` to detect distribution drift between two DataFrames.
- Generate an HTML report with `DataExplorer.to_html()`.
- Use `try` / `except` to catch and re-raise errors with added context.

Drill answers

1. Default config typically produces 3–5× more alerts because its correlation threshold (0.80) flags every pair of macro indicators. Null and skewness alerts may also increase.
2. With threshold 0.80 you will see pairs like CPI vs employment, CPI vs FX rates, etc. These are all structurally expected — the threshold is too tight for macro data.
3. Typically CPI has the largest pre-2008 vs post-2008 shift because the GFC triggered a regime change in inflation dynamics. Employment rate may also shift.
4.
```python
def interpret(alert: dict) -> str:
    t = alert["type"]
    col = alert.get("column", "N/A")
    v = alert.get("value", "N/A")
    templates = {
        "high_nulls": f"{col} has {v:.1%} missing — impute or drop",
        "high_skewness": f"{col} skew={v:.2f} — log-transform or winsorise",
        "constant": f"{col} has no variance — drop column",
        "high_correlation": f"{col} |r|={v:.2f} — consider dropping one",
    }
    return templates.get(t, f"{t} on {col}: {v}")
```
5. Raw FX data shows high-cardinality alerts (every date is unique), which are suppressed after aggregation because the month_date column has fewer unique values.

Lesson 1.8: Data Pipelines and End-to-End Project

Why This Matters

Every previous lesson taught one piece of the puzzle. Lesson 1.1 taught you to look at raw data. Lessons 1.2 and 1.3 taught you to filter and aggregate. Lesson 1.4 taught you to join. Lesson 1.5 taught you to compute trends. Lesson 1.6 taught you to plot. Lesson 1.7 taught you to profile. This lesson puts all of it together into a single pipeline: load → profile → clean → feature-engineer → preprocess → visualise → re-profile. This is the shape of almost every exploratory data analysis project you will ever do, regardless of domain.

You will work on a deliberately messy dataset — Singapore taxi trip data with GPS errors, negative fares, zero-length trips, missing coordinates, and schema drift. The mess is not accidental; it is what real data looks like before anyone has touched it. Your job is to turn the raw mess into a model-ready dataset without losing signal to the noise and without introducing bugs along the way.

This lesson also introduces `PreprocessingPipeline`, the third Kailash engine in Module 1. Where DataExplorer profiles, PreprocessingPipeline prepares: it imputes missing values, scales numerics, encodes categoricals, and splits into train/test sets. It is the bridge between raw data and the model-training steps you will meet in Module 3.

Core Concepts

FOUNDATIONS: The ETL pattern

Every data pipeline, regardless of scale or domain, has the same three stages:

- **Extract.** Get the data from somewhere. Read a file, hit an API, query a database, receive a stream.
- **Transform.** Clean it, enrich it, reshape it, compute features. This is where 80% of the work happens.
- **Load.** Put the cleaned data somewhere downstream — a file, a database, a model, a dashboard.

The acronym is ETL (extract-transform-load). Sometimes you see ELT (extract-load-transform), which is the same thing with a different ordering for architectures where you prefer to land raw data first and transform it in the warehouse. The conceptual stages are the same.

For this lesson the pipeline will be:

1. **Extract.** Load `sg_taxi_trips.parquet` from the course data loader.
2. **Profile.** Use DataExplorer to identify quality issues.
3. **Clean.** Drop impossible rows (GPS outside Singapore, negative fares, trips under 60 seconds).
4. **Engineer.** Extract hour-of-day, day-of-week, peak-period, and haversine distance.
5. **Preprocess.** Use PreprocessingPipeline to scale, encode, and split.
6. **Visualise.** Produce diagnostic charts.
7. **Re-profile.** Confirm that the cleaned data has fewer alerts than the raw data.

Each stage has a clear handoff: the output of one stage is the input of the next. You can re-run any stage independently, which is essential for iteration.

FOUNDATIONS: Null handling

Real datasets have missing values. The three decisions you have to make are:

1. How to detect a null. Polars treats `null` (the typed missing marker) distinctly from NaN (not-a-number, used for undefined float results) and from empty strings. When loading from CSV, missing values appear as nulls; when reading from some other sources, they may appear as empty strings or a sentinel value like `-999`. Always check the null count per column after loading to know what you are dealing with.

2. Whether to impute or drop. If the column is critical and the nulls are a small fraction, impute — fill with a sensible default (median for numeric, mode for categorical). If the column is not critical, drop it. If the nulls are concentrated in specific rows (and those rows are unusable for other reasons too), drop the rows.

3. Which imputation strategy. Median is the safe default for numeric columns — it is robust to outliers and does not introduce bias. Mean is acceptable for symmetric distributions. Mode is the default for categoricals. For more sophisticated imputation (KNN, model-based), see `kailash_ml.imputation` — but for Module 1 the median is enough.

The Polars methods:

- `pl.col("x").is_null()` — Boolean column, True where the value is null.
- `pl.col("x").fill_null(value)` — replace nulls with `value`.
- `pl.col("x").fill_null(strategy="forward")` — replace nulls with the previous non-null.
- `df.drop_nulls(subset=["x", "y"])` — drop rows where any of the named columns is null.

FOUNDATIONS: Domain-aware cleaning

Generic rules ("drop negative values") are not always right. Domain-aware rules are.

For a Singapore taxi dataset:

- **GPS bounding box.** Any latitude outside `[1.15, 1.47]` or longitude outside `[103.60, 104.05]` is not in Singapore — it is a GPS error, a driver in Malaysia, or a data pipeline bug. Drop.
- **Fare range.** Negative fares are impossible. Zero fares might be cancelled rides but are not useful as training data. Extreme fares (top 0.1%) are usually data errors (meter left running). Drop or cap.
- **Duration range.** Trips under 60 seconds are not real paid trips — they are meter misfires. Trips over 3 hours are implausible for a country the size of Singapore. Drop.
- **Speed.** After computing distance and duration, you can compute speed. Speeds above 120 km/h are impossible in Singapore (the expressway speed limit is 90 km/h). Drop.

Each rule embeds domain knowledge. Generic outlier detection (like "drop values more than 3σ from the mean") would not know that Singapore's GPS box is what it is, or that 120 km/h is the hard ceiling. Always encode the domain knowledge you have; never rely on generic rules alone.

FOUNDATIONS: Feature engineering for temporal data

Raw timestamps are useless to most ML models — the model cannot directly learn "Tuesday 8 AM is different from Saturday 8 AM". You have to decompose the timestamp into features the model can exploit:

- **Hour of day.** `pickup_datetime.dt.hour()`. An integer 0–23.
- **Day of week.** `pickup_datetime.dt.weekday()`. 0–6 in Polars (0 is Monday, 6 is Sunday).
- **Day of month, month of year.** For seasonal effects.
- **Is weekend.** A Boolean derived from day of week.
- **Time period** (morning peak, evening peak, off-peak, late night). A categorical derived from hour.
- **Time since epoch.** A continuous value that can capture overall trend.

The hour-of-day decomposition is crucial for demand modelling. Trip volume has a huge diurnal pattern — two peaks at morning and evening commute, a dip in the middle of the night. Without the hour feature, a model would treat 8 AM trips and 3 AM trips as equivalent. With it, the model can learn that 8 AM trips are short and expensive (commute) while 3 AM trips are longer and to entertainment districts.

FOUNDATIONS: Feature engineering for spatial data

Raw latitude and longitude are also not great features. A few more useful derivatives:

- **Haversine distance between pickup and dropoff.** The great-circle distance on a sphere — a direct measure of trip length.
- **Bearing.** The compass direction from pickup to dropoff. Sometimes predictive (airport-bound trips go east; nightlife trips cluster around Clarke Quay).
- **Distance from a reference point** (city centre, airport, MRT station). "How far is pickup from Raffles Place?" captures "is this a CBD trip?".
- **Spatial binning.** Divide the city into a grid and turn each pickup/dropoff into a grid cell ID.

The haversine distance is worth implementing directly once, because it is a recurring pattern in any geospatial pipeline:

```
_RAD = pl.lit(3.141592653589793 / 180)

taxi = taxi.with_columns(
    (
        2 * 6371  # Earth radius in km
        * (
            (
                ((pl.col("dropoff_lat") - pl.col("pickup_lat")) * _RAD / 2).sin().pow(2)
                + (pl.col("pickup_lat") * _RAD).cos()
                * (pl.col("dropoff_lat") * _RAD).cos()
                * ((pl.col("dropoff_lng") - pl.col("pickup_lng")) * _RAD / 2).sin().pow(2)
            ).sqrt().arcsin()
        )
    ).alias("haversine_km")
)
```

The formula is the haversine great-circle distance:

$$d = 2r \arcsin\left(\sqrt{ \sin^2\left(\frac{\Delta \phi}{2}\right) + \cos(\phi_1) \cos(\phi_2) \sin^2\left(\frac{\Delta \lambda}{2}\right) } \right)$$

where ϕ are latitudes, λ are longitudes (both in radians), $\Delta\phi$ and $\Delta\lambda$ are the differences, and $r = 6371$ km is Earth's mean radius. For country-scale distances like Singapore's this is accurate to within 0.5% — more than sufficient for feature engineering. For sub-metre precision you would want the more complex Vincenty formula, but you will not need it.

FOUNDATIONS: PreprocessingPipeline

`PreprocessingPipeline` automates the final steps before model training:

- **Split** the data into train and test sets.
- **Impute** remaining nulls (median for numeric, mode for categorical).
- **Scale** numeric features (standardise to mean 0, standard deviation 1).
- **Encode** categorical columns (one-hot or ordinal).
- **Infer** the task type (regression if the target is continuous, classification if categorical).

The call:

```
from kailash_ml import PreprocessingPipeline

pipeline = PreprocessingPipeline()
result = pipeline.setup(
```

```
    data=pipeline_df,
    target="fare",
    train_size=0.8,
    seed=42,
    normalize=True,
    categorical_encoding="onehot",
    imputation_strategy="median",
)

result.train_data        # Polars DataFrame, 80% of rows
result.test_data         # Polars DataFrame, 20% of rows
result.numeric_columns   # list of numeric feature columns
result.categorical_columns  # list of categorical feature columns (after encoding)
result.task_type         # "regression" or "classification"
```

The key parameter is `target` — the column you are trying to predict. The pipeline excludes it from the feature set and uses it to infer the task type. If the target is continuous (like `fare`), the task is regression. If it is categorical (like `fare_bucket`), the task is classification.

`train_size=0.8` means 80% of rows go to training and 20% to test. `seed=42` makes the split reproducible. `normalize=True` standardises numeric columns to zero mean and unit variance. `categorical_encoding="onehot"` converts each categorical column into a set of binary columns (one per unique value). `imputation_strategy="median"` fills remaining nulls with the median (for numerics) or mode (for categoricals).

The result object is what Module 3 will consume directly. No further preprocessing is needed before training — that is the whole point.

ADVANCED: Why standardise numeric features

Linear models (linear regression, logistic regression) and many neural network optimisers are sensitive to feature scale. If one feature has values in the range 0–1 and another in the range 0–1,000,000, the optimiser will effectively ignore the small-scale feature because its gradient is tiny. Standardising both features to mean 0 and std 1 puts them on equal footing.

Tree-based models (decision trees, random forests, gradient boosting) do not need standardisation because they only care about the order of values, not the magnitude. If you are training only tree models, you can skip standardisation. But if you might train any model that uses gradients (as we will in Module 3), standardising is insurance.

The formula for standardisation is:

$$z = \frac{x - \mu}{\sigma}$$

where μ is the column mean and σ is the column standard deviation, both computed on the *training* set. The same μ and σ are then applied to the test set; you do not recompute them. This is the statistical equivalent of "train/test contamination" — computing scaling parameters on the test set would leak information. PreprocessingPipeline handles this correctly by default.

Worked Example: Taxi Trip Cleaning Pipeline

Step 1: Load and inspect

```
from __future__ import annotations

import asyncio
import polars as pl
from kailash_ml import DataExplorer, ModelVisualizer, PreprocessingPipeline
from kailash_ml.engines.data_explorer import AlertConfig
from shared import MLFPDataLoader

loader = MLFPDataLoader()
taxi_raw = loader.load("mlfp01", "sg_taxi_trips.parquet")

print(f"Shape: {taxi_raw.shape}")
print(f"Columns: {taxi_raw.columns}")
print(taxi_raw.describe())

for col in taxi_raw.columns:
    nc = taxi_raw[col].null_count()
```

```
    if nc > 0:
        print(f"  {col}: {nc:,} nulls ({nc / taxi_raw.height:.1%})")
```

You should see a range of issues in the describe output: a minimum fare that is negative, latitude values outside Singapore's bounding box, trip durations of zero or in the tens of thousands of seconds. Every one of these is a red flag — but the describe output is passive. Your job is to actively decide what to do about each.

Step 2: Profile the raw data

```
async def profile_raw():
    explorer = DataExplorer(alert_config=AlertConfig(
        high_null_pct_threshold=0.02,
        skewness_threshold=2.0,
        high_cardinality_ratio=0.80,
        zero_pct_threshold=0.10,
        high_correlation_threshold=0.90,
    ))
    sample = taxi_raw.sample(n=min(200_000, taxi_raw.height), seed=42)
    profile = await explorer.profile(sample)

    print(f"Alerts: {len(profile.alerts)}")
    for alert in profile.alerts:
        print(f"  [{alert['severity'].upper()}] {alert['type']}: {alert.get('column', 'N/A')}")
    return profile


profile_raw = asyncio.run(profile_raw())
```

The alerts identify the problem columns systematically. You will likely see `high_skewness` on fare and distance, `high_nulls` on coordinate columns, possibly `duplicates` if the raw data has repeated rows.

Step 3: Domain-aware cleaning

```
SG_LAT_MIN, SG_LAT_MAX = 1.15, 1.47
SG_LNG_MIN, SG_LNG_MAX = 103.60, 104.05

taxi_clean = taxi_raw.clone()
rows_before = taxi_clean.height

# GPS filter
lat_cols = [c for c in taxi_clean.columns if "lat" in c.lower()]
lng_cols = [c for c in taxi_clean.columns if "lng" in c.lower() or "lon" in c.lower()]

for lat_col in lat_cols:
    taxi_clean = taxi_clean.filter(
        pl.col(lat_col).is_null()
        | ((pl.col(lat_col) >= SG_LAT_MIN) & (pl.col(lat_col) <= SG_LAT_MAX))
    )

for lng_col in lng_cols:
    taxi_clean = taxi_clean.filter(
        pl.col(lng_col).is_null()
        | ((pl.col(lng_col) >= SG_LNG_MIN) & (pl.col(lng_col) <= SG_LNG_MAX))
    )

# Fare filter
if "fare" in taxi_clean.columns:
    taxi_clean = taxi_clean.filter(pl.col("fare") > 0)
    fare_p999 = taxi_clean["fare"].quantile(0.999)
    taxi_clean = taxi_clean.filter(pl.col("fare") <= fare_p999)

# Duration filter
if "trip_duration_sec" in taxi_clean.columns:
    taxi_clean = taxi_clean.filter(pl.col("trip_duration_sec") > 60)
    taxi_clean = taxi_clean.filter(pl.col("trip_duration_sec") <= 10_800)

# Drop rows with missing coordinates (can't compute distance)
critical = lat_cols + lng_cols
if critical:
```

```
    taxi_clean = taxi_clean.drop_nulls(subset=critical)

print(f"Rows: {rows_before:,} -> {taxi_clean.height:,} ({taxi_clean.height / rows_before:.1%} retained)")
```

The retention rate tells you how dirty the data was. Typical: 85% or higher is good; 70% or below suggests systematic issues worth investigating upstream.

Note the use of `pl.col(lat_col).is_null() | (range check)`. The `|` keeps rows with null coordinates *and* rows with in-range coordinates; we explicitly drop null-coordinate rows in the final `drop_nulls` step. This two-stage approach lets you make the null handling decision explicit rather than silently intertwining it with the range check.

Step 4: Feature engineering

```
# Parse datetime columns
datetime_cols = [c for c in taxi_clean.columns if "time" in c.lower() or "date" in c.lower()]
for col in datetime_cols:
    if taxi_clean[col].dtype == pl.Utf8:
        taxi_clean = taxi_clean.with_columns(pl.col(col).str.to_datetime().alias(col))

pickup_col = next(
    (c for c in taxi_clean.columns if "pickup" in c.lower() and ("time" in c.lower() or "date" in
        c.lower())),
    None,
)

if pickup_col:
    taxi_clean = taxi_clean.with_columns(
        pl.col(pickup_col).dt.hour().alias("hour_of_day"),
        pl.col(pickup_col).dt.weekday().alias("day_of_week"),
        pl.col(pickup_col).dt.month().alias("month"),
        (pl.col(pickup_col).dt.weekday() >= 5).alias("is_weekend"),
    )

    taxi_clean = taxi_clean.with_columns(
        pl.when((pl.col("hour_of_day") >= 7) & (pl.col("hour_of_day") <= 9)).then(pl.lit("morning_peak"))
        .when((pl.col("hour_of_day") >= 17) & (pl.col("hour_of_day") <= 20)).then(pl.lit("evening_peak"))
        .when((pl.col("hour_of_day") >= 22) | (pl.col("hour_of_day") <= 5)).then(pl.lit("late_night"))
        .otherwise(pl.lit("off_peak"))
        .alias("time_period")
    )

# Haversine distance
if len(lat_cols) >= 2 and len(lng_cols) >= 2:
    pickup_lat, dropoff_lat = lat_cols[0], lat_cols[1]
    pickup_lng, dropoff_lng = lng_cols[0], lng_cols[1]
    _RAD = pl.lit(3.141592653589793 / 180)

    taxi_clean = taxi_clean.with_columns(
        (
            2 * 6371
            * (
                (
                    ((pl.col(dropoff_lat) - pl.col(pickup_lat)) * _RAD / 2).sin().pow(2)
                    + (pl.col(pickup_lat) * _RAD).cos()
                    * (pl.col(dropoff_lat) * _RAD).cos()
                    * ((pl.col(dropoff_lng) - pl.col(pickup_lng)) * _RAD / 2).sin().pow(2)
                ).sqrt().arcsin()
            )
        ).alias("haversine_km")
    )

    if "trip_duration_sec" in taxi_clean.columns:
        taxi_clean = taxi_clean.with_columns(
            (pl.col("haversine_km") / (pl.col("trip_duration_sec") / 3600)).alias("avg_speed_kmh")
        )
        taxi_clean = taxi_clean.filter(
            (pl.col("avg_speed_kmh") > 0) & (pl.col("avg_speed_kmh") <= 120)
        )

if "fare" in taxi_clean.columns and "haversine_km" in taxi_clean.columns:
```

```
    taxi_clean = taxi_clean.with_columns(
        (pl.col("fare") / pl.col("haversine_km")).alias("fare_per_km")
    )
```

Six new features: `hour_of_day`, `day_of_week`, `month`, `is_weekend`, `time_period`, `haversine_km`, `avg_speed_kmh`, `fare_per_km`. Each embeds a piece of domain knowledge. Each is something a model can learn from, whereas the raw pickup datetime is not.

The speed filter at the end is a derived-feature sanity check: after computing speed, any trip with an impossible speed was either a GPS error that slipped through the bounding-box filter or a duration error. Drop them.

Step 5: PreprocessingPipeline

```
exclude = set(
    ["fare", "fare_per_km"]  # target and its derivative
    + datetime_cols          # raw datetimes (use extracted features)
    + lat_cols + lng_cols     # raw coordinates (use haversine)
)
feature_cols = [
    c for c in taxi_clean.columns
    if c not in exclude and taxi_clean[c].dtype in (
        pl.Float64, pl.Float32, pl.Int64, pl.Int32, pl.Utf8, pl.Boolean, pl.Categorical
    )
]

# PreprocessingPipeline expects string columns as Categorical
for col in feature_cols:
    if taxi_clean[col].dtype == pl.Utf8:
        taxi_clean = taxi_clean.with_columns(pl.col(col).cast(pl.Categorical))

taxi_sample = taxi_clean.sample(n=min(50_000, taxi_clean.height), seed=42)
pipeline_df = taxi_sample.select(feature_cols + ["fare"])

pipeline = PreprocessingPipeline()
result = pipeline.setup(
    data=pipeline_df,
    target="fare",
    train_size=0.8,
    seed=42,
    normalize=True,
    categorical_encoding="onehot",
    imputation_strategy="median",
)

print(f"Task: {result.task_type}")
print(f"Train: {result.train_data.shape}")
print(f"Test: {result.test_data.shape}")
```

The task_type should be `"regression"` (fare is continuous). The train/test split is 80/20. Every numeric feature is now standardised. Every categorical feature (like `time_period`) is now a set of one-hot binary columns. The result is model-ready: you could hand `result.train_data` directly to a linear regression or a gradient boosting model without further preprocessing.

Step 6: Visualise key patterns

```
viz = ModelVisualizer()

# Fare distribution after cleaning
fig = viz.feature_distribution(
    values=taxi_clean["fare"].drop_nulls().to_list(),
    feature_name="Fare (S$) — Cleaned",
)
fig.write_html("taxi_fare_distribution.html")

# Hourly volume
if "hour_of_day" in taxi_clean.columns:
    hourly = taxi_clean.group_by("hour_of_day").agg(pl.len().alias("trip_count")).sort("hour_of_day")
    fig_h = viz.training_history(
        metrics={"Trip Volume": hourly["trip_count"].to_list()},
        x_label="Hour of Day",
        y_label="Number of Trips",
```

```
    )
    fig_h.write_html("taxi_hourly_volume.html")
```

The hourly volume chart shows the characteristic double-peak pattern: morning commute around 7–9 AM, evening commute around 5–8 PM, a trough at 3–5 AM. Late-night demand picks up again around 10 PM–1 AM (Clarke Quay, Orchard, nightlife). Demand is an emergent property of the city's schedule.

Step 7: Re-profile and compare

```
async def profile_clean():
    explorer = DataExplorer(alert_config=AlertConfig(
        high_null_pct_threshold=0.01,
        skewness_threshold=2.0,
    ))
    sample = taxi_clean.sample(n=min(200_000, taxi_clean.height), seed=42)
    profile = await explorer.profile(sample)

    print(f"Alerts before cleaning: {len(profile_raw.alerts)}")
    print(f"Alerts after cleaning:  {len(profile.alerts)}")

    report_html = await explorer.to_html(sample, title="Taxi Trips — Cleaned")
    with open("taxi_clean_profile.html", "w") as f:
        f.write(report_html)
    return profile


profile_clean = asyncio.run(profile_clean())
```

The alert count comparison is your quality proof. If `alerts_after < alerts_before`, the cleaning worked. A 50%+ reduction is a reasonable outcome for a single-pass clean; getting to zero usually requires multiple passes or domain-specific transformations (log of fare, log of distance) that we have not applied here.

Step 8: Pipeline summary

```
print(f"Stage 1 Load:       {taxi_raw.height:,} rows")
print(f"Stage 2 Profile:    {len(profile_raw.alerts)} alerts")
print(f"Stage 3 Clean:      {taxi_clean.height:,} rows retained")
print(f"Stage 4 Engineer:   {len([c for c in taxi_clean.columns if c not in taxi_raw.columns])} new
         features")
print(f"Stage 5 Preprocess: {result.train_data.shape[0]:,} train / {result.test_data.shape[0]:,} test")
print(f"Stage 6 Visualise:  charts saved")
print(f"Stage 7 Verify:     {len(profile_clean.alerts)} alerts remaining")
```

A single block that documents the entire pipeline's effect — raw row count, alert counts at entry and exit, clean row count, feature count, train/test split, alert reduction. This is the kind of summary you paste into a PR description or a report. Each number is concrete and auditable.

Try It Yourself

Drill 1. Change the fare percentile cap from 99.9% to 99.5% and rerun. How many more rows does the tighter cap drop? Is the distribution visibly different?

Drill 2. Add a `distance_to_raffles_place` feature: use Raffles Place as a reference point (approximate coordinates `(1.283, 103.851)`) and compute the haversine distance from each pickup to that point. Which time period has the shortest average distance to Raffles Place?

Drill 3. After PreprocessingPipeline, write a filter that keeps only the train_data rows where `hour_of_day` (in encoded form) matches a specific hour. Does it work after one-hot encoding? What does this tell you about the trade-off of encoding?

Drill 4. Modify the pipeline to use `mean` imputation instead of `median`. Compare the alert counts on the cleaned data. Did either strategy produce new alerts (indicating the imputation distorted a column)?

Drill 5. Write a function `run_pipeline(dataset_name: str)` that encapsulates the full pipeline from Step 1 to Step 8, so you can run it on any Module 1 dataset with one call. Test it on the HDB and economic datasets too.

Cross-References

- **Module 2** (Feature Engineering): will apply the feature-engineering patterns from this lesson at a larger scale using `FeatureEngineer` and store the results in a `FeatureStore`.
- **Module 3** (Supervised ML): will consume `PreprocessingPipeline.result.train_data` directly as input to `TrainingPipeline`. The pipeline boundary you built here is where training takes over.
- **Module 4** (Drift and Monitoring): will schedule `DataExplorer.compare` in a streaming loop as a drift monitor.

Reflection

You should now be able to:

- Describe the seven stages of an end-to-end data pipeline (load, profile, clean, engineer, preprocess, visualise, re-profile) and give an example of what happens at each.
- Distinguish generic from domain-aware cleaning rules and explain why both are needed.
- Engineer temporal features from a timestamp column using `.dt.hour()`, `.dt.weekday()`, `.dt.month()`.
- Engineer spatial features including the haversine distance using pure Polars expressions.
- Use `PreprocessingPipeline.setup()` to produce a model-ready train/test split from a prepared DataFrame.
- Compare pre- and post-cleaning profiles to measure cleaning effectiveness.
- Write a try/except wrapper around an async pipeline entry point.

Drill answers

1. Tightening from 99.9% to 99.5% typically drops ~0.4% more rows. The distribution's upper tail is visibly shorter; the max fare drops significantly.

2. ```
   RP_LAT, RP_LNG = 1.283, 103.851
   taxi_clean = taxi_clean.with_columns(
       (
           2 * 6371 * (
               (
                   ((RP_LAT - pl.col("pickup_lat")) * _RAD / 2).sin().pow(2)
                   + (pl.col("pickup_lat") * _RAD).cos() * pl.lit(math.cos(RP_LAT * math.pi / 180))
                   * ((RP_LNG - pl.col("pickup_lng")) * _RAD / 2).sin().pow(2)
               ).sqrt().arcsin()
           )
       ).alias("dist_to_raffles")
   )
   ```

   Morning peak usually has the shortest mean distance — commute trips are CBD-bound.
3. After one-hot encoding, `hour_of_day` is split into many binary columns (`hour_of_day_7`, `hour_of_day_8`, etc.). Filtering by a single hour requires filtering the binary column. The trade-off: one-hot makes the model's life easier but the raw data harder to query.
4. Mean imputation may introduce `high_skewness` alerts on columns whose distribution was far from normal — the mean pulls values toward a tail.

5. ```
   async def run_pipeline(dataset_name: str, target: str):
       raw = loader.load("mlfp01", dataset_name)
       explorer = DataExplorer(alert_config=AlertConfig())
       profile_raw = await explorer.profile(raw)
       # (clean, engineer, preprocess, visualise, re-profile as in the lesson)
   ```

Chapter Summary

You started this chapter not knowing what a variable was. You are ending it having run a full end-to-end data pipeline on a real messy Singapore dataset. That is a non-trivial jump. Before moving to Module 2, take five minutes to consolidate the picture.

The shape of what you learned

Module 1 has a clear internal arc. Lessons 1.1 through 1.3 are pure Python and Polars fundamentals — the alphabet of data work. Lessons 1.4 through 1.6 are the vocabulary: joins let you combine datasets, window functions let you express time-series patterns,

visualisation lets you communicate what you have found. Lessons 1.7 and 1.8 are the grammar: Kailash engines that take your hand-built patterns and run them automatically, so you can apply them at scale without re-typing.

The through-line is the idea that you should never trust a number you did not produce yourself. Every lesson teaches you one more class of numbers you can produce and trust. At the start of the chapter you were at the mercy of whatever dashboard someone else had built. At the end you can write your own.

The four Polars patterns to internalise

The four patterns below make up roughly 80% of the Polars code you will ever write. Keep them in mind; when a task maps naturally onto one of them, use it. When a task does not, pause and ask whether you are over-complicating things.

Pattern 1: Filter-select-sort. `df.filter(...).select(...).sort(...)`. Use for exploratory queries: "show me this subset of the data, these columns, in this order". Lesson 1.2.

Pattern 2: Group-aggregate. `df.group_by(col).agg(expressions)`. Use for summary tables: "one row per group with these statistics". Lessons 1.3 and 1.4.

Pattern 3: With-columns (feature engineering). `df.with_columns((expression).alias("name"))`. Use for derived values: "add a new column computed from existing ones". Lessons 1.2, 1.4, 1.5, 1.8.

Pattern 4: Window-over. `df.with_columns(pl.col("x").window_function().over("partition"))`. Use for per-row contextual features: rolling means, YoY changes, ranks within group. Lesson 1.5.

Every complex pipeline you build will combine these four patterns. You do not need more patterns for Module 1 or most of Module 2.

The three Kailash engines and their contracts

DataExplorer. Profile a DataFrame, surface quality issues as alerts, compare two DataFrames for drift, generate HTML reports. Input: a DataFrame. Output: a profile object. Use at the beginning and end of every pipeline.

PreprocessingPipeline. Impute, scale, encode, and split a DataFrame into train/test sets ready for model training. Input: a DataFrame with a designated target column. Output: a result object with train_data and test_data. Use at the end of cleaning, just before training.

ModelVisualizer. Build interactive charts (histogram, scatter, bar, heatmap, line) from Polars DataFrames. Input: a DataFrame and chart configuration. Output: a Plotly Figure. Use throughout the pipeline for exploration and reporting.

These three engines cover 90% of your data-pipeline needs in Modules 1 and 2. The other 10% you will handle in pure Polars, which is fine — Polars and the engines are designed to play together.

What Module 2 builds on

Module 2 is "Feature Engineering and Experiment Design". It assumes:

- You can write Polars filters, aggregations, and window functions without looking things up.
- You understand the difference between mean, median, variance, and standard deviation, and can explain when each is appropriate.
- You can profile a DataFrame and interpret the alerts.
- You can create train/test splits with PreprocessingPipeline.
- You can make an interactive chart of anything and export it as HTML.

If any of these feels uncertain, spend an hour on the corresponding lesson's "Try It Yourself" drills before moving on. Module 2 will not slow down to re-teach.

Module 2 introduces:

- **FeatureStore** — a versioned store for feature groups, built on the same join patterns you learned in Lesson 1.4.
- **FeatureEngineer** — automated feature generation: interactions, polynomial features, lag features, target encoding. All of these build on the `with_columns` pattern you learned in Lesson 1.5.
- **ExperimentTracker** — a log of your runs, parameters, and metrics, for reproducibility and comparison.
- **Inferential statistics.** Regression, logistic regression, ANOVA, hypothesis testing, power analysis, Bayesian updating. The statistical foundations that Module 3 will build a full ML pipeline on.

You will meet MLE, Fisher information, Bayesian priors, and hypothesis testing formally in Module 2. The intuitive versions you met in this chapter's THEORY sections are warm-up; Module 2 will do the derivations.

What you should do before moving on

Three things:

1. **Re-run the worked examples** from at least Lessons 1.3, 1.5, and 1.8. Type them out, do not copy-paste. The muscle memory matters.
2. **Run the end-to-end pipeline on a dataset of your own choice.** Any Singapore dataset from `data.gov.sg` will do. Load it, profile it, clean it, visualise it, generate a report. Fifteen minutes, and you will solidify the pattern.
3. **Tell someone what you learned.** Teaching is the best test of understanding. Pick a non-technical friend and explain the HDB flash crash story, why the median is preferable to the mean for property prices, and what a histogram reveals that a summary statistic cannot. If you can explain it, you know it.

Then take a day off. Come back to Module 2 rested. You will need it — Module 2 is longer and more formal than Module 1, and the payoff is cumulative.

Glossary

Every technical term introduced in this chapter, defined plainly.

Aggregation. The process of collapsing many rows into a single summary value, usually within groups. Mean, median, count, sum, and standard deviation are aggregations. See `group_by` and `agg`.

Alert. A structured warning from DataExplorer indicating that a column or dataset crosses a configurable quality threshold. Each alert has a type (like `high_skewness`), a severity, a column, and a value.

AlertConfig. The configuration object for DataExplorer that controls which thresholds trigger alerts. Tuning AlertConfig is domain-specific work: defaults are not appropriate for every dataset.

Async / await. Python keywords for asynchronous functions. An `async def` function returns a coroutine; `await` pauses execution until the coroutine completes. DataExplorer's `profile`, `compare`, and `to_html` methods are async.

Bar chart. A chart showing one bar per category, with bar height proportional to a value. Best for comparing a metric across categories.

Bimodal distribution. A distribution with two distinct peaks. Indicates two sub-populations mixed together. The HDB flash-crash distribution was bimodal.

Boolean. A value that is either `True` or `False`. The result of a comparison like `price > 500_000`. Python's `bool` type.

Cardinality. The number of unique values in a column. High-cardinality columns (ratio near 1) often cannot be one-hot encoded and need binning or target encoding.

Categorical. A column whose values are discrete categories (town names, flat types) rather than continuous numbers. Encoded as one-hot or ordinal for ML.

Coefficient of variation (CV). The standard deviation divided by the mean, often expressed as a percentage. A scale-invariant measure of spread.

Column. One dimension of a DataFrame. A named sequence of values all of the same type.

Correlation. A number between -1 and +1 measuring the strength and direction of a relationship between two variables. Pearson measures linear relationships; Spearman measures monotonic ones.

DataExplorer. The Kailash ML engine for automated dataset profiling.

DataFrame. A two-dimensional rectangular table of data with named columns. The fundamental data structure in this course.

Describe. A Polars method that computes basic statistics (count, null count, mean, std, min, max, quartiles) for every column in one call.

Duplicate. A row that is exactly identical to another row in the dataset. DataExplorer flags a dataset with more than the configured threshold of duplicates.

ETL. Extract-Transform-Load. The three-stage pattern for every data pipeline: get data, clean it, write it out.

Expression (Polars). A description of a computation on a column, not the computed result. Created with `pl.col("name")` and chained with methods. Evaluated only when passed to `.filter`, `.with_columns`, `.agg`, etc.

f-string. A Python string literal prefixed with `f` that supports variable interpolation and formatting inside curly braces. `f"Price: S${price:,.0f}"`.

Feature engineering. The process of creating new columns from existing ones to help a downstream ML model. Extracting hour-of-day from a timestamp is feature engineering.

Filter. Keep only the rows of a DataFrame where a given Boolean condition is true. `df.filter(pl.col("price") > 500_000)`.

Forward fill. Replacing a NULL with the most recent non-NULL value in a time-ordered series. Standard for upsampling quarterly data to monthly.

Function. A named, reusable block of code that takes parameters and returns a value. Defined with `def`.

Gestalt principles. Rules about how the human visual system groups visual elements. Proximity, similarity, closure, continuity, connection.

Group-by. Splitting a DataFrame into groups based on one or more key columns, then aggregating each group separately. The SQL `GROUP BY`.

Haversine distance. The great-circle distance between two points on a sphere, computed with the haversine formula. Used for geospatial feature engineering.

Heatmap. A grid of coloured cells where colour encodes a numeric value. Used for correlation matrices and confusion matrices.

Histogram. A chart showing the distribution of a numeric column by binning values and counting per bin.

Imputation. Filling in missing values with a substitute (median, mean, mode, or model-based estimate) so the data can be used by downstream models.

Inner join. A join that keeps only rows where the key exists in both tables. Non-matching rows are dropped from both sides.

Join. An operation that combines two tables by matching rows on a shared key. Inner, left, right, and outer are the four types.

Lazy frame. A Polars query plan that is not executed until `.collect()` is called. Enables query optimisation (predicate pushdown, projection pushdown).

Left join. A join that keeps all rows from the left table, filling right-side columns with NULL where no match is found. The most common join type in practice.

Line chart. A chart connecting data points with line segments in x-axis order. Standard for time-series data.

Mean. The arithmetic average: sum of values divided by count. Sensitive to outliers.

Median. The middle value when the data is sorted. Robust to outliers. Preferred for skewed distributions like income and prices.

Method chaining. Calling multiple methods on a DataFrame in sequence, where each method's output is the next method's input. The idiomatic style of Polars code.

Mode. The most frequently occurring value. Appropriate for categorical data, less useful for continuous data.

ModelVisualizer. The Kailash ML engine for producing interactive charts (histograms, scatter plots, bar charts, heatmaps, line charts) from Polars DataFrames.

Null. A typed marker for "missing value". Different from zero, empty string, or NaN. Polars has first-class null support.

Outlier. A value far from the bulk of the distribution. Sometimes real, sometimes an error. Always worth investigating before including in aggregations.

Parquet. A columnar file format that stores typed data efficiently. Much faster than CSV for large numeric datasets.

Polars. The DataFrame library used throughout this course. Fast, memory-efficient, polars-native (no pandas bridge), written in Rust.

PreprocessingPipeline. The Kailash ML engine that imputes, scales, encodes, and splits a DataFrame into train/test sets ready for model training.

Quantile. A percentile of a distribution. The 25th quantile (Q1) is the value below which 25% of the data falls. The median is the 50th quantile.

Rank. A column that assigns each row a position within its partition. `rank(method="ordinal", descending=True)` gives 1 to the highest value, 2 to the next, and so on.

Right skewed. A distribution with a long tail on the right. Most values are small with a few very large ones. Typical of prices, incomes, populations. Mean > median.

Rolling mean. A moving average: each value is replaced with the mean of itself and the previous $k - 1$ values. Smooths time-series noise.

Scatter plot. A chart with one point per observation, one variable on each axis. Reveals relationships, outliers, and clusters.

Schema. The list of column names and their types. Use `df.columns` and `df.dtypes` in Polars.

Select. Keep only the named columns of a DataFrame, dropping the rest. `df.select("col1", "col2")`.

Shift. Move values in a column forward or backward by a fixed number of positions, with NULLs filling the gap. Combined with `.over(partition)` for within-group shifts.

Skewness. A measure of asymmetry in a distribution. Zero for symmetric, positive for right-skewed, negative for left-skewed. DataExplorer flags columns with high absolute skewness.

Sort. Order the rows of a DataFrame by one or more columns, ascending or descending.

Spearman correlation. A rank-based correlation that captures monotonic (not necessarily linear) relationships.

Standard deviation. The square root of the variance. A measure of spread in the original units.

Standardisation. Rescaling a numeric column to have mean 0 and standard deviation 1. Done by PreprocessingPipeline when `normalize=True`.

String. A sequence of text characters, written inside quotes in Python. Polars type is `pl.String` (alias for `pl.Utf8`).

Summary statistic. A single-number summary of a column: mean, median, std, min, max, quantile.

Tuple. An ordered, immutable sequence of values in Python. `df.shape` returns a tuple like `(rows, cols)`.

Type. The kind of value a variable or column holds. Python types include `int`, `float`, `str`, `bool`. Polars column types include `Int64`, `Float64`, `String`, `Boolean`, `Date`, `Datetime`.

Variance. The mean of the squared deviations from the mean. In squared units; for original units use the standard deviation.

Window function. A computation that produces a value for each row based on a set of related rows, without collapsing the DataFrame. Rolling means, YoY changes, and ranks are window functions. See `.over()`.

YoY (year-over-year). The percentage change between a value and the same value from exactly one year earlier. A common smoothing technique for seasonal time series.

Further Reading

The following are standard references that expand on material covered in this chapter. You do not need them to complete Module 1, but they are the books and papers practitioners refer to when they want to go deeper.

On data work in general

- Wickham, Hadley, and Garrett Grolemund. *R for Data Science*. O'Reilly, 2017 (second edition 2023). The canonical beginner-to-intermediate reference for tidy data work. Uses R and the tidyverse, not Python and Polars, but the concepts translate

directly. The chapters on "Explore" and "Wrangle" are the complement to what you just learned. Free online at `r4ds.hadley.nz`.

- McKinney, Wes. *Python for Data Analysis*. O'Reilly, 2012 (third edition 2022). The pandas reference, written by pandas' author. We do not use pandas in this course, but the conceptual material on aggregation, joins, and reshaping is the same. Chapter 9 ("Data Aggregation and Group Operations") is particularly relevant to Lessons 1.3 and 1.5.

On Polars specifically

- Polars documentation, `pola.rs`. The official reference, well-maintained and increasingly comprehensive. The "User Guide" sections on expressions, lazy evaluation, and window functions are excellent.

- Vink, Ritchie. *Polars: The Definitive Guide*. O'Reilly, 2024. The first book dedicated to Polars, co-authored by Polars' creator. Covers performance, the query engine, and advanced patterns beyond what this textbook touches.

On visualisation

- Tufte, Edward. *The Visual Display of Quantitative Information*. Graphics Press, 1983 (second edition 2001). The foundational book on chart design, and the source of most of the principles in Lesson 1.6. The chapter on "chartjunk" and the "lie factor" are essential reading for anyone producing charts for others.

- Cleveland, William. *The Elements of Graphing Data*. Hobart Press, 1985. The empirical complement to Tufte: experiments on what the human visual system can and cannot parse accurately. The chapter on the "cycle plot" for seasonal data is particularly useful for time-series analysts.

- Wilke, Claus. *Fundamentals of Data Visualization*. O'Reilly, 2019. Modern, well-illustrated, and free online at `clauswilke.com/dataviz/`. The chapters on "Common pitfalls of color use" and "Handling overlapping points" are directly applicable to Lesson 1.6's scatter-plot work.

- Plotly documentation, `plotly.com/python`. The reference for Plotly, which is what ModelVisualizer uses under the hood. Most ModelVisualizer customisations are just Plotly method calls on the returned Figure.

On statistics (background for Module 2)

- Wasserman, Larry. *All of Statistics*. Springer, 2004. A compact, technically rigorous introduction to modern statistics for people with a mathematics background. Covers probability, estimation, hypothesis testing, Bayesian inference, and bootstrap — all in about 400 pages. Will be useful in Module 2.

- Efron, Bradley, and Trevor Hastie. *Computer Age Statistical Inference*. Cambridge, 2016. A history of statistics from classical methods to modern machine learning, with working code examples. The chapter on the bootstrap is particularly elegant. Free online at `web.stanford.edu/~hastie/CASI/`.

- Anscombe, Francis. "Graphs in Statistical Analysis." *The American Statistician*, 1973. The original Anscombe's quartet paper. Four pages, and worth reading in full.

On data quality and profiling

- Redman, Thomas. *Data Driven: Profiting from Your Most Important Business Asset*. Harvard Business Review Press, 2008. The business case for data quality, written for managers but with technical depth. Chapter 5 ("The data quality problem is bigger than you think") is especially relevant to the motivation for Lesson 1.7.

- Sadowski, Caitlin, and Yarden Katz. *Data Quality for the Numerate*. O'Reilly (forthcoming). The practical manual — how to build data quality into a pipeline instead of bolting it on afterward. Uses concepts that map directly to DataExplorer's alert categories.

On Singapore-specific data sources

- `data.gov.sg` — the Singapore government's open data portal. The HDB resale dataset, economic indicators, weather, taxi trips, and many more Singapore datasets are published here for free download. All the course datasets in Module 1 originate here (or are synthetic extensions of real data).

- `onemap.gov.sg/apidocs` — the OneMap API documentation, referenced briefly in the deck as an example of REST data extraction. Provides geocoding, routing, and map tile services for Singapore. If you do Drill 5 of Lesson 1.8 with a new dataset, OneMap is often the fastest way to enrich addresses.

- Monetary Authority of Singapore (MAS) statistics portal. Daily exchange rates, monetary aggregates, and financial stability indicators — the source of the FX data in Lesson 1.7.

Papers on the specific topics this chapter skimmed

- Tukey, John. *Exploratory Data Analysis*. Addison-Wesley, 1977. The book that named and popularised the field. Tukey's stem-and-leaf plots are out of fashion but his philosophy — "the greatest value of a picture is when it forces us to notice what we never expected to see" — is the animating principle of this entire chapter.
- Huber, Peter. *Robust Statistics*. Wiley, 1981 (second edition with Ronchetti 2009). The rigorous reference for robust statistics. If you want to know why the MAD is multiplied by 1.4826, this is where the derivation lives.
- Pearson, Karl. "Notes on the History of Correlation." *Biometrika*, 1920. The history of the correlation coefficient, written by its co-inventor. Short, readable, and useful context.

You have finished the reading for Module 1. The exercises in `modules/mlfp01/` are next. Good luck.